CARS-SF

Design-Defense Handbook for the RPAC and Thesis Panel

Context-Aware Rural Service Slicing Framework with autonomous self-healing

Prepared for the thesis presenter

October 2026 · numbers generated from `results/` by `experiments/defense_numbers.py`

How to use this handbook

Every design question is answered in five parts: **Why** (the requirement), **Choice** (what was built), **Alternative** (what else was possible), **Trade-off** (what was gained and given up) and **Evidence**. Each piece of evidence carries a *maturity badge*. The four levels are different kinds of evidence, never interchangeable:

L1 implemented the code exists L2 unit-tested isolated tests pass L3 validated (emulation) works with the real connected components *in the stated environment* L4 real network tested on real infrastructure.

No CARS-SF claim is at L4. The strongest evidence is L3, and the environment is always named: Mininet with Linux bridges (sudo-free harness), Mininet with a real Open vSwitch (privileged container), or the Open5GS 5G core with a UERANSIM-simulated RAN. When an examiner asks "does it work?", answer with the level and the environment, never just "yes".

Honest limit marks a weakness to volunteer before it is found; **If pressed** gives a fallback line for hostile follow-ups. All numbers are macros generated from the result tables, so they cannot drift from the data.

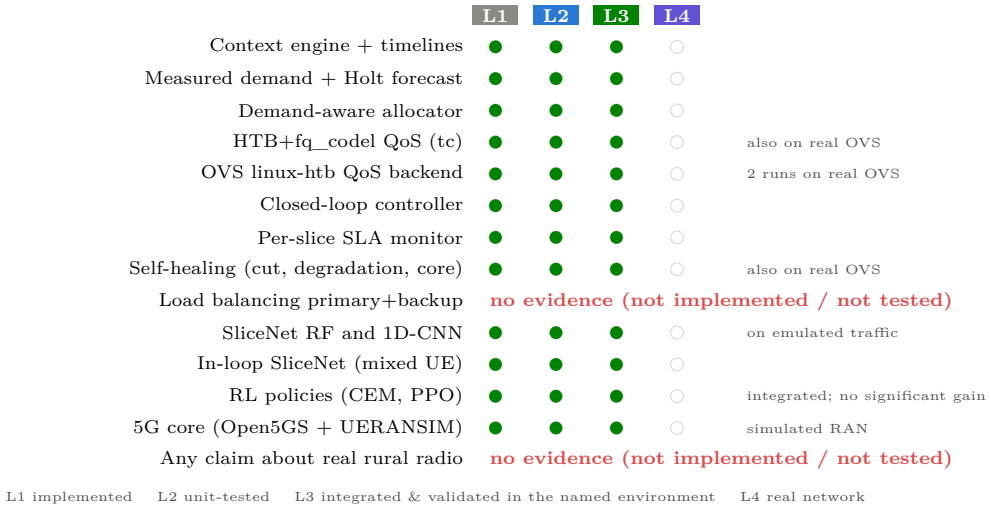


Figure 1: Evidence maturity per component. Nothing reaches L4: every network result comes from emulation (Mininet with Linux bridges or a real Open vSwitch), or from the Open5GS 5G core with a simulated RAN.

The ten numbers to know cold

1. 1,550 emulated runs (200 in the main study), all with packet-matched measurements; median validation score 84/100.
2. Main study: weighted satisfaction lies between 0.944 and 0.955 across all seven schedulers. **No allocation policy is significantly better** after Bonferroni correction (smallest $p = 0.018$).
3. **Slicing vs none:** healthcare SLA-met share rises by 0.117 ($p = 0.0010$, $n = 25$, survives Bonferroni). This is an exploratory finding, not a pre-registered hypothesis.
4. Ablation (school): removing HTB borrowing raises healthcare p95 from 10.0 to 165.0 ms. Surplus redistribution matters only once borrowing is disabled (0.789 → 0.651). Removing AQM *raises* throughput satisfaction (0.903) but general-internet p95 explodes (160 → 1433 ms). Removing context awareness changes nothing measurable.
5. Fiber cut: core-bound outage averages 0.165 s with healing vs 15.126 s without. On real OVS, recovery takes 0.094 s.
6. Backhaul degradation: the predictive detector switched over before the cut in 100.0% of runs (outage 0.019 s vs 0.214 s reactive-only).
7. SliceNet on measured traffic: RF macro-F1 0.947, 1D-CNN 0.858, on held-out runs×UEs (4,697 windows, 18 runs).
8. Synthetic RSNSD scores macro-F1 0.104 zero-shot on measured flows (near chance), 0.898 with half the measured groups added.
9. RL: PPO vs the rule-based policy differs by $\Delta = -0.0042$ ($p = 0.360$). **No significant gain.**
10. Real 5G core (10 Mb/s UPF, disaster): healthcare p95 349.0 ms with CARS-SF vs 797.9 ms without. No gain in school, where healthcare is uncontended (4.5 vs 5.4 ms), so H8 as pre-registered is **not supported**.

The claims boundary: what the thesis does and does not claim

Claims (L3, emulation):
- service-centric slices enforced at a shared rural bottleneck measurably protect critical-service SLAs;
- the closed loop detects and heals backhaul and core faults without loops, in sub-second time on real OVS;
- service classes can be recognised from traffic shape and used to slice a multi-service device.

Does not claim:
- radio-layer QoS or any real-deployment performance;
- that the demand-aware allocator beats simple slicing on average (it does not, significantly);
- that RL adds value here (it does not);
- that context awareness itself improves outcomes (the ablation shows no measurable effect);
- that RSNSD represents real rural traffic (the transfer study says it does not);
- load balancing across primary and backup (not implemented).

Contents

I The problem statement and research motivation 5

1 What exactly is the problem? 5

2 Why rural networks? 7

3 Why these application domains? 8

4 Research questions and hypotheses 9

II Literature review, novelty and research contribution 11

5 Positioning against existing work 11

6 Research contribution and attribution 13

III Requirements, architecture and end-to-end data flow 14

7 Requirements engineering 14

8 High-level architecture 16

9 End-to-end execution 18

10 Architectural alternatives and trade-offs 20

IV Context modelling and application traffic generation 21

11 Context engine 21

12 Traffic and workload modelling 23

V Mininet, SDN, Open vSwitch and networking foundations 25

13 Why Mininet and Open vSwitch? 25

14 Topology and resource modelling 27

15 TCP/IP, transport behaviour and traffic control 28

VI Telemetry, monitoring and feature engineering 30

16 Telemetry collection 30

17 Feature extraction 32

VII RSNSD: dataset design, generation and validation 33

18 Dataset motivation and schema 34

19 Dataset generation and label quality 35

20 Data validation and leakage 36

VIII SliceNet and the machine-learning pipeline 38

21 Why machine learning? 38

22 Training and model evaluation 39

23 Runtime inference and deployment 41

IX Context-aware priority, allocation and QoS enforcement 42

24 Priority and policy design 42

25 Resource allocation 44

26 QoS and SLA enforcement 45

X Monitoring, fault detection and self-healing 47

27 Fault detection 47

28 Recovery planning and action selection 49

29 The closed-loop control system 51

30 Fault tolerance of the controller itself 52

XI Reinforcement learning and intelligent scheduling 53

31 Does the RL component solve a real problem? 53

32 Training, safety and integration 55

XII 5G architecture and real-network integration 56

33 Understanding the 5G system 56

34 Open5GS, UERANSIM and deployment claims 58

XIII Experiments, statistics and reproducibility 59

35 Experimental design 59

36 Validity of the measurement campaign 60

37 Results by study 61

XIV Critical evaluation, limitations and the hostile round 62

38 Critical evaluation 62

39 The hostile round: one-line answers 63

XV QoS-targeted control: making the slices meet their SLAs on the go 64

40 Diagnosis: why the audited system could not meet the goal 64

41 Module A: QoS visibility 65

42 Module B: allocate for QoS targets, per direction 65

43 Module C: react within a second, act ahead when possible 66

44 Module D: residual reinforcement learning 67

45 Module E: SliceNet as the classifier for every flow 67

46 Module F: evaluation in contended regimes 68

47 How the evaluation was kept honest 69

XVI Final evaluation: closing the research objective 70

48 Audit and baseline 70

49 Claims, tested once 70

50 Real Open vSwitch 71

51 Verdict 72

A Evidence index 72

B Live demonstration commands 72

C Methodology change log 73

D Glossary 73

Part I
The problem statement and research motivation

1 What exactly is the problem?

Q1. Define the CARS-SF problem in one precise sentence.

When several socially critical services share one scarce, failure-prone rural backhaul, decide continuously how much of that bottleneck each *service* receives (from the operating context and measured demand), enforce that decision in the data plane, and restore connectivity on its own when the backhaul degrades or fails, while keeping critical-service SLAs.

Q2. What does CARS-SF stand for, and what does each part of the name mean operationally?

Context-**A**ware **R**ural **S**ervice **S**licing **F**ramework.

- *Context-aware*: the priority matrix is chosen by `ContextInferenceEngine` from clock, season, alerts and admin override (`context/engine.py`).
- *Rural*: one shared uplink is the only contention point, with a standby backup backhaul (Fig. 4).
- *Service slicing*: four classes (healthcare, education, agriculture, general), each with its own HTB class, floor and SLA (`registries/slices.yaml`), not eMBB/URLLC/mMTC.
- *Framework*: swappable stages behind protocols (`core/interfaces.py`), composed from `registries/schedulers.yaml`.

The thesis title also says "autonomous self-healing": the `SelfHealingEngine`.

Q3. Is the primary research problem rural connectivity, service-aware allocation, context awareness, slicing, self-healing, or their integration?

The integration: a single closed loop in which context, measured demand, service slicing and self-healing act on the *same* bottleneck through shared constraints. The healer constrains the allocator rather than overwriting it (`apply_constraints`). Rural connectivity is the setting, not the contribution.

Honest limit. Integration is a weaker novelty claim than a new algorithm. Defend it with what integration *enabled* that the parts alone cannot: backup-capacity ceilings that prevent bufferbloat after failover, and congestion-aware prediction that prevents false failover (both found and fixed in this work).

Q4. Which is the central contribution, and which are supporting engineering components?

Central:

- (C1) service-centric slicing with a context-driven, demand-aware allocator enforced at the rural bottleneck;
- (C2) a loop-free self-healing state machine with predictive switchover, integrated with allocation;
- (C3) a packet-matched measurement methodology that makes C1 and C2 evaluable.

Supporting: traffic models, harness, matrix runner, 5G deployment. **Secondary research results:** SliceNet on measured traffic and the synthetic-to-measured transfer study. RL is an evaluated alternative with a *negative* result.

Q5. What concrete failure or inefficiency exists in a conventional rural network that CARS-SF addresses?

- (a) A single FIFO uplink lets bulk downloads and video inflate everyone's queue. In the main study, "no slicing" meets the healthcare SLA in 0.773 of windows vs 0.890 with slicing.
- (b) Static per-technology shares (3GPP-style) ignore that a disaster changes *which* service matters.
- (c) A fiber cut with no automated failover means an outage of the order of the repair time. Without self-healing, the injected cut cost 15.126 s of core-bound outage in each run (until the scripted repair).

Q6. What do you mean by resilient connectivity? Which measurable properties constitute resilience?

Six measured properties:

- time to detect (MTTD) and time to act (control-plane MTTR);
- service outage: the longest delivery gap of affected flows, from matched packets;
- packets lost during failover;
- SLA compliance while degraded;
- correct localisation (link vs core switch);
- safe fail-back with no loops or flapping.

All six are produced per run (`healer.resilience_summary`, `analysis/metrics.outage_metrics`). Availability over months is *not* measured.

Q7. What do you mean by context-aware? What qualifies as context, and how does it change a decision?

Context is information *external* to the traffic that changes social priority:

- time of day and weekday (school hours);

- season (harvest months);
- alerts (flood, fire, medical emergency, fiber cut);
- an administrative override.

It selects the priority matrix (e.g. disaster: healthcare weight 10 vs 1), and in timeline runs it also changes the offered traffic. It does *not* change floors or SLA targets.

Evidence. timeline runs switch contexts live; unit tests check the precedence rules (`tests/test_context.py`). **L3 validated (emulation)**

Q8. What do you mean by self-healing? How does it differ from detecting a fault, raising an alert or manual repair?

Self-healing closes the loop: it detects (link watcher, telemetry, prediction), diagnoses (link vs core switch), acts on the network (quarantine, backup activation, MAC flush), adapts the allocation (constraints, ceilings), and reverts safely once the fault has cleared (hold-down + OAM check). An alert stops at detection; manual repair needs a human.

Honest limit. Post-action verification is partial. Escalation to DEGRADED happens when the core is still unreachable after failover, but there is no generic "verify KPI, otherwise roll back" stage for every action (see Q514).

Q9. What is the exact system boundary of CARS-SF?

Inside:

- the control loop (`core/`, `slicing/`, `self_healing/`, `scheduler/`, `classification/online.py`);
- the qdiscs and filters on the two ends of the rural uplink;
- the s_edge-side backhaul ports.

Outside:

- end hosts and their congestion control;
- the core and edge servers;
- the radio access network;
- physical repair.

The traffic generators, topology and capture are *experimental apparatus*, not part of the framework.

Q10. What does CARS-SF control, what does it observe, and what is outside its control?

Controls:

- HTB rates and ceilings per class, in both directions;
- classification rules (host, app, per-flow);
- the admin state of the s_edge backhaul and backup ports;
- the forwarding database (MAC flush).

Observes:

- interface counters and HTB class stats;
- RTT probes and link state;
- OVS stats (on OVS);
- packet captures (post-run, plus live for the mixed UE).

Outside its control: offered load, TCP behaviour, the far end of the backhaul, the radio.

Q11. Is CARS-SF a protocol, a management framework, a decision engine, a dataset, an ML model, or a combination?

Most defensible: *a network-management framework with a closed-loop decision engine*, plus an evaluation methodology. It is not a protocol and does not change packet formats. Datasets (RSNSD, the measured windows) and models (SliceNet, PPO) are *artefacts* it produces and consumes.

Q12. What is the unit of decision-making?

The **service slice** (four HTB classes) for allocation, re-planned every 5 s. The **flow** (5-tuple) only for in-loop classification of a mixed UE. The **link** for self-healing. There are no per-packet decisions: the kernel's HTB and fq_codel handle packets.

Q13. What is the unit of failure detection and recovery?

Detection: a link (carrier transition on `s_edge-bh`), reachability per tier (edge vs core RTT probes), and the per-slice SLA. Recovery: path-level (primary ↔ backup backhaul) plus slice-level constraints.

Q14. What does the system optimise, and what constraints must never be violated?

It maximises priority-weighted demand satisfaction $U = \sum_s w_s \min(1, x_s/d_s) / \sum_s w_s$, approximately and greedily (Q432).
Hard constraints:

- $\sum_s x_s \le C$;
- $x_s \ge \min(f_s, d_s)$ for the floors f (healthcare 5, education 2, agriculture 1, general 1 Mb/s);
- the L2 domain is never looped;
- learned policies can only rescale priorities, never bypass floors or constraints.

Q15. Maximise average performance, or guarantee minimum quality for critical applications?

Minimum guarantees first: floors are allocated before any priority share, and HTB `rate` is a reservation. Above the floors it maximises weighted satisfaction.

Honest limit. The floors are bandwidth guarantees, not latency guarantees. Latency is protected empirically (fq_codel + priority), not provably (Q461).

Q16. Improving average performance vs preserving critical connectivity in an emergency?

The average can stay flat while the critical service collapses. In the main study, weighted satisfaction is statistically indistinguishable across schedulers (0.944–0.955), yet healthcare SLA compliance differs by 0.117 between slicing and no slicing. CARS-SF is judged on the second kind of metric.

Q17. Which user-facing outcomes improve if CARS-SF works?

- Telemedicine and ECG streams keep low delay and loss under load (healthcare p95 ≈25.7 ms in the main study).
- Classrooms and downloads get the surplus instead of a fixed share.
- A fiber cut is a sub-second blip instead of a minutes-long outage.
- A shared multi-service device (a village kiosk) still gets per-service treatment.

Q18. What is the smallest implementation that could demonstrate the central hypothesis?

One bottleneck, two classes (critical vs bulk), HTB+fq_codel, a static context switch and an injected link cut with a standby path. Measure the critical SLA and outage with and without. The `ablation` and `resilience` studies are essentially this, scaled up.

Q19. What functionality is unnecessary for proving that hypothesis?

RL policies, SliceNet, the 5G deployment, the RSNSD generator, PPO calibration and the OVS QoS backend. They answer *secondary* questions (RQ3, RQ5, RQ6, RQ8). The thesis presents them as such.

Q20. What is explicitly outside the scope of the thesis?

- radio scheduling and handover;
- real field deployment;
- multi-site topologies with several bottlenecks;
- multipath load balancing;
- controller high availability;
- security of context feeds;
- energy and power-failure modelling;
- real-trace traffic.

2 Why rural networks?

Q21. Why rural connectivity rather than generic enterprise networking?

Rural sites typically combine one scarce shared backhaul, little redundancy, long repair times and a few socially critical services (clinic, school, farm cooperative) whose importance swings with events (harvest, flood). Enterprise networks usually have redundant links and overprovisioning, so allocation under scarcity and autonomous healing matter less there.

Honest limit. This characterisation is argued from the problem framing and literature, not from rural measurements made in this thesis.

Q22. Which rural characteristics make the problem technically different?

- A single contention point carries every service.
- Demand composition changes with context more than with load.
- The backhaul is long and fragile, and any backup has lower capacity.
- Repair takes long, so autonomy matters.

Q23. Limited bandwidth, intermittent backhaul, power, sparse infrastructure, latency or poor redundancy?

Represented: limited bandwidth (100 Mb/s shared, swept 50–150), backhaul failure and degradation, poor redundancy (a single standby backup at half capacity), and long backhaul delay (20 ms primary, 40 ms backup). Not represented: power instability, sparse radio coverage.

Q24. Which constraints are directly represented in the testbed?

- bottleneck capacity (HTB root = 100 Mb/s);
- propagation delays (netem on destination-facing ports);
- a backup path of 50 Mb/s and 40 ms;
- failures: fiber cut, gradual degradation (delay +15 ms and loss +0.3% per tick), core switch failure.

See `topology/rural_topology.py` and `self_healing/injector.py`.

Q25. Which are only assumed or discussed conceptually?

Power failures, radio coverage gaps, weather-driven radio degradation, the cost of a backup link, multi-village topologies and actual rural traffic mixes. The radio fields in RSNSD are scenario-injected and excluded from ML.

Q26. What evidence supports heterogeneous, competing requirements of rural services?

The application registry assigns per-application SLAs (e.g. ECG 20 ms, telemedicine 50 ms, bulk library 300 ms, irrigation 500 ms) and distinct traffic models.

Honest limit. These are literature-typical values chosen by design (`registries/applications.yaml`), not measured in rural deployments. The heterogeneity is *assumed* and then modelled.

Q27. Rural network vs low-capacity network: interchangeable in your formulation?

Not quite. A low-capacity network explains the scarcity. "Rural" adds a single fragile backhaul with a weaker backup, socially prioritised services, and context swings. The allocation results would carry over to any single-bottleneck network; the resilience results depend on the rural assumptions.

Q28. Why not solve it by increasing bandwidth?

- Capacity is the expensive, slow resource in rural areas.
- The capacity sweep (`sensitivity_capacity`) shows the proposed scheduler's advantage is modest at every capacity. At 50 Mb/s in disaster it protects healthcare better (SLA share 0.517 vs 0.503), without higher utility.
- More bandwidth does not fix a cut fiber.

Q29. Why might adding bandwidth fail for intermittent or physical failures?

A cut link has zero capacity whatever its nominal size. Recovery needs a *different* path, plus fast detection and safe switching. Bandwidth helps only the backup's quality while on backup.

Q30. No alternate route, no spare capacity, no radio coverage?

Then CARS-SF can only *degrade gracefully*: it detects the fault, gives the surviving capacity (e.g. edge-hosted services) to critical slices, and reports. This is exactly the core-switch-failure case: the outage of core-hosted services lasts until repair (15.118 s, the same as without healing), while edge services keep running.

Q31. Can software restore connectivity when the physical infrastructure is completely unavailable?

No. Software can only use surviving physical paths faster and more safely, and re-prioritise. The thesis states this explicitly through the core-switch result (Fig. 13).

Q32. What minimum infrastructure does CARS-SF assume?

- one Linux-capable aggregation switch at each end of the bottleneck (tc / HTB);
- a standby backhaul for healing;
- the ability to probe edge and core servers;
- UEs whose addresses map to institutions (or SliceNet for mixed devices).

Q33. How do you model limited resources without making the network so constrained that your method looks artificially good?

- Design demands run from 80 Mb/s (normal) to 145 Mb/s (school), i.e. 0.8–1.45 of capacity, not extreme overload.
- Capacity is swept from 50 to 150 Mb/s and load from 0.5× to 1.5×.
- The result is reported honestly: at 0.5× load the proposed scheduler is *not* better than static (0.969 vs 0.976).

Q34. How representative are the topologies of actual rural deployments?

Structurally plausible (access aggregation, a long backhaul, an edge MEC, a core), but parameterised, not surveyed. It is one topology with one bottleneck. Treat the results as properties of this class of topology.

Q35. One backhaul village vs a region with multiple community gateways?

With multiple gateways you get several bottlenecks and real path choice, so allocation becomes a multi-link problem (per-path HTB, routing). Healing gains load balancing (ECMP or OpenFlow groups instead of a standby link), and the controller probably becomes hierarchical (per-village agents). See Q130 and Q156.

Q36. Why are rural networks the right setting, not just a motivating example?

Because the three mechanisms matter most exactly there: scarcity (allocation matters only above 1× load in our sweep), context swings (disaster weights), and fragile single backhauls with long repair (self-healing).

3 Why these application domains?

Q37. Why e-healthcare, e-education and smart agriculture?

They are the canonical rural public-service pillars, and they span the QoS space: latency-critical low-rate (ECG, voice), high-throughput elastic (classroom video, library downloads), and sparse periodic IoT (soil, weather). General internet is the best-effort fourth class.

Q38. What are the distinctive QoS requirements of each domain?

From `registries/applications.yaml`:

- Healthcare: one-way latency 10–50 ms, loss ≤1%, jitter ≤10 ms.
- Education: 150–300 ms, high throughput.
- Agriculture: 250–1000 ms, low rate, loss tolerant.
- General: 300–500 ms, best effort.

Slice-level targets are in `registries/slices.yaml`.

Q39. Why is a domain label alone insufficient to determine the priority of every flow?

A domain contains apps with different needs (ECG 0.08 Mb/s at 20 ms vs telemedicine video at 4 Mb/s with 50 ms), and context changes urgency. CARS-SF therefore uses domain → slice for *bandwidth*, per-app SLAs for *evaluation*, and fq_codel *within* a slice so that its sparse flows are not delayed by its bulk flows.

Honest limit. There is no per-app priority within a slice.

Q40. How does emergency healthcare traffic differ from routine monitoring?

In the models: emergency voice (talkspurt VoIP) and ambulance on/off bursts vs periodic health-monitoring reports. Under the `medical_emergency` context the healthcare weight rises to 15 and its demand to 95 Mb/s. In the network the distinction is by context, not by flow.

Q41. Is a live classroom always more important than agricultural telemetry?

No. The school context weights education 5 vs agriculture 2, the harvest context the opposite (3 vs 5), and floors keep sparse IoT alive in all cases. Within a slice, fq_codel keeps sparse telemetry from waiting behind video.

Q42. How do you distinguish an application needing throughput from one needing low latency?

By declared SLA (registry) and by mechanism: throughput comes from HTB rate and surplus, latency from AQM (fq_codel) plus HTB priority. The ablation shows that removing borrowing raises school healthcare p95 from 10.0 to 165.0 ms.

Q43. Can two applications in the same domain have conflicting QoS requirements?

Yes: ECG (strict latency, tiny rate) vs telemedicine video (bursty, larger). They share the healthcare class. fq_codel's per-flow queues isolate the sparse ECG flow from video bursts, but bandwidth is shared. Per-app sub-classes are a possible extension.

Q44. How should CARS-SF respond when several critical services compete for the same bottleneck?

Floors first, then priority-weighted shares capped at demand, then the surplus by priority among unsatisfied slices (Fig. 8). If even the floors exceed capacity, each floor is clipped to $C/4$ (Q427).

Q45. What policy prevents indefinite starvation of best-effort traffic?

Three mechanisms:

- a general-class floor of 1 Mb/s;
- work-conserving HTB (best effort borrows idle capacity);
- healing boosts that expire after 6 ticks.

Honest limit. 1 Mb/s keeps it alive but satisfies little. In disaster runs the general slice's satisfaction is low by design.

Q46. How would you extend the framework to industrial IoT, disaster response or public safety?

Add a slice entry (class, floor, SLA) in `slices.yaml`, applications in `applications.yaml`, weights per context in `scenarios.yaml`, and retrain SliceNet. QoS class tables, the allocator and monitoring are data-driven.

Honest limit. The service list is hard-coded as a 4-tuple in several modules (`SERVICE_CLASSES`). A fifth class needs those constants generalised (Q159).

Q47. Were the domains chosen for real requirements, distinct traffic, or easy classification?

For requirements and policy relevance. Distinct traffic patterns follow from the applications. Classification was deliberately made *harder* by excluding ports and DSCP from features. Even so, the traffic models were written by us, so their separability is partly designed (Q334).

4 Research questions and hypotheses

Q48. State the primary research question in a testable form.

On a shared rural bottleneck under the design contexts, does service-centric slicing (context-driven, demand-aware, with self-healing) achieve higher priority-weighted demand satisfaction and critical-service SLA compliance than technology-centric (3GPP), static or no slicing? (RQ1, `docs/thesis/research_design.md`.)

Q49. State the null hypothesis for the main evaluation.

H_0: for each baseline b, the paired difference in weighted utility between the proposed scheduler and b has median zero across (scenario, seed) pairs (Wilcoxon signed-rank).

Q50. State the alternative hypothesis.

H_1: the median paired difference is positive (proposed better). It was pre-registered as H1a. **Result: not supported.** The proposed scheduler is not significantly better than any baseline in weighted utility. The only nominally significant difference ($p = 0.018$) is in the other direction (PPO marginally ahead; see the RL part), and none survives Bonferroni correction.

Q51. What is your independent variable?

The scheduler (seven levels), crossed with context (five levels). In the other studies the IVs are the ablation variant, capacity, load, UEs per service, healing variant and classification mode.

Q52. What are your dependent variables?

Weighted demand satisfaction, per-slice demand satisfaction, SLA-met share, p95/p99 one-way delay, jitter, loss, Jain fairness, outage, failover loss, MTTD/MTTR, and macro-F1.

Q53. Which variables must be held constant?

Topology and capacities, traffic models and seeds (identical across schedulers for a given seed), run duration (40 s), window length (5 s), UEs per service (2), host load (4 parallel workers, pacing without busy-waiting) and code version (the git hash is recorded per run).

Q54. Which baseline would falsify the claim that context awareness is useful?

The `no_context` ablation, which uses the normal priority matrix in every context. Context awareness is falsified if it performs as well as `full`. Result: school utility 0.861 vs 0.865 (full); disaster 0.963 vs 0.962. **By this test, context awareness is falsified in our setting**: demand-capping plus HTB borrowing absorb the effect of the weights.

Honest limit. Say this plainly. Context still decides *which* slice wins under scarcity, but its measured contribution to outcomes here is nil.

Q55. What result would show self-healing adds nothing beyond static slicing?

Equal outage with and without healing. Observed: fiber cut 0.165 vs 15.126 s; degradation 0.019 vs 15.310 s. For a core switch failure, healing indeed adds nothing to the outage (15.118 vs 15.404 s), and we report that.

Q56. What result would show the ML classifier is unnecessary?

If address-based (host) slicing served the mixed UE as well as in-loop SliceNet. Observed:

- disaster: equal (1.000 vs 1.000);
- school: SliceNet better (0.958 vs 0.798).

H5b required a gain in both, so it is **not supported**. More fundamentally, for single-institution UEs, address rules suffice and SliceNet *is* unnecessary. Its value is confined to multi-service devices.

Q57. What would count as a negative result, and how would you report it?

Any pre-registered hypothesis whose test fails. The report generator marks verdicts mechanically (`experiments/report.py`). In the final predeclared evaluation (Part XVI), C1, C2, C3b and C5–C8 are *unsupported*; earlier, H1a, H1b, H5b, H7, H9 and H11–H13 were reported as *not supported* in `docs/thesis/results.md`, next to the supported ones.

Q58. Which claims are hypotheses, which are implementation goals, and which are demonstrated findings?

- **Demonstrated** (L3):
  - slicing vs none for healthcare SLA (exploratory, significant);
  - self-healing outage reduction (H4a, H4b);
  - SliceNet F1 $\geq$0.9 on held-out runs (H5a);
  - poor synthetic transfer (H6);
  - self-healing reduces core-bound outage against no healing (H10, added during the audit, not pre-registered);
  - 5G-core benefit under contention (H8 is marked supported by its directional criterion, but only the disaster effect is substantive: the school difference is about 1 ms with $n = 3$ and changed sign between the pre- and post-audit runs).
- **Not supported**: H1a, H1b, H5b, H7, H9 (context-aware vs context-unaware); H11 on its stated mean metric, H12 (residual RL vs heuristic) and H13 (prediction adds to B), see §46.
- **Marginal**: H3 (PPO > proposed by -0.0042 in utility, $p = 0.360$ in the final run). It is not Bonferroni-significant and was not reproduced in the three earlier full runs, so it is not claimed.
- **Supported after the audit**: H10 (self-healing vs none), H14 (SliceNet on every flow, non-inferior), and healthcare SLA gains of the QoS-targeted control (Q721).
- **Implementation goals only**: OVS QoS backend at scale, load balancing (absent), multi-site.

Part II

Literature review, novelty and research contribution

Read this before Part II

The repository's literature folder holds three papers. In `tracker.md`, the AI-slicing and self-healing literature reviews are still *open*. The answers below use those three papers plus canonical references; references marked † are not in the repository and must be checked against the original before citing. **Do not cite anything you have not read.** The honest framing for the panel is a targeted review around three anchors, with a systematic review protocol defined (Q76–77) but not yet fully executed.

5 Positioning against existing work

Q59. What are the three closest research papers to CARS-SF?

1. S. S. Badawadagi, M. K. CM, "SliceNet: A Hybrid CNN Model for Network Slicing in 5G Communications", ICEEICT 2024.
2. X. Limani et al., "Demonstrating Isolation Principles for 5G Network Slicing", IEEE NFV-SDN 2024.
3. A. M. Alwakeel, "Network Slicing and Stand Alone 5G Architectures: Paving the Way for Smart Cities and IoT", ICEIEC 2025.

Also relevant, cited by [1]: A. Thantharate et al., "DeepSlice", IEEE UEMCON 2019†.

Q60. What problem does each paper solve?

1 predicts the 3GPP slice type (eMBB/URLLC/mMTC) for a device from a KPI table, using a hybrid CNN, including for device types not seen in training (reported accuracy 96.97%).
2 demonstrates isolation between slices across RAN, transport and 5G core in a real 5G SA proof of concept.
3 surveys 5G SA architecture and NFV-based slicing for smart cities and IoT.

Q61. What are their assumptions, evaluation environments and limitations?

1 Tabular KPIs (LTE/5G category, QCI, packet loss, delay; about 31.6k train and 31.6k test rows, 17 columns) whose label is a deterministic function of declared attributes. There is no network in the loop. Classes are technology-centric, and evaluation is on a single held-out file.
2 Real hardware; it is a demonstration of isolation mechanisms, not an allocation policy, and has no context or self-healing.
3 A conceptual and architectural survey with no experimental evaluation of allocation.

Q62. Which existing approaches already do context-aware network resource allocation?

Context-aware or intent-based slicing and policy-driven QoS (3GPP PCF policy control, intent-based networking) exist in general. CARS-SF's specific angle is *socio-temporal* context (school hours, harvest, disaster alerts) selecting *service* priorities on a rural bottleneck.

Honest limit. A systematic search for context-aware rural slicing has not been completed. Avoid claiming "first".

Q63. Which existing approaches already do fault detection and recovery?

Fast reroute and link protection (MPLS FRR, BFD-triggered failover†, RFC 5880), OAM (ITU-T Y.1731), SON self-healing (3GPP SON†) and ETSI ZSM closed-loop automation†. CARS-SF's healing is conceptually standard protection switching. What is specific is coupling it to slice allocation (backup capacity ceilings, graceful degradation) and congestion-aware predictive switchover.

Q64. Which approaches combine traffic classification with network orchestration?

ML-based application identification feeding QoS or slicing decisions is common in the literature, and [1] maps predictions to slices conceptually. CARS-SF closes the loop: a live prediction installs a per-flow HTB rule, and the SLA outcome is measured (`classification/online.py`).

Q65. How does CARS-SF differ from a conventional SDN controller with predefined rules?

An SDN controller is a *mechanism* (it installs match-action rules). CARS-SF is a *policy loop*: it computes allocations from context and measured demand, enforces them with queueing disciplines (a controller alone does not shape or schedule), and heals. It could run *on* an SDN controller; the OVS backend uses OpenFlow `set_queue` rules.

Q66. How does it differ from static QoS enforcement with Linux tc?

Static tc is the `static_equal` baseline: fixed shares, no context, no demand, no healing. CARS-SF changes HTB rates at run time (re-planning every 5 s or on events), adds floors, borrowing ceilings and failure-driven constraints.

Q67. How does it differ from ordinary priority-based scheduling?

Strict priority starves lower classes and wastes reserved capacity. CARS-SF allocates *weighted* shares capped at measured demand, redistributes the surplus, and keeps floors. HTB prio only orders borrowing. The `proportional` baseline is weighted but demand-blind (Fig. 8).

Q68. How does it differ from existing ML traffic classification?

- Labels are *service* classes, not protocols or applications.
- Features are traffic shape only; QoS outcomes, ports and DSCP are excluded to avoid circularity and shortcuts.
- Evaluation holds out whole run×UE groups.
- Predictions are used in the loop.

Honest limit. The traffic is generated by our models, so the F1 is an emulation result (Q334).

Q69. How does it differ from a conventional network-slicing orchestration system?

Slice orchestrators (lifecycle of slice instances, NFV placement, as in [2] and [3]) create and isolate slices. CARS-SF assumes the slices exist and decides their *share of a scarce shared link* over time, plus recovery. These are complementary layers.

Q70. Which parts of CARS-SF are established techniques rather than novel?

HTB, fq_codel, netem, Mininet, OVS, water-filling allocation, Holt forecasting, random forests, 1D-CNNs, PPO, CEM, protection switching, Wilcoxon tests. All of them are established.

Q71. What exactly is novel about the integration, if the components exist?

The coupling and the evaluation:

- healing exposes *constraints* (backup-capacity ceilings, degraded mode) that the allocator honours;
- the predictor discriminates local congestion from backhaul degradation using the allocator's own queues;
- the loop is loop-free by construction (quarantine before backup, backup down before primary);
- outcomes are measured with packet-matched one-way delay on the same platform.

Q72. Is the novelty algorithmic, architectural, data-centric, methodological or empirical?

Architectural and empirical first, methodological second (the packet-matched evaluation harness and transfer study). It is *not* algorithmic: the allocator is a variant of weighted water-filling.

Q73. What is the strongest defensible novelty claim?

"An evaluated closed-loop framework that couples context-driven service slicing with loop-free self-healing on a shared rural bottleneck. In emulation (L3, including real OVS), it shows that service slicing raises critical-service SLA compliance ($+0.117$, $p = 0.0010$) and that healing cuts backhaul-failure outages to sub-second, while the choice among reasonable allocation policies and RL adds no significant benefit."

Q74. State that claim without "intelligent", "adaptive", "autonomous" or "AI-driven".

"We built and measured a controller that assigns shares of a rural uplink to four public-service classes according to time-of-day and emergency events, and that switches to a standby backhaul when the primary fails. Classifying by service raises clinic traffic's on-time delivery; the switching cuts outages from about 15 s to under a second in emulation."

Q75. What is the closest counterexample to your novelty claim?

A standard deployment that combines operator-configured QoS classes, DiffServ marking and BFD-triggered backup switchover already gets most of our measured benefit, with no context engine and no ML. Our own results are consistent with that: slicing presence and fast failover drive the gains. The answer: the contribution is the integrated, measured evidence for that in the rural setting, plus the coupling (constraints, congestion-aware prediction), not a superior allocator.

Q76. What search methodology did you use for the literature review?

Honest limit. Targeted, not systematic: forward and backward snowballing from three anchor papers [1–3].

Planned protocol to complete before the defense:

- databases: IEEE Xplore, ACM DL, Scopus;
- queries: ("network slicing" OR "QoS") AND (rural OR "developing regions") AND (context OR "self-healing" OR "traffic classification");
- years 2015–2026;
- PRISMA-style counts of records screened, excluded and included.

Q77. Which databases, keywords, years and inclusion criteria?

As in Q76. Inclusion: peer-reviewed, evaluates allocation, healing or classification with measurements or simulation. Exclusion: vendor white papers, position papers without evaluation (these may be cited as background only).

Q78. How do you distinguish peer-reviewed evidence from vendor claims or informal articles?

Cite only venue-published work for empirical claims, standards (3GPP, IETF RFCs, ITU-T) for definitions, and project documentation (Open5GS, UERANSIM, Mininet) for tool behaviour, marked as such.

Q79. What research gap remains after the closest related work?

[1] classifies without a network and with technology classes; [2] isolates slices without deciding shares or healing; [3] has no evaluation. None measures *service*-level outcomes under context changes and backhaul failures on a shared bottleneck with packet-level evidence.

Q80. How would an independent researcher reproduce your comparison?

`makesetupdatamodels`, then `experiments/run_matrix.py` (`configs/experiments/thesis.yaml`), then `aggregate.py` and `report.py`. Every run records params, seed and git hash (`run_meta.json`). A sudo-free path (user namespaces) and a Docker image for real OVS exist. Comparing against [1] would need their dataset; we did not reproduce [1].

6 Research contribution and attribution

Q81. List your contributions in order of importance.

1. The closed-loop framework coupling context-driven service slicing and loop-free self-healing (with event-driven failover and congestion-aware prediction).
2. The evaluation methodology: endpoint packet matching, a 340-run matrix, group-held-out ML evaluation.
3. The empirical findings: slicing presence matters, allocation-policy choice and RL do not, healing works, synthetic data does not transfer.
4. SliceNet on measured traffic and its in-loop use.
5. A 5G-core (Open5GS) enforcement demonstration.

Q82. Which contribution can be evaluated independently?

The measurement methodology (validated by exact propagation floors 8.00/28.00 ms and unit tests on synthetic pcaps), SliceNet (offline F1), and the transfer study.

Q83. Which contribution depends on another being correct?

All network-outcome claims depend on the measurement pipeline. In-loop SliceNet depends on SliceNet and the QoS enforcer. RL results depend on simulator calibration.

Q84. What does RSNSD contribute beyond a collection of synthetic records?

A service-semantic schema (89 columns, provenance-tagged), scenario-aware generation (19 scenarios, 32 applications) and a reproducible generator.

Honest limit. Its value as *training data for measured traffic* is refuted by our own transfer study (zero-shot F1 0.104). Its defensible role is as a schema and bootstrap; the measured dataset supersedes it.

Q85. What does SliceNet contribute beyond training a classifier on RSNSD?

It is trained on *measured* windows (4,697 from 18 runs), uses only shape features at a single vantage point, is evaluated on held-out run×UE groups (RF 0.947, CNN 0.858), and is deployed in the loop for a mixed-service UE.

Q86. What does the allocator contribute beyond a predefined priority order?

Demand-awareness (measured offered load, elastic probe-up, Holt forecast), floors, surplus redistribution, and healing constraints.

Honest limit. The main study does not show a significant utility gain over simpler policies; the gain appears only when HTB borrowing is disabled (ablation) or at $\geq 1\times$ load.

Q87. What does self-healing contribute beyond executing a recovery action?

- detection in milliseconds through an event-driven link watcher;
- diagnosis (link vs core switch, by elimination);
- loop-free ordering;
- hold-down plus OAM-gated fail-back;
- suppression of its own link events;
- prediction from RTT trends, with congestion discrimination;
- allocation constraints during the failure.

Q88. What does the closed loop contribute beyond chaining modules?

Shared state and feedback. The allocator sees healing state; the predictor sees allocator queues; the demand estimator sees enforcement saturation (probe-up); re-planning is event-triggered. Two real bugs were found *only* at the seams: bufferbloat on the backup link, and false failover under AQM-less queues.

Q89. Which contribution has the strongest experimental evidence?

Self-healing (deterministic, large effect: outage 0.165 vs 15.126 s, replicated on real OVS), and slicing vs no slicing for healthcare SLA (Bonferroni-significant).

Q90. Which contribution currently has the weakest evidence?

Context awareness (no measurable ablation effect), RL (null result), and anything about real rural deployment (no L4 evidence).

Q91. Which parts did you personally design, implement, evaluate or validate?

Answer truthfully for your own case. The repository history attributes the framework, registries, traffic models, measurement pipeline, controller, healing engine, SliceNet pipeline, experiment tooling and 5G deployment to this project. Be ready to state which were AI-assisted and how you verified them.

Q92. Which parts were adapted from existing tools or libraries?

Mininet, Open vSwitch, Linux tc (HTB, fq_codel, netem), tcpdump and dpkt, scikit-learn, PyTorch, stable-baselines3 and Gymnasium, Open5GS and UERANSIM (Docker images by Gradiant), pandas and SciPy.

Q93. What would remain as a contribution if the ML component were removed?

Almost everything that is significant: service slicing with healing, the measurement methodology, and the empirical findings. ML contributes the transfer result and the mixed-UE capability only.

Q94. What would remain if the recovery mechanism were replaced by a static policy?

A static "always use the backup when the primary is down" rule *is* our reactive core. What remains specific is event-driven speed, safe fail-back, fault localisation, predictive switchover (outage 0.019 vs 0.214 s reactive-only) and the allocation coupling.

Q95. What would remain if RSNSD were replaced with a different dataset?

Everything except the transfer study. The in-loop classifier already uses the measured dataset, not RSNSD.

Q96. What would persuade a skeptical reviewer that this is more than software integration?

Falsifiable pre-registered hypotheses with reported negative results; packet-level measurement validated against ground truth (exact delay floors); statistics (CIs, paired tests, Bonferroni); integration bugs found and fixed (busy-wait timing distortion, partial-window bias, uplink-only utilisation, false failover) with all runs redone. That is research method, not plumbing.

Part III

Requirements, architecture and end-to-end data flow

7 Requirements engineering

Q97. What are the functional requirements of CARS-SF?

F1 Infer the operating context from clock, calendar, alerts and admin input.
F2 Map traffic to a service slice (by address, application or prediction).
F3 Estimate per-slice demand from measurements.
F4 Compute a feasible allocation with floors.
F5 Enforce it in both directions of the bottleneck.
F6 Monitor per-slice SLAs.
F7 Detect, diagnose and recover from backhaul and core faults.
F8 Fail back safely.
F9 Log every decision with its inputs.
F10 Produce measured datasets for learning.

Q98. What are its non-functional requirements?

N1 Detection in under 1 s; critical outage under 1 s.
N2 Re-plan within one control interval (5 s).
N3 Never form an L2 loop.
N4 Learned components cannot bypass floors.
N5 Negligible control-plane overhead.
N6 Reproducible runs (seeded, versioned).
N7 Swappable stages.
N8 Sudo-free evaluability.

Q99. Which requirements are measurable, and which are qualitative?

Measurable: N1 (MTTD/MTTR, outage), N2 (controller timestamps), N3 (by construction plus tests), N5 (probe and tc command rates), N6 (re-running a seed gives the same traffic schedule; unit test). Qualitative: N7 (demonstrated by three QoS backends and seven schedulers), F2–F10 (demonstrated, not scored).

Q100. How did you prioritise requirements?

By harm. Safety first (no loops, floors respected, no fail-back onto a bad link), then critical SLA, then efficiency (surplus), then extensibility (ML, 5G).

Q101. Which requirements are safety-critical?

N3 (a loop would cause a broadcast storm and take down the village network), N4, the floors, and safe fail-back (OAM-gated). All are enforced by construction and covered by unit tests (`tests/test_self_healing_engine.py`).

Q102. System requirement vs implementation detail?

"Critical outage under 1 s after a backhaul cut" is a requirement. "50 ms link-watcher polling with an event that wakes the loop" is one implementation meeting it.

Q103. What is the expected monitoring interval?

Telemetry and SLA monitoring every 1 s; RTT probes once per second per service, in a background thread; link state every 50 ms; re-planning every 5 s.

Q104. What maximum detection and recovery time is acceptable for critical services?

Target: detection under 1 s and service outage under 1 s, roughly the tolerance of interactive voice and video (beyond that, calls drop or freeze). Achieved: MTTD about 0 s with the link watcher; outage 0.165 s (Linux bridges) and 0.33–0.53 s on real OVS.

Honest limit. The 1 s budget is a design choice, not derived from a clinical standard.

Q105. What availability target would you set, and how would you measure it?

For critical slices, 99.9% of 1-minute intervals SLA-compliant, measured by `SliceMonitor` over long runs.

Honest limit. Not measured: runs are 40–45 s, so we report outage per failure event instead.

Q106. How much control-plane overhead is acceptable?

Below 1% of bottleneck capacity and a few percent of one CPU core. Measured costs:

- four ICMP probes per second (negligible bandwidth);
- 8 `tcclasschange` commands per re-plan;
- a controller step of about 7 μs; a PPO re-plan of about 0.15 ms.

Endpoint captures are *evaluation* overhead, not a control-plane requirement.

Q107. How should the system behave when requirements conflict?

By a fixed precedence:

1. safety (loop-freedom, floors);
2. healing constraints (no core slice above backup capacity);
3. context priorities;
4. demand and efficiency.

For context, admin override beats alerts, which beat the calendar.

Q108. Which requirements are satisfied, partially satisfied, or unverified?

- **Satisfied (L3):** F1–F10 except as below; N1–N4; N6; N8.
- **Partial:** F7 (no generic post-action verify-and-rollback); F9 (no correlation IDs across artefacts); N5 (not stress-tested).
- **Unverified:** availability over time; behaviour at scale (hundreds of nodes); behaviour under real radio.

Q109. How are requirements mapped to modules and test cases?

Req.	Module	Evidence
F1	`context/engine.py`	`test_context.py`, timeline study
F3	`slicing/demand_estimator.py`	`test_slicing.py` (probe-up, Holt)
F4	`slicing/allocators.py`	`test_slicing.py` (floors, capacity)
F5	`slicing/qos_enforcer.py`	`test_qos.py` (exact tc commands); all runs
F7, F8, N3	`self_healing/engine.py`	`test_self_healing_engine.py`; resilience study; real OVS
F10	`features/`	`test_features.py` (OWD, loss on synthetic pcaps)
N1	engine + fast path	`test_link_event_fast_path...`; OVS MTTR 0.094 s

Q110. What would be the acceptance criteria for declaring CARS-SF complete?

- All unit and integration tests green.
- The full matrix reproducible from a clean checkout.
- Every hypothesis evaluated, positive or negative.

- Each claim in the thesis carrying a maturity level.
- One sudo run on real OVS for each failure type.
- For a deployment claim: an L4 field trial.

8 High-level architecture

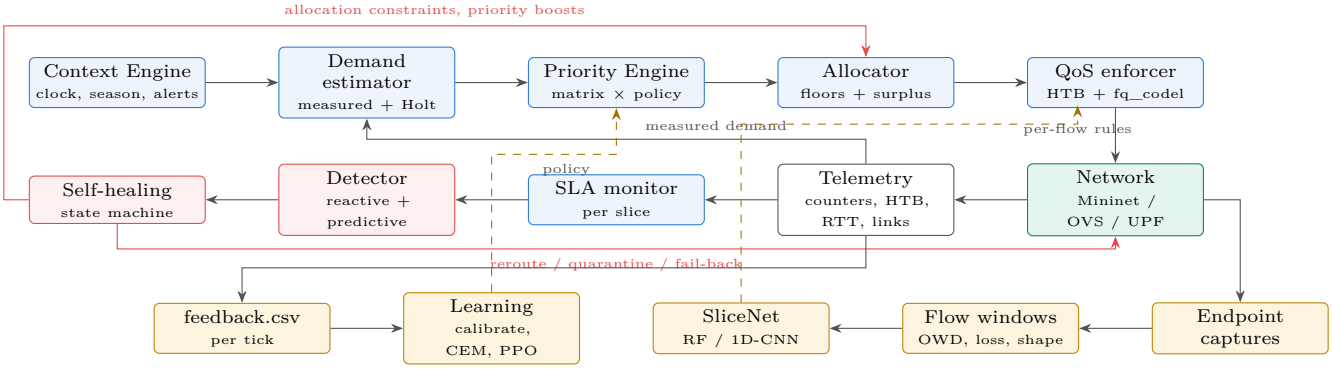


Figure 2: End-to-end architecture. Row 1: live decision path (1 s ticks, re-plan every 5 s). Row 2: observation and self-healing. Row 3: offline data, training and analysis; dashed arrows are artefacts deployed back into the loop.

Q111. Draw the complete architecture from traffic generation to recovery verification.

Fig. 2. The tick order is in Fig. 3, and "recovery verification" is the state machine's escalation and fail-back guards (Fig. 12).

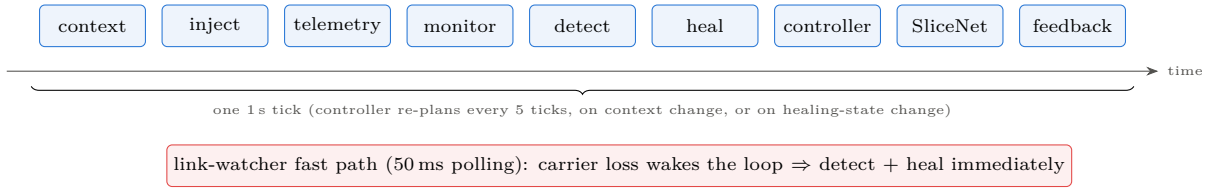


Figure 3: Order of operations per tick (`core/orchestrator.py`).

Q112. Why did you choose a modular architecture?

Why. Research needs to swap one mechanism at a time (allocator, policy, QoS backend) and compare.

Choice. Protocol-typed stages (`core/interfaces.py`) composed by name from `schedulers.yaml`.

Alternative. A monolithic controller (faster to write, impossible to ablate cleanly), or microservices (operational overhead).

Trade-off. Ablations and baselines are config changes; the price is indirection and interfaces that are only checked by duck-typing.

Evidence. Seven schedulers and three QoS backends (tc, OVS, UPF) run on the same controller. **L3 validated (emulation)**

Q113. Responsibilities of each module.

- **Context**: infers the context id.
- **Traffic generator**: model-driven, seeded flows (apparatus).
- **Topology**: builds the network and owns the path primitives.
- **Telemetry**: per-tick deltas, class stats, probes, link events.
- **Features**: packet matching and windows (post-run; shape features also live).
- **Dataset**: builds and validates the dataset.
- **Classifier**: SliceNet, offline and in-loop.
- **Allocation**: rates.
- **QoS**: tc / OVS / UPF commands.
- **Monitor**: per-slice SLA records.
- **Detector**: failure and prediction events.
- **Planner / healer**: path actions and constraints.
- **Verifier**: escalation, hold-down and OAM guards inside the healer (no separate module).

Q114. Which modules are on the live decision path?

Context, demand estimator, priority engine (plus an optional CEM or PPO policy), allocator, QoS enforcer, telemetry, SLA monitor, detector, predictor and healer. The in-loop classifier is on the path only when `classification=slicenet`.

Q115. Which modules operate offline?

Packet matching, feature extraction and dataset building (after each run), the validator, SliceNet training, RL training and calibration, aggregation and reporting, and the RSNSD generator.

Q116. Which modules are experimental alternatives rather than the active production flow?

CEM and PPO policies, the no-surplus, proportional, static, 3GPP and no-slicing allocators, the OVS QoS backend, the oracle demand mode, the legacy `RecoveryPlanner`, and the legacy `MininetSims/` and `RSNSDv2/` (excluded from tests). The default (production) composition is the `proposed` scheduler with the tc backend and host classification.

Q117. What is the canonical source of truth for the architecture?

The code plus the registries. Prose: `ARCHITECTURE.md`, whose §12 maps every original architecture module to code with its status. `architecture.md` is the original vision, kept with a status banner.

Q118. How do you prevent the architecture document from describing capabilities that aren't integrated?

- The status table states maturity (load balancing is marked *not implemented*).
- Every claim in the results is generated from data (`report.py`, `defense_numbers.py`).
- Integration tests assert the integrated behaviours (context switching, predictive failover, fault localisation, in-loop classification).

Honest limit. The original document did over-claim ("QoS Enforcer: prototype"). The verification pass that started this work found the Mininet path had never run.

Q119. Which module owns the current state of the network?

No single owner. Telemetry owns the observed state (snapshot per tick). The controller owns the intended allocation (`controller.allocation`). The healer owns the path state (NORMAL / ON_BACKUP / DEGRADED). The kernel owns the actual state.

Honest limit. Intended and actual state are not reconciled by reading back tc (Q139).

Q120. Which module owns the service requirements and SLA definitions?

`registries/slices.yaml` (slice SLAs, floors, class ids) and `registries/applications.yaml` (per-application SLAs and models). They are read by the monitor, the extractor (SLA labels), the allocator (floors) and QoS (classes).

Q121. How are modules coupled?

- Direct in-process calls inside the orchestrator loop.
- YAML configuration.
- Two threads (probe worker, link watcher) with locks, plus a `threading.Event` that wakes the loop.
- Subprocesses (generators, sinks, tcpdump).
- Shell commands to the network (`tc`, `ip`, `ovs-*`).
- Files for post-run artefacts.

Q122. Why is that communication mechanism appropriate?

One host, one experiment, millisecond control timescales: in-process calls are the simplest and fastest, and keep runs deterministic and debuggable. A broker would add latency and failure modes without a distribution need.

Q123. What happens when one module is slow, unavailable, or returns invalid data?

- Probes run in a background thread, so a lost probe never blocks the loop; it reports `None`.
- Telemetry readers catch errors and return empty or zero values.
- Ticks are wall-clock aligned, so a late tick does not shift later ones.
- A missing RL model falls back to rule-based, and this is recorded (`policy_fallback_to_rule`).
- A missing SliceNet model disables in-loop classification.

Honest limit. tc command failures are not detected (exit codes are not checked), and missing telemetry can look like zero traffic (Q279).

Q124. Which interfaces have explicit input and output schemas?

The stage protocols in `core/interfaces.py` (types only). The dataset schema (`SCHEMA_COLUMNS` and `registries/features.yaml`, validated by `DatasetValidator`). The scheduler registry is validated by name.

Honest limit. The telemetry snapshot is a convention-based dict and the YAML registries have no JSON-schema validation.

Q125. How are units, timestamps, identifiers and missing values represented?

- Units are in the names (`_mbps`, `_ms`, `_pct`, `_bytes`).
- Timestamps are Unix epoch seconds (float) from one host clock.
- Identifiers: `run_id`, `application_id`, `ue_host`, `flow_id`, `window_index`.
- Missing values: `None` in telemetry (a lost probe), `NaN` in datasets (no delivered packets).

Q126. How are configurations validated before execution?

Scheduler names are checked against the registry; CLI options are restricted by argparse `choices`; scenario and timeline ids raise on unknown values; the allocator and QoS raise on unknown classes.

Honest limit. Value ranges in YAML (negative demand, floors above capacity) are not validated up front.

Q127. How do you prevent inconsistent definitions of service priority across modules?

One source: `scenarios.yaml` priority weights, read by `slicing/context_engine.py`, which overlays the registry on its legacy table. HTB prio and class ids come from `slices.yaml`.

Honest limit. `SERVICE_CLASS_IDS` is duplicated in the QoS enforcer and the telemetry collector, with the same values; it should be read from `slices.yaml`.

Q128. How are configuration versions recorded with experiment results?

`run_meta.json` stores all parameters plus the git commit, and `summary.json` stores the scheduler spec and all options.

Honest limit. Uncommitted registry edits are not captured by the hash. The matrix was always run from committed code.

Q129. What are the major architectural failure points?

- The single orchestrator process (no high availability).
- Host CPU contention distorting timing (observed and fixed).
- Silent tc failures.
- Telemetry stalls that look like zero traffic.
- The link watcher depending on carrier propagation (which differs between OVS and Linux bridges).
- Disk for captures (about 100 MB per run).

Q130. What would you redesign first to support hundreds of nodes?

- Split into per-site agents (local enforcement plus fail-safe defaults) and a regional controller.
- Replace the polling of `/sys` and `tc` with streaming telemetry (sFlow, IPFIX, gNMI).
- Keep state in a store with leases.
- Use OpenFlow or P4 path control instead of admin-state toggling.
- Make allocation a per-link optimisation.

9 End-to-end execution

Q131. Walk one application flow from creation to final QoS enforcement.

1. The `TrafficOrchestrator` plans the telemedicine app on `h_health` with demand-scaled parameters and a fixed seed.
2. Before traffic starts, the QoS setup installs HTB classes and a host filter `src10.0.1.10âĘŠ1:10` on `s_rural-up` (and `dst` on `s_edge-down`).
3. The generator sends VBR UDP; the packets hit the u32 filter and the fq_codel leaf of class 1:10 at the current rate and ceiling.
4. Telemetry counts bytes; the controller re-plans every 5 s and issues `tcclasschange`.
5. After the run, captures on `h_health` and `srv_edge` are matched, giving one-way delay and loss per 5 s window, labelled `healthcare.telemedicine`.

Q132. Walk through a medical-emergency scenario from context change to verified recovery.

Timeline `medical_night`: tick 6 raises `medical_emergency`.

1. The context engine returns the new context with `changed=True`.
2. The traffic orchestrator re-spawns generators with the emergency demand (healthcare 95 Mb/s).
3. The controller re-plans immediately (reason: context change) with weights 15/1/1/1 from the prior demand, because the estimator restarts with a warm-up.
4. The QoS classes change.
5. The SLA monitor records the healthcare slice every tick; latency spikes trigger ×2 boosts for 6 ticks.

Honest limit. "Verified recovery" here means the SLA monitor shows compliance afterwards. There is no explicit "SLA restored" event that ends the episode.

Q133. Which component first observes the emergency?

The context engine, through an alert event in the timeline (in production: an alert feed or operator). The network does not "observe" an emergency; it observes its consequences (demand, RTT).

Q134. How does the emergency change service priorities?

The priority matrix switches to the context's weights (healthcare 15 vs 1), and the allocator gives healthcare its weighted share up to demand. The floors stay unchanged.

Q135. How is traffic associated with a service class?

- **host** (default): UE address → service.
- **app**: registry server and port (oracle; also used for 3GPP slice types).

- **slicenet**: per-flow prediction for unknown or mixed UEs.

The dataset labels come from spawn metadata (ground truth).

Q136. How does that classification reach the allocation engine?

It doesn't need to. Classification decides *which HTB class* packets enter; the allocator decides *each class's rate*. They meet in the kernel. Demand per class is measured per class.

Q137. How does the allocation engine's decision become network configuration?

`SliceController.step` → `QoSEnforcer.apply_allocation`, which issues `tcclasschangeâĂęrateXceilY` on `s_rural-up` and `s_edge-down` (ceiling = capacity, or the allocation for core slices while on backup). On OVS: `ovs-vsctlsetqueueâĂę`. On the 5G UPF: the same tc commands via `dockerexec`.

Q138. How does monitoring determine whether the configuration worked?

Indirectly, through outcomes: per-class throughput (counters), drops (HTB stats), probe RTT, and SLA compliance per slice per tick. Post-run, packet-matched KPIs per window.

Q139. What if the allocation decision is correct but the network configuration fails?

Honest limit. Today it goes undetected at configuration time. The controller assumes success, and only the SLA monitor would show the violation later.

Fix (small): after `apply_allocation`, read back `tc-sclassshow` (`get_queue_stats` exists), compare rate and ceiling, retry once, and raise a `config_failure` event to the healer or operator.

Q140. What if the configuration succeeds but the SLA remains violated?

The detector raises `latency_spike` (RTT above 200 ms) or `congestion` (more than 50 drops per tick). The healer boosts the affected slice's priority ×2 for 6 ticks (with a cooldown), and the controller re-plans. The predictor may also flag a backhaul trend.

Honest limit. There is no escalation if repeated boosts fail (Q508).

Q141. How do you prevent a module from reporting success when a downstream module has failed?

- Runs are judged on measured data-plane outcomes, never on module return codes.
- Integration tests assert data-plane facts (a PCAP dataset exists, the outage was measured, actions were taken).
- A run whose dataset falls back to telemetry is marked `degraded` by the matrix runner (none of the 340).

Q142. What evidence shows the complete path works, not just each component?

- The 7 integration tests (timeline, failover, prediction, core localisation, baselines, in-loop SliceNet).
- 340 matrix runs, every one producing packet-matched data.
- Real-OVS runs in a privileged container.
- The 5G-core runs.

All are L3 in the named environment.

Q143. Which exact command or entry point starts the experiment?

- Real OVS: `sudo.venv/bin/pythonrun_experiment.py--scenariofiber_cut--duration45--apply-qos`.
- Sudo-free: `python-mtests.integration.harnessfiber_cut45apply_qos=true`.
- Studies: `experiments/run_matrix.py`.
- 5G: `deployment/5g/run_5g.py`.

Q144. Which processes, threads, namespaces, interfaces and background services are involved?

- The orchestrator process with a probe thread and a link-watcher thread.
- A Mininet shell per node.
- 12 network namespaces (8 UEs and 4 servers) plus the root namespace for switches.
- About 24 veth pairs; about 40 generator and sink processes; 12 tcpdump processes.
- On OVS: `ovsdb-server` and `ovs-vswitchd`.

The harness wraps all of this in one user+net+mount namespace.

Q145. What artefacts are generated after a run?

`results/runs/<run_id>/`: `summary.json`, `dataset.csv`, `sequences.npz`, `feedback.csv`, `slice_monitor.csv`, `validation_report.txt`, `run_meta.json`. Also captures in `generated/pcap/<run_id>/` and generator logs in `generated/logs/<run_id>/`.

Q146. How do you trace one decision across logs, telemetry, configuration and metrics?

By tick and timestamp: `controller_history` (tick, reason, demand estimate, priorities, allocation, applied) → `feedback.csv` (per tick: allocation, throughput, RTT) → dataset windows (`window_index`, flow times) → healing actions (timestamps, trigger).

Honest limit. There is no shared decision ID; correlation is by time.

Q147. Data pipeline that produces a dataset vs runtime pipeline that controls a network?

The runtime loop acts every second on aggregate telemetry. The data pipeline reconstructs per-flow, per-window facts after the run from packet captures (ground-truth labels, one-way delay). The first is causal and online; the second is retrospective and exact.

Q148. Which modules are shared by both pipelines?

Topology, traffic models, telemetry, the registries, and `features/shape.py` (shape features and sequences, used both for training and for the in-loop classifier).

Q149. Where can their behaviour diverge?

- The in-loop classifier sees one vantage point (the UE capture) and the cumulative flow history; training used 5 s windows from the sender side of matched packets.
- Rate features are normalised by the observed span (at least one window) to reduce the mismatch.
- Live telemetry is aggregate; dataset QoS is per flow.

Q150. How do you ensure the runtime uses the same feature definitions and taxonomy as offline training?

The same functions (`shape_features`, `packet_sequence`); the trained model stores its feature column order and normalisation; the four-class taxonomy comes from one registry. The in-loop classifier picks whichever model scored higher on held-out data.

Honest limit. There is no automatic train/serve skew check.

10 Architectural alternatives and trade-offs

Q151. Why not implement everything in one Python process?

It nearly is: one orchestrator process with threads. Generators and tcpdump are separate processes because they must run *inside* host namespaces and must not block the loop.

Q152. Why not use a message broker between every module?

Alternative. Kafka or MQTT between modules.

Trade-off. It would decouple and distribute, but add milliseconds, ordering issues and a broker as a single point of failure, with no need on one host. It becomes justified with per-site agents (Q130).

Q153. Why not use an OpenFlow SDN controller as the central component?

Why. The problem is shaping and scheduling at a bottleneck and path protection. OpenFlow can steer packets to queues but does not schedule them.

Choice. The kernel qdiscs do the scheduling. The topology uses `OVSBridge` in standalone mode (no controller); the OVS backend uses OpenFlow `set_queue`.

Alternative. Ryu or ONOS with meters and queues. It would add path computation for multipath healing.

Trade-off. Simpler and fewer moving parts; the cost is no flow-level path control (hence no load balancing).

Q154. Why not use a rule engine instead of an ML decision layer?

We essentially *do*: the default decision layer is rule-based (context weights, floors, water-filling, threshold detectors). ML is used only where rules lack information (traffic identity for a mixed UE) and as an evaluated alternative (RL). It is not justified by results.

Q155. Why not use a centralised optimiser with a global view?

For a single link with a separable concave objective, weighted water-filling with caps is essentially what an LP or convex solver returns (Q432). A global optimiser pays off with multiple links and paths. The global view here is just one link.

Q156. Advantages and disadvantages of centralised, distributed and hierarchical control?

- Centralised: simple, consistent, but a single point of failure and a partition risk.
- Distributed: resilient, local, but needs coordination for consistent shares.
- Hierarchical: local enforcement and fast failover at the site, regional policy above.

CARS-SF is centralised per site, and that is the right granularity for one village. Regional deployment should be hierarchical.

Q157. How would you handle a partition that isolates the controller from some switches?

Switches keep their last HTB configuration (the kernel retains it), so enforcement continues. Healing on the cut-off side stops.

Honest limit. No local fail-safe agent exists today. Add one with a lease: if the controller is silent for N seconds, revert to the context-independent safe profile and run local failover.

Q158. What should keep operating locally if the central controller is unavailable?

Floors and the last allocation (already persistent in the kernel); local link-failure switchover (protection switching should be local by design); loop-safety interlocks.

Q159. How would you add a new service class without modifying every module?

Registry entries in `slices.yaml`, `applications.yaml` and `scenarios.yaml` are data-driven for QoS classes, monitoring and traffic.

Honest limit. Several modules hard-code the 4-tuple `SERVICE_CLASSES` (allocators, env, analysis). Generalising that tuple to read the registry is required.

Q160. How would you add a new telemetry backend without rewriting allocation logic?

Implement `TelemetrySource` so that it returns the same snapshot keys (`service_throughput_mbps`, `tc_class_stats`, `service_rtt_ms`, `link_events`...). Allocation consumes only those through the demand estimator.

Q161. Which abstractions are essential, and which add unnecessary complexity?

Essential: the allocator, policy, QoS backend and telemetry snapshot. Questionable: the legacy `RecoveryPlanner` (superseded but kept for old tests) and the two context classes (`ContextManager` is a thin logger).

Q162. How would you separate policy, decision-making and enforcement more cleanly?

Policy = registries plus a policy object (weights, floors, SLAs) with a version. Decision = a pure function `(state,policy) âĘŠintent`. Enforcement = backend adapters that *acknowledge* and read back. Today the healer mixes decision and enforcement (it calls topology primitives directly).

Q163. What is the smallest set of interfaces that preserves modularity?

Four: `TelemetrySource.poll`, `Allocator.allocate`, `QoSBackend.apply_allocation` and `Healer.handle/constraints`. Context and policy can be functions.

Q164. Which design decisions were driven by research needs rather than production requirements?

- endpoint packet capture (evaluation ground truth);
- seeded generators;
- the oracle demand mode;
- seven baselines;
- the sudo-free namespace harness;
- per-run artefact directories;
- the predictive detector's OAM shortcut (reading the emulated impairment).

Part IV

Context modelling and application traffic generation

11 Context engine

Q165. What does context mean in CARS-SF's mathematical and operational model?

A discrete variable $c_t \in \mathcal{C} = \{\text{normal, school, harvest, disaster, medical_emergency, fiber_cut, backhaul_degradation, core_switch_failure}\}$, inferred each tick by

$$c_t = f(\text{clock}_t,\ \text{calendar},\ A_t,\ o_t),$$

where A_t is the set of active alerts and o_t the admin override. Operationally, c_t selects the priority weights $w(c_t)$ and the design demand $d(c_t)$ from `scenarios.yaml`.

Q166. What context variables are represented?

- time of day and weekday;
- month (harvest months 3, 4, 10, 11);
- school days and hours (Mon–Fri, 08–15);
- harvest daylight hours (06–18);
- alerts (flood, fire, earthquake, disaster, medical_emergency, fiber_cut);
- an admin override.

Source: `registries/timelines.yaml`, `context/engine.py`.

Q167. Which are static, slowly varying, event-driven or continuously changing?

- Static: the calendar configuration.

- Slowly varying: season (month).
- Continuously changing: time of day.
- Event-driven: alerts and the admin override.

Network condition is *not* context; it is handled by detection and healing.

Q168. How are emergencies, congestion, link failures, holidays and power failures represented?

- Emergencies: alerts mapped to contexts (flood, fire and earthquake → disaster).
- Link failures: scenarios with a `failure` spec driving the injector.
- Congestion: emerges from offered load; it is not a context.

Honest limit. Holidays and power failures exist only in the RSNSD generator's 19 scenarios, not in the live CARS-SF context registry.

Q169. Why is a scenario registry better than hard-coded conditions spread through the code?

Why. One definition of priorities and demand per context for the allocator, the traffic, the RL environment and the analysis.

Choice. `scenarios.yaml`, read everywhere.

Alternative. `ifcontext==...` branches (the original `DemandGenerator` did this, and it diverged: `medical_emergency` silently fell back to the defaults).

Trade-off. Consistency and a new context added by editing data; the cost is no schema validation of the YAML (Q126).

Evidence. The bug that motivated this was found in the verification audit. **L3 validated (emulation)**

Q170. What are the context engine's inputs and outputs?

In: tick, timeline events (alerts, clear, admin), clock start and minutes per tick. Out: `\{tick,clock,context,reason,alerts,changed\}` per tick, plus history.

Q171. Does context describe the environment, the application's urgency, the network's condition, or all three?

The environment and the *societal* urgency it implies (school is in session, a flood is happening). Application urgency is in the per-app SLAs; network condition is in telemetry. Keeping them separate prevents a congested link from being read as "an emergency".

Q172. How do you distinguish a change in demand from a change in capacity?

Demand is measured where traffic is still unshaped: access-port rx counters, and HTB sent+dropped. Capacity changes are healing states (backup at 50 Mb/s, core down) that enter as constraints. The predictor adds a discriminator: rising RTT *with* local queue growth or drops is congestion (demand); rising RTT *without* it is the backhaul (capacity or quality).

Q173. How does the system prevent two context sources from issuing conflicting priorities?

A strict precedence produces exactly one context: admin override > alerts (medical > disaster > fiber cut) > fixed scenario > school hours > harvest > normal (`ContextInferenceEngine.infer`; unit-tested).

Q174. How are simultaneous events combined?

They are not blended. Precedence picks one context.

Honest limit. Flood plus school hours yields disaster weights, with no "school during a flood" mix. Blending (e.g. a max or weighted combination of weight vectors) is possible future work.

Q175. What happens if an emergency begins while the network is already congested?

The context switch triggers an immediate re-plan with emergency weights and the emergency's design demand as the prior, because the estimator restarts with warm-up. The floors still hold. Congestion boosts from the healer multiply on top. In the timeline runs, disaster begins mid-run under load.

Q176. What happens if an emergency ends while the system is still recovering from a link failure?

Context and healing are independent state machines. The controller re-plans for the new context, and the healing constraints (backup ceiling) still apply until fail-back.

Honest limit. This exact overlap is not in the test matrix: the timelines do not inject faults.

Q177. How do you model event duration and transitions?

Timeline events carry a raise tick and a clear tick (flood from tick 12 to tick 24). The calendar produces deterministic transitions. The RL environment uses Markov switching ($p_{stay} = 0.92$).

Q178. How do you avoid abrupt policy oscillations when context changes rapidly?

- Contexts change only on events or calendar boundaries.
- The controller applies a change only if a slice moves by at least 1 Mb/s.
- Healing boosts have a cooldown.

Honest limit. There is no dwell-time hysteresis on context itself. A flapping alert feed would flap priorities; a minimum context dwell time is a one-line extension.

Q179. Is context categorical, numerical, probabilistic, or a combination?

Categorical: one id, one-hot in the RL observation. The calendar inputs are numerical, but inference is rule-based and deterministic, with no probabilities.

Q180. How are unknown or unsupported context values handled?

An unknown *alert* maps to disaster (the conservative default). An unknown context id raises in `get_context`, and the orchestrator's demand lookup falls back to normal. An unknown timeline raises before the run.

Q181. How would noisy or incorrect context affect the allocation decision?

Wrong weights shift shares between slices, but demand-capping limits the waste: a slice given a high weight but low demand gets only its demand, and the surplus is redistributed. The floors bound starvation. The `no_context` ablation (always "normal" weights) changed school utility from 0.865 to 0.861, i.e. no measurable damage. In this setting even fully wrong context is harmless; equally, correct context is not what drives outcomes.

Q182. How do you validate that a simulated context produces the intended downstream behaviour?

The integration test `test_timeline_switches_context_and_traffic` asserts that school and disaster occur and that the controller re-planned with reason "context". The timeline study's `feedback.csv` shows per-tick context, allocation and throughput (Fig. `timeline_throughput`).

Q183. How would real context enter the system?

- Operator UI (admin override).
- Alert feeds such as the Common Alerting Protocol from civil protection.
- School timetables and agricultural calendars from local institutions.
- Application metadata (a telemedicine session start) as an alert-like event.

The engine already exposes `raise_alert`, `clear_alert` and `set_admin_override`.

Q184. What are the security and trust implications of externally supplied context?

A spoofed "disaster" alert could deprioritise everything except healthcare. Mitigations:

- authenticated or signed feeds;
- allow-listed sources;
- rate limits and minimum dwell time;
- audit logs (the history is kept);
- a bounded blast radius, because the floors still hold.

Honest limit. None of the authentication is implemented.

12 Traffic and workload modelling

Q185. Why generate application-specific traffic instead of one generic source?

Why. Slicing and classification are only meaningful if services differ in burstiness, direction, rate and transport the way real applications do.

Choice. Seven models (VBR video, VoIP, IoT periodic, Pareto on/off, paced bulk TCP, web request/response, DASH) across 16 applications (`traffic/models.py`, `traffic/generator.py`).

Alternative. iperf3 CBR (the original code), or trace replay.

Trade-off. Realistic mixes and ground-truth labels; the cost is that the parameters are assumed (Q188).

Evidence. Unit tests check rate conformance and I-frame bursts. **L2 unit-tested**

Q186. Which generators are used per domain?

- **Healthcare**: telemedicine VBR video (up, 4 Mb/s), ECG periodic (50 pkt/s, 200 B), emergency voice (VoIP talkspurts), ambulance on/off (1.5 Mb/s).
- **Education**: live classroom VBR video (down, 20), library bulk TCP (down, 40), exam web (3), LMS web (8).
- **Agriculture**: soil (every 2 s), weather (every 5 s), drone video (up, 8), irrigation on/off (0.2).
- **General**: web (4), DASH streaming (15), social (3), banking (0.5).

All rates are nominal and are scaled to each context's demand.

Q187. How are packet sizes, inter-arrivals, durations, protocols and offered loads determined?

Per-model parameters in `applications.yaml`:

- video: fps, GOP, I/P ratio 4, lognormal coefficient of variation 0.25;
- VoIP: 172 B at 50 pkt/s, talkspurt exponential means 1.0/1.35 s;
- IoT: period and size with ±jitter;
- on/off: Pareto shape 1.5;
- web: Pareto response sizes, shape 1.3.

Offered load: each service's nominal rates are scaled so they sum to the context demand, then split across UEs. Durations last for the run.

Q188. Which parameters are from published evidence, which are measured, and which are assumptions?

- Standard-derived: G.711-style voice framing (50 pkt/s, 160 B payload); typical video frame rates and GOP structure.
- Classic models from the literature: Pareto on/off for self-similar traffic[†]; heavy-tailed web sizes.
- Measured: nothing from real rural networks.
- Assumptions: all rates and the per-context demands.

Q189. How do you model bursty traffic?

Pareto on/off (heavy-tailed bursts at an elevated rate so the mean is preserved), VBR video I-frames sent as 1 ms bursts, and web responses with Pareto sizes.

Q190. How do you model periodic telemetry?

`iot_periodic`: fixed size every `period`$\times(1 \pm \text{jitter})$. Demand scaling adds *devices* (instances), not rate.

Q191. How do you model sustained video or classroom traffic?

GOP-structured VBR. The I-frame is 4× a P-frame, frame sizes are lognormal, and each frame is split into $\leq$1200 B packets in a 1 ms burst at 25–30 fps. DASH streaming is TCP chunk downloads every 2 s.

Q192. How do you model emergency voice or medical streams?

Voice: 50 pkt/s during exponential talkspurts. ECG: periodic 200 B every 20 ms. Telemedicine: VBR video uplink.

Q193. How do you represent a sudden increase in demand?

A context switch re-spawns generators with the new demand scaling (timelines), and the load sweep (0.5–1.5×). Within a context, bursts come from the models themselves.

Q194. How do you model correlated bursts across several services?

Only through context switches, where all services change together.

Honest limit. There is no explicit cross-service burst correlation (a copula or shared modulating process). Independent seeds make bursts independent, which probably flatters statistical multiplexing.

Q195. How do you ensure generated workloads are reproducible?

Every generator gets a deterministic seed (`seedÂů100003+appÂů101+ue`), the schedules are pure functions of it, and a test confirms identical schedules for identical seeds. Realised packet timing still varies with OS scheduling.

Q196. How are random seeds managed across multiple generators?

The run seed comes from the matrix. Per-application, per-UE seeds are derived from it; instances add $7919 \cdot i$. A context switch increments the run seed so new traffic is not a replay.

Q197. How do you validate that a generator produces the distribution it claims?

`tests/test_traffic_models.py` checks the long-run rate against the nominal over 120 s (CBR 2%, video 8%, IoT 5%, on/off and VoIP 35% because of heavy tails), I-frame bursts above 3× the median frame, ordering and bounds. At run time, the dataset shows delivered rates per application.

Q198. How do you avoid unrealistic traffic that makes classification artificially easy?

- Ports, DSCP, TTL and timestamps are excluded from features.
- Evaluation holds out runs and UEs.
- Traffic is multi-model within each service.

Honest limit. The four services' model mixes are still ours. The F1 of 0.947 measures separability of *our* generators, not of real services. Real-trace validation is needed.

Q199. How do you distinguish offered load from delivered throughput?

Offered: the generator schedule, the access-port rx and HTB sent+dropped. Delivered: bytes received at the destination endpoint (matched packets). The gap is loss.

Q200. How do you tell whether the bottleneck is the sender, receiver, virtual link, queue or path?

- Delay floors equal configured propagation exactly (8.00/28.00 ms), so excess delay is queueing.
- HTB class backlog and drops locate the queue.
- Generator CPU (7% per 40 Mb/s) rules out the sender.
- In the 5G case, comparing UPF-shaped and unshaped modes exposed UERANSIM's user-space GTP as a bottleneck (Q598f).

Q201. Why not replay real packet traces?

No rural service traces with service labels exist that we can access. Replay also cannot react to slicing (TCP replays do not back off), and labels and privacy are hard. Synthetic, closed-loop generators react to the network.

Q202. Benefits and drawbacks of synthetic workloads vs real traces?

Benefits: control, ground-truth labels, reproducibility, scaling to any context, closed-loop TCP behaviour. Drawbacks: realism is unproven, unknown unknowns are missing, and classification may be optimistic.

Q203. How do you avoid training and evaluating on effectively identical generated flows?

By group hold-out: test windows come from runs and UEs never seen in training (144 groups). Seeds differ across runs.

Honest limit. The same *generator family* appears in train and test. Generalisation to new applications is untested.

Q204. How would you test generalisation to a previously unseen application?

Leave-one-application-out: train without, e.g., `general.social_media`, test on it, and measure whether it maps to the right service. Expect degradation. Add an "unknown" rejection threshold (Q396).

Q205. What happens if a traffic generator fails midway?

Its log (`generated/logs/<run>/app@host.log`) records the error, and the flow disappears from later windows. The validator's class-coverage score drops if a whole service vanishes. Nothing restarts it.

Honest limit. An integration assertion on per-app record counts would catch this; it is only implicit today.

Part V

Mininet, SDN, Open vSwitch and networking foundations

13 Why Mininet and Open vSwitch?

Q206. Why Mininet instead of NS-3, OMNeT++, a hardware testbed or a physical 5G network?

Why. The claims concern real queueing disciplines (HTB, fq_codel), real TCP congestion control and real control commands. These are kernel behaviours a discrete-event simulator re-implements.

Choice. Mininet: real Linux network stack, real `tc`, real applications, cheap to repeat 340 times, and the same commands run on real OVS.

Alternative. ns-3 (exact virtual time, scales, but models TCP and AQM rather than running them); a hardware testbed (realistic, costly, slow to repeat); a 5G testbed (radio realism; not available).

Trade-off. Real software behaviour and fast iteration, at the cost of timing fidelity under CPU contention and no radio.

Evidence. 340 runs; real-OVS replication; propagation floors exact to <0.01 ms. **L3 validated (emulation)**

Q207. What does Mininet emulate, and what does it not?

Emulates:

- hosts as processes in network namespaces;
- links as veth pairs with tc shaping and delay;
- switches (OVS or Linux bridge).

Does not emulate:

- CPU isolation (all nodes share the host CPUs);
- NIC hardware and line-rate timing;
- radio;
- wide-area loss processes beyond netem.

Q208. How does Mininet use Linux network namespaces?

Each host is a shell started with `mnexec -n` in its own network namespace: its own interfaces, routes, ARP table and sockets, but a shared kernel, filesystem and clock. Switches stay in the root namespace. The sudo-free harness wraps the whole emulation in one user+net+mount namespace.

Q209. What is the role of virtual Ethernet pairs?

A veth pair is a virtual cable: frames written to one end appear at the other. One end sits in the host's namespace, the other is attached to a switch port. Qdiscs on either end shape that direction.

Q210. How are hosts, switches, links and controllers represented?

- Hosts: `Host` (namespaced shells).
- Switches: `OVSBridge` (OVS bridge in standalone fail-mode) or `LinuxBridge`.

- Links: `TCLink` (veth + tc).
- Controller: none, because OVSBridge forwards with the NORMAL action (MAC learning) and needs no OpenFlow controller. This fixed a crash, since the reference controller binary is not installed.

Q211. What is Open vSwitch, and why is it appropriate?

A production-grade programmable software switch: a kernel datapath with a userspace slow path, configured through OVSDB, programmed by OpenFlow, with QoS queues (`linux-htb`). It is the data plane of many SDN and NFV deployments, so OVS-validated behaviour transfers to real virtual switching.

Q212. What is the difference between a Linux bridge and Open vSwitch?

A Linux bridge is a simple L2 learning switch with no flow tables. OVS has OpenFlow match-action tables, a flow cache, OVSDB configuration and QoS integration. Relevant behavioural differences we *measured*:

- carrier-loss propagation to the peer is slower on OVS (recovery 1.04 s before the event-driven fix, about 0.09 s after);
- MAC relearning makes the OVS outage 0.3–0.5 s vs 0.05 s on Linux bridges.

Q213. What is the difference between the SDN control plane and the data plane?

The control plane decides how traffic should be treated (rules, routes, allocations); the data plane forwards and queues packets. In CARS-SF the controller is our Python loop and the data plane is the kernel or OVS datapath with its qdiscs.

Q214. How does an OpenFlow rule affect packet forwarding?

A rule = match fields + priority + actions. A packet is matched against the table, the highest-priority match wins, and its actions run (output, set_queue, modify, drop). In the OVS QoS backend: `priority=20,ip,nw_src=<UE>,actions=set_queue:q,normal` places the packet in queue q and then forwards it normally.

Q215. How are flow-table entries matched and prioritised?

By priority, then (with equal priority) undefined order, so overlaps should be avoided. In our design, app rules (priority 30) beat host rules (priority 20), and the default NORMAL flow catches the rest. The tc u32 filters follow the same idea: flow (prio 1) > app (10) > host (20).

Q216. How are packets handled when no rule matches?

In OpenFlow, the table-miss entry decides (send to the controller, or drop). In our standalone OVSBridge the default rule is NORMAL. In tc, unmatched packets go to the HTB `default` class: general (1:40), or eMBB in 3GPP mode.

Q217. How do you verify that the intended forwarding rule is actually installed?

`ovs-ofctldump-flows` (collected in telemetry on OVS) and `tc-sclassshow` / `tcfiltershow`. Unit tests assert the exact commands (`test_qos.py`).

Honest limit. The controller does not read back and compare after each change (Q139).

Q218. How do you distinguish a control-plane decision from actual data-plane behaviour?

By measuring the data plane independently: endpoint captures give per-packet delivery and delay, and HTB counters give per-class bytes and drops. A decision counts only if those outcomes change (e.g. the ablation shows ceiling changes moving p95 from 10.0 to 165.0 ms).

Q219. What are the limitations of a virtual testbed on one machine?

Shared CPUs (contention distorts timers), no hardware queues, a limited aggregate rate, all clocks identical (good for one-way delay, unrealistic for synchronisation issues), and no radio.

Q220. How can host CPU scheduling distort latency and throughput measurements?

Starved processes send late (burstier); starved kernel timers release netem and HTB packets late (inflated delay); softirq backlog adds jitter.

Evidence. Observed: a busy-waiting generator pushed load to 40–56 on 20 cores. All 111 affected runs were archived; the generator now sleeps (100%→7% CPU per 40 Mb/s flow), with 4 parallel workers keeping load below the core count. **L3 validated (emulation)**

Q221. How do virtual link parameters translate into tc configuration?

`TCLink(bw,delay)` installs HTB (rate) with a netem child (delay) at the root of each veth end. Our QoS enforcer *replaces* the root qdisc on both bottleneck ends, which would silently discard any netem delay there. That is why propagation delay is emulated on the destination-facing ports instead (§14).

Q222. Which aspects of radio propagation, scheduling, handover and interference are absent?

All of them: no PHY or MAC, no PRB scheduling, no HARQ, no fading, no mobility. The 5G deployment adds the *core* (NAS, PDU sessions, slices) but UERANSIM simulates the RAN without radio impairments.

Q223. What conclusions can you legitimately draw from Mininet experiments?

Relative comparisons of allocation, AQM and healing policies at a shared IP bottleneck with real kernel queueing and real TCP; the correctness and timing of the control loop; measurement-method validity. **Not:** absolute field latencies, radio-limited behaviour, scalability to large networks.

Q224. Which conclusions need a radio-aware simulator or a real 5G testbed?

Radio scheduling interaction (PRB allocation per slice), coverage and handover effects, wireless loss, RAN-slice isolation (as in the isolation paper [2]), and end-to-end latency budgets including the air interface.

14 Topology and resource modelling

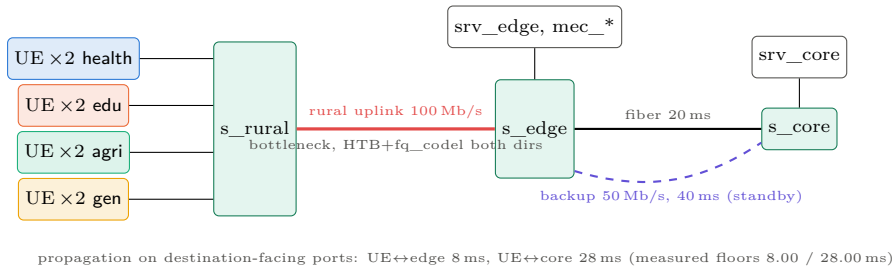


Figure 4: Emulated rural network: one shared contention point, a primary and a standby backhaul.

Q225. Why this rural topology?

It is the smallest topology containing all three phenomena under study:

- one shared rural uplink (contention);
- two service tiers (edge MEC vs core cloud), so that failures affect services unequally;
- a primary and a weaker standby backhaul (healing).

Q226. What do the switches, hosts, links and backup paths represent?

- `s_rural`: village aggregation (cell site / gNB aggregation).
- `s_edge`: edge cloud (MEC, clinic and agri servers).
- `s_core`: core cloud (education and internet services).
- UEs: 2 per service institution.
- The primary backhaul is fiber; the backup is microwave or satellite-class (50 Mb/s, 40 ms).

Q227. Which links are bottlenecks?

Only the rural uplink (100 Mb/s, both directions). Access links are deliberately unshaped. On backup, the 50 Mb/s backup link bounds core-bound traffic, and CARS-SF caps core slices to 95% of it.

Q228. How are link capacities, delays, loss rates and queue limits configured?

- Capacity: HTB root 100 Mb/s (swept 50–150).
- Delay: netem on destination-facing ports (8 ms edge path, 28 ms core path, backup +40 ms).
- Loss: zero by default; injected by the degradation scenario (+0.3% per tick).
- Queues: fq_codel defaults (target 5 ms, interval 100 ms, limit 10240 packets); the no-slicing baseline uses pfifo 1000.

Q229. Are the topology parameters measured, from the literature, or hypothetical?

Hypothetical but plausible (fiber vs microwave delays, a 100 Mb/s rural uplink), and swept for sensitivity.

Honest limit. Not calibrated against a specific deployment.

Q230. How do you verify the topology after initialisation?

A connectivity ping (healthcare UE → edge) on start; the first-tick RTT probes for every service; and, post-run, the measured propagation floors equal configuration exactly (8.00 and 28.00 ms).

Q231. How do you identify a failed link and distinguish it from a slow link?

- Failed: carrier loss (operstate) on `s_edge-bh`, seen by the 50 ms link watcher.
- Slow: rising RTT trend for core-bound probes with flat edge probes and *no* local congestion evidence (predictor).
- Congested: drops, backlog or saturation at the local bottleneck.

Q232. What happens if a switch fails instead of a link?

Core-switch failure scenario: all `s_core` ports go down. Carrier loss is seen as a cut, so the engine switches to backup; the backup also ends on `s_core`, so the core stays unreachable. After 2 ticks the diagnosis is "core switch failure" (DEGRADED): core slices are held at their floors and edge slices get the capacity. Diagnosed in 100.0% of runs.

Q233. What if the primary and backup routes share a physical failure domain?

Then the backup does not help, which is exactly our core-switch case (both paths terminate on `s_core`). The system localises the fault and degrades gracefully, but cannot restore core services. Lesson: logical redundancy $\neq$ physical diversity.

Q234. How do you prevent an alternate route from creating a Layer-2 loop?

By construction, with no STP:

1. the backup is admin-down on the `s_edge` side while the primary is up;
2. on failure, the primary is *quarantined* (admin-down) *before* the backup comes up;
3. on fail-back, the backup goes down *before* the primary comes up;
4. only `s_edge`-side ports are driven;
5. the healer ignores link events caused by its own actions.

Evidence. Unit test asserting the exact ordering; no broadcast storms in 27 resilience runs plus the real-OVS runs. **L3 validated (emulation**

Q235. How do MAC learning, ARP and forwarding state affect failover?

After a path change, switches still map MACs to the dead port until they relearn. ARP caches are unaffected (same IP/MAC pairs). Mitigation: flush the FDB on all three switches (`ovs-appctlfdb/flush`, `bridgefdbflush`). The residual OVS outage (0.3–0.5 s) is mostly relearning.

Q236. How do you handle stale forwarding entries after a path change?

An explicit FDB flush on activation and fail-back. Linux bridges also flush entries on a port that loses carrier.

Q237. How do you prove the backup route works before switching traffic to it?

Honest limit. We do not pre-verify the backup. Switching happens, then reachability is checked: if the core is still unreachable after 2 ticks, the engine escalates to DEGRADED.

Better: continuous OAM on the standby (periodic probes over an isolated VLAN, or BFD on the standby link), so a dead backup is known before it is needed.

Q238. Logical redundancy vs physically independent redundancy?

Logical: two paths in the topology graph. Physical: independent conduits, power and end equipment. Our backup is logically redundant but shares the `s_core` failure domain (Q233).

Q239. How would you model a power failure affecting multiple devices?

Take a set of nodes down at once (all ports of the affected switches), as in our core-switch injection, possibly with correlated recovery times. Not implemented as a scenario.

Q240. How would you model partial connectivity rather than complete failure?

Backhaul degradation: netem delay +15 ms and loss +0.3% per tick, then a cut. `inject_latency` also exists. Further options: rate reduction, intermittent flapping, asymmetric loss.

Q241. How do you prevent a topology change from invalidating previously collected telemetry?

Counters are read per interface name and converted to deltas with negative values clamped to 0, so a reset or recreated interface yields one zero sample, not garbage. Datasets carry `run_id` and are never mixed across topologies without that key.

Q242. How would the topology scale to more villages, gateways, services and bottlenecks?

The builder is parametric in UEs per service (tested at 1, 2 and 4) and in capacity. Multiple villages need multiple `s_rural` instances, per-uplink QoS and a regional controller (Q130).

Honest limit. Single-machine emulation will run out of CPU at around tens of nodes with this traffic.

15 TCP/IP, transport behaviour and traffic control

Q243. Why is TCP behaviour relevant to your results?

Elastic TCP flows (library, web, DASH) adapt to whatever share they get, so their demand is not directly observable, and they fill queues (bufferbloat) unless AQM intervenes. The demand estimator's elastic probe-up and the fq_codel choice both exist because of TCP.

Q244. How would TCP congestion control respond to loss or increased delay?

Linux default CUBIC: multiplicative decrease of cwnd on loss, cubic regrowth, largely insensitive to delay until loss. fq_codel turns standing queues into early drops (or ECN marks), so TCP backs off before delay builds.

Q245. Why might throughput fall even when nominal capacity is unchanged?

- rising RTT (backhaul degradation) slows cwnd growth;
- random loss (degradation) triggers backoff;
- receiver window limits;
- HTB ceilings;
- competition from unresponsive UDP in a FIFO queue.

Q246. How does UDP differ from TCP under congestion?

UDP does not back off. In a FIFO queue it can starve TCP and inflate delay for everyone. HTB per-slice classes and fq_codel per-flow queues isolate it.

Q247. How can retransmissions distort application-level latency measurements?

A retransmitted segment's delivery time includes timeout or fast-retransmit delay that a per-packet one-way delay does not show. We match each transmission separately (flagging retransmissions), so our one-way delay is network delay, not application latency.

Honest limit. Application-level latency for TCP (request to response) is not measured.

Q248. What is the difference between RTT and one-way latency?

RTT is the sum of both directions plus host turnaround. One-way delay is a single direction and needs synchronised clocks. Our endpoints share one kernel clock, so the packet-matched one-way delay is exact (validated by the floors). Online probes measure RTT, and monitoring uses RTT/2.

Q249. What is jitter, and how does its definition affect the result?

We report the mean absolute difference of consecutive one-way delays per window (RFC 3550-style, without the 1/16 smoothing). Other definitions (standard deviation of delay, p95−p50) give different magnitudes; we report one definition consistently.

Q250. Packet loss vs queue drops vs application-level delivery failure?

- Packet loss: a packet transmitted but never received (endpoint matching).
- Queue drops: HTB or fq_codel drop counters (a subset of loss).
- Application failure: data not delivered in time. For TCP, loss becomes delay through retransmission; for real-time UDP, loss means missing data.

Q251. What is a Linux qdisc?

A queueing discipline attached to an interface's egress that decides how packets are queued and dequeued. Classless qdiscs (pfifo, fq_codel, netem) and classful ones (HTB) form trees with filters that pick the class.

Q252. How does HTB enforce rate ceilings and borrowing?

Each class has a guaranteed `rate` and a maximum `ceil`. A class under its rate sends freely; between rate and ceil it *borrows* its parent's spare tokens, in `prio` order and quantum-weighted; above ceil it waits. We use rate = allocation and ceil = link capacity (work-conserving) or ceil = rate (strict).

Q253. How do queue limits affect delay and drops?

A larger limit means fewer drops but more standing delay (bufferbloat): 1000 packets at 100 Mb/s is about 120 ms. fq_codel bounds sojourn time instead of length (target 5 ms).

Evidence. Without AQM, the school no-AQM ablation, the OVS queues and the 5G no-slicing mode all show large p95; the 5G no-slicing healthcare p95 was 797.9 ms. **L3 validated (emulation)**

Q254. What is the purpose of netem?

It emulates delay, jitter, loss, duplication and reordering on an interface. We use it for propagation delay and backhaul degradation.

Q255. What are the limits of using traffic shaping to represent network conditions?

Shaping is deterministic and memoryless unless configured otherwise. It does not reproduce wireless loss bursts or link-layer retransmission. Timer granularity and CPU contention add artefacts. Delay placement interacts with other qdiscs (Q221).

Q256. How does FQ-CoDel differ from a simple FIFO queue?

FQ: per-flow hashing into sub-queues with deficit round robin, so sparse flows (ECG, voice) are served ahead of bulk flows. CoDel: drops when sojourn time stays above 5 ms for an interval, which keeps standing queues short. FIFO: one queue, tail drop, no isolation.

Q257. Why can strict priority scheduling cause starvation?

A higher class with continuous backlog always wins, so lower classes never transmit. CARS-SF avoids pure strict priority: each class has a guaranteed rate (floor or allocation), and `prio` only orders borrowing of spare capacity.

Q258. How does weighted fair queuing compare with strict priority?

WFQ shares capacity in proportion to weights (no starvation, bounded delay per weight). Strict priority minimises high-class delay at the risk of starvation. HTB with rates is closer to WFQ with guarantees; HTB prio adds mild priority for spare capacity.

Q259. What is the difference between rate limiting, shaping and policing?

Shaping delays excess packets (queue, smooth output). Policing drops or marks excess immediately (no queue). Rate limiting is the generic term. HTB shapes; we do not police.

Q260. How do you determine whether the QoS policy actually influences the intended class?

Per-class HTB counters (bytes, drops, backlog) per direction each tick; packet-matched per-slice delay and loss; ablations that change one QoS parameter and observe the effect; and the classification tests (u32 filter strings).

Q261. How do you distinguish a configured ceiling from the throughput a flow actually receives?

The configured value is in `controller_history`; delivered throughput per window comes from captures. They differ by demand (a flow may use less), by borrowing (it may use more, up to ceil), and by TCP dynamics.

Q262. How do you inspect and debug interfaces, routes, sockets, qdiscs and OVS rules?

`ip-slink`, `iproute`, `ss-tinp`, `tc-sqdisc/class/filtershowdevX`, `ovs-vsctlshow`, `ovs-ofctldump-flows/dump-ports`, `ovs-appctlfdb/show` and `ofproto/trace`, `tcpdump-iX`, `bridgefdbshow`. Inside Mininet: `mnexec-a<pid>`.

Q263. The controller claims a route changed, but packets still use the old path. How do you debug it?

1. Check the port states (`iplink`) on both ends.
2. Check the counters on both links (`ip-slink`) to see which one carries the bytes.
3. Inspect the FDB (`ovs-appctlfdb/shows_edge`): stale MAC entries pointing to the old port are the usual cause.
4. Use `ofproto/trace` for one packet.
5. Run tcpdump on both candidate links.

Fix: flush the FDB, or check that the quarantine actually took effect.

Part VI

Telemetry, monitoring and feature engineering

16 Telemetry collection

Q264. What network metrics does CARS-SF observe at runtime?

Per 1 s tick (`telemetry/collector.py`):

- per-service offered uplink and delivered downlink (access-port counters);
- uplink, downlink and max utilisation of the bottleneck;
- backhaul and backup throughput;
- HTB class sent bytes and packets, drops and backlog, per direction;
- per-service RTT (ICMP probes);
- link carrier state and timestamped transitions (50 ms watcher);
- OVS flow bytes and counts (OVS only);
- host CPU and memory.

Q265. Which metrics come from OVS counters, tc, active probes or application-level measurement?

OVS: flow bytes and counts. tc: per-class bytes, drops, backlog. Probes: RTT and reachability. Kernel interface counters: rates and link state. Application-level (per-flow one-way delay, loss, throughput): only post-run from captures, plus live captures for the in-loop classifier.

Q266. Why is a single telemetry source insufficient?

Counters give rates but no delay. Probes give delay but no per-class volume. HTB stats show drops but not their cause upstream. Link state shows cuts but not degradation. The predictor needs RTT trends *and* local queue evidence to tell congestion from backhaul degradation; that confusion caused a false failover on OVS until both signals were combined.

Q267. How often is telemetry collected, and why that interval?

- 1 s ticks: enough to see 5 s control decisions, cheap, and well above timer noise.
- 50 ms link polling: carrier loss must be acted on in under 1 s.
- Probes once per second per service, in a background thread so a lost probe (1 s timeout) never blocks a tick.

Q268. Trade-off between sampling frequency, detection speed, CPU overhead and noise?

Faster sampling detects sooner but costs CPU (a `tc` fork per read) and measures noisier rates over shorter intervals. We put fast sampling only where speed matters (link state, as an event) and keep aggregate rates at 1 s.

Q269. Are counters cumulative or instantaneous?

Kernel interface and HTB counters are cumulative. Backlog is instantaneous. Telemetry converts cumulative counters to per-tick deltas and rates.

Q270. How do you calculate a per-interval byte rate from cumulative counters?

$r = 8\,\max(B_t - B_{t-1}, 0)/\Delta t/10^6$ Mb/s, with Δt the measured wall-clock interval between reads (not the nominal 1 s).

Q271. How do you handle counter resets, recreated interfaces, wraparound or a switch restart?

Negative deltas are clamped to 0 (one zero sample). Sysfs counters are 64-bit, so wraparound is irrelevant. A missing interface is skipped, its key absent, so the rate reads 0.

Honest limit. A restart therefore shows as a short zero, not as an explicit "telemetry gap".

Q272. What happens when a counter decreases unexpectedly?

The delta is clamped to zero for that interval and the next interval uses the new baseline. It is not logged as an anomaly.

Q273. How do you calculate utilisation when capacity is known?

$\max(r_{up}, r_{down})/C$: the rural uplink is full-duplex, so it is saturated if either direction is.

Honest limit. Until the real-OVS validation, utilisation counted only the uplink, which made downlink-heavy contexts (school) look about 17% utilised. It was fixed and the whole matrix re-run.

Q274. How do you distinguish offered, forwarded, queued and dropped traffic?

Offered: access-port rx (uplink) or HTB sent+dropped (downlink). Forwarded: HTB sent. Queued: HTB backlog. Dropped: HTB drops (queue) and, post-run, matched-packet loss (end to end).

Q275. How are per-flow, per-class, per-link and per-slice measurements aggregated?

Per class = per slice (one HTB class per slice per direction). Per link: interface counters. Per flow: only from captures, in 5 s windows. Slice KPIs aggregate flow windows (p95 of per-window p95, mean loss, summed delivered rate).

Q276. Can aggregate utilisation conceal severe congestion on one link?

Yes, and it did. Uplink-only utilisation hid downlink saturation (Q273). Likewise, a link at 60% average can still have one saturated class. That is why decisions use per-class drops and backlog, not just utilisation.

Q277. How do you synchronise measurements from different interfaces?

All readers run in one process within milliseconds on one host clock, and each snapshot carries a timestamp. Probe results are asynchronous: the snapshot holds the *latest completed* probe (at most about 1 s old).

Q278. How do you handle delayed, missing, duplicated or out-of-order samples?

Ticks are wall-clock aligned and deltas use measured intervals, so late ticks are handled correctly. A missing probe becomes `None`. Duplicates and reordering cannot occur within one process.

Honest limit. There is no explicit staleness bound on probe results.

Q279. How do you distinguish zero traffic from unavailable telemetry?

For probes: `None` means unavailable, distinct from a measured RTT.

Honest limit. Interface and HTB readers return 0 or empty on error, which is indistinguishable from zero traffic. The fix is to return `None` and carry a per-source health flag.

Q280. How do you detect stale telemetry?

Honest limit. Not implemented. If the probe thread died, the snapshot would keep reporting its last RTT.

Fix: timestamp each probe result and treat results older than 2 intervals as missing; add thread liveness checks.

Q281. What happens if the monitoring process fails?

Monitoring runs inside the orchestrator process, so a crash ends the run. The kernel keeps the last QoS configuration (enforcement survives) and Mininet is torn down by `finally`.

Honest limit. There is no supervisor or restart.

Q282. How do you validate telemetry against independent measurements?

- Probe RTT/2 matches the packet-matched one-way delay floor (16 ms RTT ↔ 8 ms one-way).
- Per-service counter rates match the dataset's delivered throughput (same order per tick).

- Delay floors match configuration exactly.
- Unit tests feed synthetic captures with known delays and losses.

Q283. How do you avoid treating a measurement artefact as a real fault?

- Faults are confirmed by physically meaningful signals (carrier loss, sustained trends over a 6-sample window, congestion evidence).
- The healer ignores self-inflicted link events within 1 s of its own actions.
- Methodological artefacts found during validation were fixed and all runs re-done: busy-wait CPU starvation, the partial final window, uplink-only utilisation, and false failover from AQM-less queues.

Q284. Which measurements are unavailable because there is no physical RAN?

CQI, SINR, RSRP and RSRQ, PRB utilisation, HARQ and BLER, handover events, radio link failures. In the dataset these columns are scenario-injected placeholders, excluded from ML.

17 Feature extraction

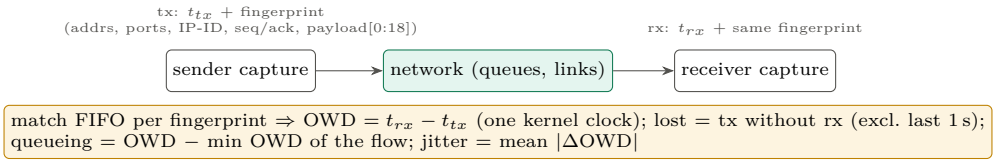


Figure 5: Packet-matched one-way delay and loss (`features/packet_capture.py`).

Q285. Why do you need a feature-extraction stage?

Classifiers and analysis need fixed-size per-flow descriptors over a time window. Raw packets are variable-length, huge (about 100 MB of captures per run) and directional. Extraction also produces the QoS ground truth (one-way delay, loss).

Q286. What are the raw inputs and outputs of the feature extractor?

In: endpoint pcaps (Ethernet or raw IP), UE addresses, spawn metadata (labels), telemetry snapshots. Out: one record per (application flow, UE, 5 s window) mapped to the 89-column schema plus measured extras, 31 `shape_*` features, and a 3×32 packet-sequence tensor (`features/extractor.py`).

Q287. Which features are computed at Layers 2, 3 and 4?

- L2: ether type, VLAN and broadcast flags. These are constant in emulation and kept only for schema compatibility.
- L3: IP lengths (size statistics), TTL, DSCP.
- L4: ports, TCP flags (SYN, ACK, FIN, RST), window size, retransmissions, UDP counts.

ML uses size, timing, direction and flag counts, but *not* ports, DSCP or TTL (shortcuts).

Q288. Which features are flow-level, statistical, temporal or derived?

- Flow-level: packet and byte counts per direction, duration.
- Statistical: size and IAT mean, std, min, max, skewness, kurtosis.
- Temporal: active and idle time, bursts and burst length, per-direction IAT.
- Derived: rates, byte share, small-packet share, coefficient of variation.
- QoS: one-way delay, p95, jitter, loss, queueing delay (labels and analysis only).

Q289. Why are duration, sizes, rates and inter-arrival statistics useful?

Services differ in them: ECG and IoT are small, periodic and regular; video is large, bursty and frame-periodic; VoIP has fixed small packets with talkspurts; bulk TCP is MTU-sized with ACK flows; web is request/response. The top RF features are forward mean size, forward byte share, backward packet count and burst count.

Q290. Which features are available before a flow completes?

All of them. Records are *windowed* (5 s), so they exist from the first window onward, and online inference uses the flow's history so far (at least 5 packets).

Q291. Which features require observing the flow over a time window?

Rates, IAT statistics, burst structure, idle and active time, byte share. The 32-packet sequence needs only the first 32 packets.

Q292. How does window length influence accuracy and decision latency?

Longer windows give more stable statistics but decide later and blur context changes; shorter windows decide earlier with noisier features.

Honest limit. Only 5 s was evaluated. A sweep (1, 2, 5, 10 s) of F1 against time-to-decision is a clear gap.

Q293. How do you handle short flows with insufficient observations?

Offline: windows with fewer than 2 packets are dropped. Online: a flow is classified only once it has at least 5 packets; until then it stays in the default (general) class, which is safe but not optimal.

Q294. How do you handle missing or invalid feature values?

Delay is `NaN` when nothing was delivered in a window (counted by the validator's measurement-coverage score). Classifiers fill `NaN` with 0; the CNN applies a sign-log transform and z-scores fitted on training data.

Q295. How do you prevent feature definitions diverging between dataset generation and runtime inference?

One implementation (`features/shape.py`) serves both. The model pickle stores its column list and the CNN stores its normalisation and column order. The ML-feature list lives in `registries/features.yaml` and is mirrored in `schema_mapper.py`.

Q296. How do you detect unit inconsistencies (bytes vs bits, s vs ms)?

Units are in every name, and conversions are explicit at one place each (`*8/1e6`, `*1000`). Unit tests assert known values (8 ms one-way delay, 10% loss, 2 ms jitter on synthetic captures). The exact propagation floors are an end-to-end unit check.

Q297. How do you handle highly correlated features?

They are not removed: random forests and CNNs tolerate collinearity, and interpretability claims are limited to importance rankings.

Honest limit. The importance of correlated features is split among them, so do not over-interpret individual importances.

Q298. How do you decide whether a feature is informative or a dataset artefact?

- Exclude known shortcuts a priori (ports, DSCP, TTL, timestamps, scenario fields, QoS outcomes).
- Evaluate on held-out run×UE groups.
- Check that top features are physically meaningful (sizes, direction share, bursts).

The strongest available artefact test was the transfer study: synthetic features did not carry over (F1 0.104).

Q299. What is the cost of extracting each feature at runtime?

Shape features are $O(n)$ numpy operations over a flow's packets, well under a millisecond per flow. The cost driver online is re-parsing the growing UE capture every interval (O(total packets)).

Honest limit. Incremental parsing is needed for long runs.

Q300. Which features risk exposing payload content or sensitive information?

Features themselves are metadata (sizes, timing), but traffic analysis can still reveal activities.

Honest limit. The 128 B capture includes about 60 B of payload, used only to fingerprint our own generator packets for delay matching. A real deployment must capture headers only and hash fingerprints.

Q301. How would you run a feature ablation to see which features actually matter?

Group-wise drop-column importance under group hold-out: remove each family (size, IAT, direction, burst, flags), retrain, and record ΔF1 with seed variance. Permutation importance gives a quicker check.

Honest limit. Only RF impurity importances exist today.

Q302. What happens when a feature becomes unavailable after a topology or instrumentation change?

Honest limit. Inference re-indexes columns and fills missing ones with 0, which degrades silently.

Fix: fail loudly when expected features are missing, version the feature set with the model, and retrain.

Part VII

RSNSD: dataset design, generation and validation

Two datasets: keep them apart in every answer

RSNSD v1 (`RSNSD/`): 108,000 *synthetic* flow records, 89 columns, 32 applications, 19 scenarios. Generated from parametric traffic and QoS models with no network in the loop; frozen.
Measured dataset (`results/runs/*/dataset.csv`): flow windows matched packet by packet from emulated runs. SliceNet's training set is 4,697 windows from 18 collection runs.
Most of the hard questions below are answered more strongly for the measured dataset. RSNSD's honest role is a schema and bootstrap: it does *not* transfer to measured traffic (zero-shot macro-F1 0.104).

18 Dataset motivation and schema

Q303. What does RSNSD stand for?

Rural Service-aware Network Slice Dataset.

Q304. What precise problem does the dataset solve?

It provides labelled examples answering "which *service* is this traffic serving (and hence which slice should it get)?" rather than "which protocol produced it?", for rural service classes under varying contexts, where no public labelled data exists.

Q305. Why was a dedicated rural, service-aware dataset necessary?

Public traffic datasets label applications or protocols (often encrypted-traffic benchmarks) or technology slices (the KPI table used by [1]). None labels rural *public-service* classes or carries context.

Honest limit. Necessity justifies a dataset; it does not justify a *synthetic* one being representative.

Q306. Which existing datasets did you evaluate before creating RSNSD?

The KPI-based slice dataset used by [1] (device and slice-type labels, about 63k rows).

Honest limit. A broader survey of public traffic-classification datasets was not documented. List the ones considered and why each was rejected (wrong label space, no context, no QoS) before the defense.

Q307. What are the dataset's observation unit, label space and intended tasks?

RSNSD: one synthetic *flow*. Measured: one application flow of one UE over a 5 s window. Labels: `service_class` (4), plus `application_id`, `slice_type_label`, `priority_level`, `sla_satisfied`. Tasks: service classification (primary), SLA-satisfaction prediction, policy learning.

Q308. Is one record a packet, a flow, a time window, an application session or an aggregate?

RSNSD: a flow. Measured: a (flow, UE, window) triple; a long-lived flow yields several records.

Q309. Service label vs application identity vs QoS class vs slice assignment?

- Service: social purpose (healthcare).
- Application: concrete app (`healthcare.ecg_streaming`).
- QoS class: requirement bundle (latency, loss).
- Slice assignment: the resource partition chosen (`1:10` or URLLC).

CARS-SF maps service → slice one-to-one; 3GPP maps application → slice type.

Q310. Which fields are raw observations, derived features, metadata or target labels?

Recorded in `registries/features.yaml` provenance:

- `PCAP` / `PCAP_MATCHED`: measured.
- `TC`, `OVS`, `PSUTIL`: telemetry.
- `CONTEXT_INJECTED`: radio and context fields.
- Labels: service, application, slice type, priority, SLA.

RSNSD labels the same columns, but its QoS fields come from models, not measurement.

Q311. How are Layer 2–4 fields represented?

As numeric columns per flow (counts, sizes, flags, ports, TTL, DSCP). L2 fields are constant in emulation. See Q287 for which ones are used in ML.

Q312. How are timestamps and temporal features encoded?

Absolute epoch start and end times (excluded from ML as shortcuts); durations, IAT statistics and active/idle times in ms; a window index. RSNSD also carries a categorical `time_of_day` (excluded).

Q313. What is the schema versioning strategy?

RSNSD: v1.0, frozen (`CHANGELOG.md`, `CITATION.cff`, `schema.json`). Measured dataset: an 89-column core plus measured extras and `shape_*` columns, versioned by git commit per run.

Honest limit. The measured schema has no explicit version string.

Q314. How do you maintain backward compatibility when adding features?

New columns are appended, never renamed. Consumers select columns by name, and the classifier stores its own list.

Q315. How are missing, invalid, duplicate and malformed records handled?

- Windows with fewer than 2 packets are not emitted.
- `NaN` delay is explicit and counted by the validator.
- The physical-consistency check catches goodput above throughput, loss outside [0,100] and negative delays.

- Truncated pcap tails are tolerated by the parser.

Q316. What constitutes a valid record?

A known service label; at least 2 packets; non-negative delays; loss in [0,100]; goodput $\leq$ throughput; required schema columns present.

Q317. How do you identify duplicate flows or duplicated generated samples?

Measured: flows are keyed by (UE IP, server IP, protocol, application port) and window. Unique packet fingerprints make duplicate matches impossible.
Honest limit. RSNSD was not de-duplicated for near-identical synthetic flows.

Q318. How are provenance and generation parameters recorded?

Per run: `run_meta.json` (parameters, seed, git hash), `summary.json` (spawn metadata, allocations) and the generator logs. Per column: the provenance map. RSNSD: generator code plus metadata reports.

Q319. How do you distinguish the release dataset from intermediate datasets?

RSNSD release: `RSNSD/dataset/raw/rsnsd_v1.0.csv`, regenerated by `makedata`. Measured release: `generated/datasets/mininet_full_dataset.\{csv,npz\}` merged from runs prefixed `collect__`. Matrix runs are evaluation data, not training data.

Q320. What guarantees that the dataset can be regenerated?

Deterministic generators (RSNSD seeds; traffic seeds), a pinned topology and code version, and `configs/experiments/dataset.yaml`.
Honest limit. Measured windows are not bit-identical across reruns, because OS timing varies. Distributions are.

19 Dataset generation and label quality

Q321. How are RSNSD records generated?

For each scenario and application profile, `RSNSD/traffic_generator` samples flow characteristics from distributions, `RSNSD/context` adds context fields, and `RSNSD/qos` models compute latency, jitter, loss, throughput and SLA. The exporter writes CSV and Parquet.

Q322. What is the exact pipeline from scenario configuration to final CSV?

- RSNSD: `generate_release.py`: balanced profile weights $\rightarrow$ dataset builder (flow + context + QoS generators) $\rightarrow$ feature extractor $\rightarrow$ CSV exporter $\rightarrow$ validation reports.
- Measured: matrix config $\rightarrow$ run (topology, traffic, captures) $\rightarrow$ packet matching $\rightarrow$ windows $\rightarrow$ schema mapping $\rightarrow$ `dataset.csv` + `sequences.npz` $\rightarrow$ merge.

Q323. How do the context engine and traffic generators influence the dataset distribution?

Context sets the demand scaling (rates) and, for timelines, switches traffic mid-run. Across contexts, the same application appears at different rates, which is exactly the variation a service classifier must be robust to. Collection covered 6 contexts $\times$ 3 seeds.

Q324. How are ground-truth service labels assigned?

From the generator identity. Each flow is matched to the spawn record that created it, by (UE IP, server IP, protocol, application port).

Q325. From generator identity, observed behaviour, ports, payload, or another mechanism?

Generator identity (configuration). The port is used only to *match* a flow to its generator, never as a feature.

Q326. If labels come from the generator configuration, what shows the traffic realistically represents the named service?

Nothing establishes realism beyond the plausibility of the model parameters (codec framing, frame rates, IoT periods).
Honest limit. Labels are correct by construction; their *representativeness* is unvalidated. This is the main threat to the external validity of SliceNet results.

Q327. Can two different services produce statistically similar traffic?

Yes. Education web (exam, LMS) and general web are both request/response TCP; agricultural drone video and telemedicine video are both VBR uplink. The confusion matrix shows exactly these confusions (education and general F1 0.902 and 0.926 vs healthcare 0.979).

Q328. Can one service produce several different traffic patterns?

Yes. Each service mixes 3–4 models (healthcare: video, periodic, VoIP, on/off). A per-window classifier must recognise all of them as one class.

Q329. How are those cases represented in the dataset?

As records with the same service label but different application ids. Per-class F1 exposes the difficulty, and the `application_id` column allows per-app analysis.

Q330. What are the class counts and imbalance ratios?

RSNSD: healthcare 33%, general 31%, agriculture 19%, education 16% (about 2:1 maximum ratio). Measured: roughly balanced per run (validator class scores are full marks), but low-rate IoT produces fewer windows. Training uses balanced class weights; reports give macro-F1.

Q331. Why are the chosen service classes mutually distinguishable?

Because the applications we assigned to them differ in direction, size and timing. That is a property of our application mixes, not a law of nature (Q327).

Q332. What happens when a record belongs to a previously unseen service?

The classifier still outputs one of the four classes (closed set).

Honest limit. There is no open-set rejection. Proposed: confidence threshold plus default class (Q396).

Q333. How do you prevent the dataset from encoding scenario identity as a shortcut for service identity?

Scenario-injected columns (network load, radio fields, time of day) are excluded from ML. Collection spans 6 contexts, so each service appears under several. Group hold-out is by run, so test runs have unseen seeds.

Q334. Could a classifier learn generator-specific artefacts instead of meaningful behaviour?

Yes. For example, exact packet sizes chosen in our models (1200 B video chunks, 172 B voice) are generator choices. Excluding ports and DSCP removes the crudest shortcuts but not these.

Honest limit. The F1 is an upper bound for real traffic. The transfer gap from RSNSD (0.104) shows how strongly generator choices shape what a model learns.

Q335. How do you test for such shortcuts?

Done: excluding obvious shortcut columns; group hold-out; inspecting importances; the cross-generator transfer test (RSNSD → measured). To do: leave-one-application-out, perturbing size distributions at test time, and evaluating on real traces.

Q336. What is the difference between realistic traffic and statistically diverse traffic?

Diverse means the generator explores a wide space. Realistic means its distribution matches real users. Seeds and contexts give diversity; only real data gives realism.

Q337. How do you establish external validity for a synthetic dataset?

By comparison against real traces of the same services (distributional tests on size, IAT and rates) and by testing a model trained on synthetic data on real data. We ran the second test synthetic-to-*emulated*, and it failed.

Q338. What evidence would be needed before claiming RSNSD represents actual rural traffic?

Labelled real captures from rural clinics, schools and farms; distribution-matching statistics; a classifier trained on RSNSD reaching acceptable F1 on them. None exists, so we make no such claim.

Q339. How would you incorporate real traces without compromising privacy?

Header-only capture, IP anonymisation (prefix-preserving, Crypto-PAn style), dropping payload, aggregation to flow windows on site, consent and ethics approval, and releasing only features, not packets.

Q340. What licence, attribution, documentation and reproducibility requirements apply to a public release?

An open data licence (e.g. CC-BY 4.0) with `CITATION.cff`; a datasheet (motivation, composition, collection, synthetic provenance, known limitations including the transfer result); generator code under MIT; a schema with provenance; and checksums.

20 Data validation and leakage

Q341. What validation checks are performed before a dataset is accepted?

`DatasetValidator`, scored out of 100 with a pass mark of 80:

- schema completeness (20);
- class coverage and balance (15);

- physical consistency (20);
- measured-delay coverage (15);
- load–delay relationship (15);
- feature quality: constant columns and outliers (15).

Median across runs: 84.

Q342. How do you verify data types, ranges, required fields and label consistency?

Required columns come from `features.yaml`; range checks on loss, delay and goodput; labels must be in the four classes (others are dropped before training).

Honest limit. dtype checks are implicit (pandas coercion), not declared.

Q343. How do you test expected feature distributions?

Generator unit tests check the long-run rates. Constant-column detection runs in the validator.

Honest limit. There are no per-feature distribution baselines or drift tests between datasets.

Q344. How do you identify outliers, and decide whether they are errors or meaningful rare events?

3×IQR fences per numeric feature contribute to the score but are never removed. Physical impossibilities are errors; heavy-tailed bursts are real. Pareto on/off and web traffic legitimately produce outliers.

Q345. How do you detect impossible correlations or inconsistent QoS values?

Consistency rules (goodput $\leq$ throughput, non-negative delays, loss bounds), and a sign check that queueing delay rises with utilisation (Spearman).

Q346. How do you validate that the exported CSV matches the underlying generated records?

The extractor's unit tests on synthetic pcaps check record values exactly (counts, delay, jitter, loss). Every run also writes `sequences.npz` aligned row by row with `dataset.csv`, and a length mismatch is rejected at merge.

Q347. How are train, validation and test sets constructed?

Measured: `GroupShuffleSplit` on run×UE groups (25% of groups held out). No separate validation set; hyper-parameters were fixed a priori (Q379). RSNSD: no run or UE keys exist, so the split is a **random row split**.

Honest limit. The RSNSD baseline F1 (0.946) is therefore optimistic. The label was corrected in the code (`split_description`).

Q348. Why can a random row split produce misleadingly high performance?

Windows from the same flow, UE and run share near-identical statistics. A random split puts siblings on both sides, so the model is graded on near-copies of training data.

Q349. How would you split by flow, scenario, time, topology or seed?

Flow: group by (run, UE, app). Scenario: leave-one-context-out. Time: train on early windows, test on later ones. Topology: train at 2 UEs or 100 Mb/s, test at 4 UEs or 50 Mb/s. Seed: by run. We used run×UE.

Q350. How do you prevent near-duplicate flows from crossing the train/test boundary?

Group hold-out by run×UE guarantees that windows of one flow never cross the boundary.

Q351. How do you detect target leakage?

An a-priori audit of every column's provenance; exclusion of labels, identifiers, injected context, ports, DSCP and timestamps; and checking that held-out performance is not suspiciously perfect (healthcare is 0.99, so we looked: its traffic is genuinely distinctive, being uplink-dominant with small periodic flows).

Q352. Could slice identifiers, generator identifiers or scenario IDs reveal the label?

Yes, which is why `network_slice_id`, `application_id`, `slice_type_label`, `priority_level`, `ue_host`, `run_id`, `scenario_id` and the injected fields are all in `EXCLUDED_FROM_ML` (a unit test asserts it).

Q353. Are any features computed from information only available after classification?

QoS outcomes (delay, loss, throughput) depend on the slice the flow was *put in*, so they are excluded from SliceNet features. Shape features are pre-classification observables.

Q354. How do you ensure the same preprocessing is applied in training and inference?

Shared feature functions; the RF uses raw values (no fitted scaler); the CNN stores its sign-log plus z-score parameters with the model; the column order is stored and enforced by re-indexing.

Q355. How do you compare RSNSD against a simple baseline or public benchmark?

Internally: the RSNSD-trained RF scored on measured data (transfer curve, Fig. 7).

Honest limit. There is no public-benchmark comparison (e.g. reproducing [1] on its dataset).

Q356. What are the dataset's limitations in representativeness, class coverage and external validation?

Synthetic (RSNSD) or emulated (measured) traffic from our own models; four classes; no unknown class; no real rural traces; no radio measurements; contexts limited to the registry; transfer from RSNSD fails.

Part VIII

SliceNet and the machine-learning pipeline

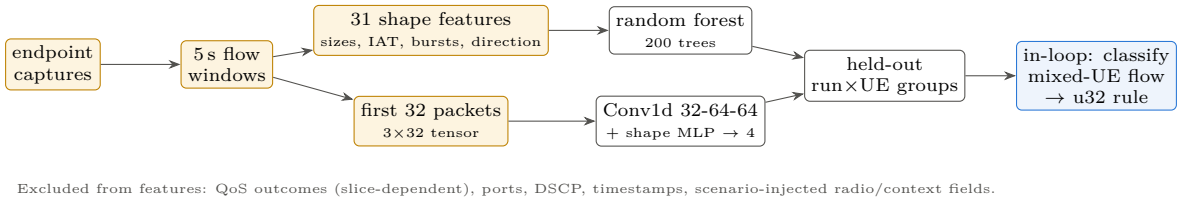


Figure 6: SliceNet: features, models, evaluation protocol and in-loop use.

21 Why machine learning?

Q357. What specific decision does SliceNet make?

Given a flow's traffic shape (31 statistics over a window and the first 32 packets) at one vantage point, it predicts its service class (healthcare, education, agriculture, general). In the loop, that prediction picks the HTB class a flow from a multi-service device enters.

Q358. Why classify services if the traffic generators already know which service produced each flow?

The generator's knowledge is *ground truth for evaluation*, never given to the controller. In deployment the network does not know the application. For single-institution UEs, the address is enough (host mode). SliceNet is for devices whose address does not identify the service (shared kiosks, gateways, NATed hotspots).

Q359. Is SliceNet for the simulator, a future live deployment, or both?

Both. It is evaluated offline on measured windows and runs live in emulation (in-loop, mixed UE, L3). A live real-network deployment is future work (L4 absent).

Q360. Why not use source IPs, ports, application metadata or static rules?

Those are better when available: the host and app modes *are* such rules and are the defaults. Ports are unreliable in practice (shared 443, NAT, dynamic ports) and DSCP is untrusted from UEs. ML is reserved for when metadata is missing or untrustworthy.

Q361. When would a rule-based classifier beat ML?

- When identity is in trusted metadata (the address of a clinic's device, an S-NSSAI in 5G);
- when traffic is encrypted but endpoints are known;
- when the deployment's traffic differs from training (ML degrades; rules do not);
- when explainability is required.

In this thesis, host rules equal SliceNet in disaster (1.000 vs 1.000) for the mixed UE.

Q362. What does a classification error cost downstream?

The flow gets another slice's allocation and AQM. Critical traffic misclassified as general loses its priority share and floor; bulk traffic misclassified as healthcare can borrow healthcare capacity and inflate its queue.

Q363. Which errors are more dangerous: false critical, or missed critical?

Missed critical traffic violates a life-relevant SLA. A false "critical" label lets bulk traffic consume the critical slice, which can also violate it. Both are dangerous; the second can be capped by per-flow policing inside the critical class.

Honest limit. Recall on healthcare is high (F1 0.979) but the asymmetric cost is not built into the decision rule.

Q364. What is the simplest baseline SliceNet should be compared with?

The majority class (macro-F1 0.25 for 4 balanced classes); a direction-share rule (uplink-dominant → healthcare or agriculture); a port oracle (app mode, 100% by construction).

Honest limit. Only the RSNSD-trained model and the RF/CNN pair were formally evaluated. Add a single-feature rule baseline.

Q365. Why this model architecture?

Why. Two representations: aggregate statistics (tabular) and the early packet sequence (temporal).

Choice. An RF for the tabular statistics (strong with little data, robust to scale and collinearity) and a small 1D-CNN fusing a packet-sequence branch with a shape MLP (local patterns such as frame bursts and request/response alternation).

Alternative. Logistic regression (linear: too weak for multimodal classes), XGBoost (likely similar to RF), LSTM or Transformer (more data-hungry).

Trade-off. The RF won (0.947 vs 0.858) at about 14× the inference latency of the CNN.

Evidence. Held-out-group evaluation, 1,207 test windows. **L3 validated (emulation)**

Q366. Why not logistic regression, random forests, XGBoost, an MLP or another deep architecture?

RF is used. Logistic regression is a good sanity baseline but cannot express the multimodal classes. XGBoost would likely tie the RF. An MLP is subsumed by the CNN's shape branch. Larger deep models are not justified at 4,697 windows.

Honest limit. Logistic regression and XGBoost were not run.

Q367. What evidence supports the architecture being suitable for the features?

Empirically: the RF beats the CNN, which suggests the discriminative information is in aggregate statistics more than in early packet order. The name "SliceNet" comes from prior work [1], and we do *not* claim the CNN is best.

Q368. What is the input tensor or feature vector?

RF: 31 shape features (`SHAPE_FEATURES`). CNN: a 3×32 tensor (signed size/1500, $\log(1 + \mathrm{IAT}_{ms})$, TCP-control flag) plus the 31-feature vector.

Q369. What are the output classes?

Healthcare, education, agriculture, general (closed set).

Q370. How are features encoded, normalised or scaled?

RF: raw values, `NaN`→0. CNN: $\mathrm{sign}(x)\log(1+|x|)$ followed by a z-score with training mean and std; sequences are pre-scaled by construction.

Q371. Where are the fitted preprocessing parameters stored?

Inside the model artefacts: the RF pickle stores its column list; the CNN `.pt` stores the mean, std, column order and state dict.

Q372. How is model versioning handled?

By file names (`slicenet_baseline.pkl`, `slicenet_mininet.pkl`, `slicenet_cnn.pt`) plus metrics JSONs with timestamps; models are gitignored and regenerated by `makemodels`.

Honest limit. There is no model registry and no hash link between a model and its training data.

Q373. What happens if the model file is missing or incompatible?

In-loop classification is disabled (`enabled=False`), so flows stay in host or default classes. The post-run scorer falls back to the RSNSD model and records which one was used.

Honest limit. An incompatible pickle would raise at load.

Q374. How do you prevent inference from using a different feature order than training?

Both models re-index input columns to the stored order before predicting.

22 Training and model evaluation

Q375. What is the exact training procedure?

1. Merge the collection runs.
2. Drop non-service labels.
3. Group split (25% of run×UE groups held out, seed 42).
4. RF: 200 trees, max depth 16, balanced class weights.
5. CNN: 80 epochs, Adam (lr 10^{-3}, weight decay 10^{-4}), batch 128, class-weighted cross-entropy.
6. Evaluate on the held-out groups.

`classification/pipeline.py`.

Q376. Which loss function and optimiser, and why?

Class-weighted cross-entropy (standard for multi-class, counteracts imbalance) with Adam (robust defaults for small CNNs).

Q377. How are learning rate, batch size, epochs and regularisation selected?

Fixed standard defaults, not tuned. Epochs were raised from 40 to 80 after the training loss had not plateaued. Regularisation: batch norm, dropout 0.3, weight decay.

Honest limit. No hyper-parameter search, which favours neither model.

Q378. How do you determine whether the model is overfitting?

By the gap between training loss (0.23 at 80 epochs) and held-out performance, and by stability under group hold-out.

Honest limit. There was no validation curve. Add a validation split carved from the training groups.

Q379. What is the role of the validation set?

For tuning and early stopping. We had none because no tuning was done; the held-out groups are a pure test set.

Q380. When do you stop training?

After a fixed 80 epochs. No early stopping.

Q381. How do you select a classification threshold, if any?

Arg-max over four classes. There is no threshold and no reject option (Q396).

Q382. What are the accuracy, precision, recall, F1 and confusion-matrix results?

- RF: macro-F1 0.947, accuracy 0.940. Per class: healthcare 0.979, education 0.902, agriculture 0.981, general 0.926.
- CNN: macro-F1 0.858, accuracy 0.857.

Confusion matrices: `docs/figures/thesis/slicenet_confusion_*`. Most confusion is between education and general, which share web request/response models.

Q383. Which metric matters most for your application, and why?

Per-class recall of healthcare (a missed critical flow loses its SLA), then macro-F1 (every class counts equally, regardless of window counts).

Q384. How does performance differ across classes?

Healthcare and agriculture are near-perfect (uplink-dominant, distinctive periodic and video patterns). Education and general are lower because both have downlink web and streaming traffic.

Q385. How does class imbalance affect the reported metrics?

Accuracy would over-weight the frequent classes (low-rate IoT yields fewer windows). We report macro-F1 and use balanced class weights.

Q386. What is the difference between macro-F1 and weighted-F1?

Macro: the unweighted mean of per-class F1 (each class equal). Weighted: weighted by class support, so it is dominated by large classes.

Q387. Why is accuracy alone insufficient?

It hides class-specific failure. A classifier that never predicts the rare critical class can still score high accuracy.

Q388. How do you quantify uncertainty in the reported results?

Honest limit. Single group split, so there is no CI on F1 yet.

Fix: repeated `GroupKFold` (5 folds × 3 seeds), reporting the mean ± CI. A bootstrap over test groups gives a CI cheaply.

Q389. How much does performance vary across random seeds or data splits?

Not measured for the final models (Q388). An indirect signal: the RF was 0.959 on the earlier 12-run dataset and 0.947 on the 18-run dataset; the CNN was 0.880 vs 0.858. So the variation is about ±0.01.

Q390. How do you decide whether an improvement is statistically meaningful?

With repeated group CV, compare models with a paired test across folds (Wilcoxon or a corrected resampled t-test). The RF-vs-CNN gap (about 0.08) is likely real given the ±0.01 variation, but is untested.

Q391. What is the model's inference latency?

CNN: 3.3 ms for a single flow, 4.5 ms for a batch of 100 (CPU). RF: 47 ms single, dominated by thread-pool start-up (`n_jobs=-1`). Setting `n_jobs=1` would make the RF sub-millisecond.

Q392. What is the model's memory and CPU footprint?

CNN: 29,764 parameters (131 KB). RF: 6.1 MB on disk. Both run on CPU only.

Q393. How does performance change when traffic patterns shift?

The extreme shift we tested (RSNSD synthetic → measured) collapses performance to 0.104. Within-family shift (new seeds, contexts and runs) keeps 0.947.

Honest limit. Moderate shifts (new apps, other codecs) are untested.

Q394. How would you test generalisation to unseen scenarios, generators or network conditions?

Leave-one-context-out, leave-one-application-out, train at 100 Mb/s and test at 50 Mb/s, train on Linux bridges and test on OVS runs, and ultimately test on real traces.

Q395. How would you detect concept drift?

Monitor the distribution of predicted classes and confidences per site, and feature drift (KS or PSI on key shape features) against training. Periodic labelled audits. Trigger retraining.

Q396. What is the fallback when the model is uncertain or meets an unknown class?

Today: none. Arg-max always assigns a class, but a flow below 5 packets stays in the default (general) class. Proposed: if the maximum probability is below τ, keep host or default treatment and re-try with more packets (safe default).

23 Runtime inference and deployment

Q397. Is SliceNet actually invoked by the operational allocation and recovery path?

Only with `classification=slicenet`, for unknown or mixed UEs: `OnlineSliceClassifier.step` runs every control interval and installs per-flow u32 rules. Allocation and recovery never use it. They operate on slices, and SliceNet only decides which slice a flow enters.

Q398. If not, what determines service identity on that path?

By default, host rules (UE address → service), installed at QoS setup. In 3GPP mode, app rules (server and port → slice type).

Q399. How would you integrate live feature extraction and inference into the control loop?

It is already integrated for the mixed UE. Read the UE capture, group packets into flows, compute shape features and sequences, predict, call `classify_flow` (`classification/online.py`). For scale: an incremental flow table (not re-parsing the pcap), done at the gateway.

Q400. How is a predicted class mapped to a service policy?

Class name → HTB classid (from `slices.yaml`) → a u32 filter on the 5-tuple, in both directions, at the highest filter priority. The flow then gets that slice's allocation and AQM.

Q401. How do you prevent a wrong prediction from granting unlimited priority to non-critical traffic?

The slice's HTB rate and ceiling bound it (no class is unlimited), and fq_codel isolates flows within a class.

Honest limit. With borrowing up to link capacity, a misclassified bulk flow in the healthcare class could still crowd it. Per-flow policing or a ceiling inside critical classes, plus a confidence threshold, are needed.

Q402. Would you use confidence thresholds, hysteresis, multiple observations or policy constraints?

Yes, all four:

- a confidence threshold;
- k-of-n agreement across windows before installing;
- stickiness (today the first decision sticks forever; it should be re-evaluated with hysteresis);
- policy constraints (per-flow caps in critical classes).

Q403. How would you reconcile a prediction with trusted application metadata?

Metadata wins (S-NSSAI in 5G, an authenticated app registration). ML fills gaps only, and disagreements are logged as drift or abuse signals.

Q404. Where should inference run: edge gateway, central controller or a dedicated service?

At the edge gateway (`s_rural` side), where the flows are seen and enforcement happens, which avoids exporting traffic metadata. The model is small (131 KB CNN).

Q405. What is the latency budget for the classification-to-enforcement path?

Decision after at least 5 packets, at the next control interval (up to 5 s), plus milliseconds of inference and a millisecond-scale `tcfilteradd`. For short critical flows this is too slow; metadata or host rules must cover them.

Q406. What if inference is slower than the monitoring interval?

It runs synchronously in the loop, so a slow model would delay the tick. The wall-clock-aligned scheduler would not accumulate drift, but telemetry would be late. Mitigation: run inference in a worker thread with the latest-result pattern, like the probes.

Q407. How would you deploy a new model without interrupting service?

Load it next to the old one, run it in shadow mode (log predictions, do not act), compare agreement and confidence, then atomically swap the reference. Existing flow rules persist.

Q408. How would you roll back a model that degrades performance?

Keep the previous artefact. Watch per-slice SLA and confidence metrics; swap back on regression. Flow rules installed by the bad model can be flushed.

Q409. What monitoring is needed to detect inference failures or distribution shift?

Inference errors and latency, the prediction class mix vs expected, confidence histograms, feature drift statistics, and the downstream SLA of classified flows.

Q410. How would you evaluate the classifier on real network traffic?

Labelled ground truth from consenting institutions (known devices or apps), header-only capture, offline evaluation first, then shadow mode in the loop before acting.

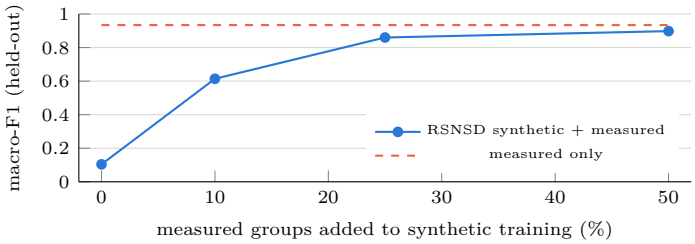


Figure 7: Synthetic-to-measured transfer: RSNSD alone is near chance on measured traffic.

Part IX

Context-aware priority, allocation and QoS enforcement

24 Priority and policy design

Q411. What determines the priority of a service?

The operating context's weight vector (`scenarios.yaml`), multiplied by an optional learned policy multiplier and by temporary healing boosts:

$$p_s(t) = w_s\big(c_t\big) \cdot m_s(c_t, \text{state}) \cdot b_s(t).$$

Q412. Which inputs influence priority?

Context (always), the learned policy (CEM or PPO, if configured), SLA violations (latency or congestion boosts of $\times 2$ for 6 ticks; saturation halves general's weight), and operator policy (via the admin override, which picks the context). Demand affects *allocation*, not priority.

Q413. Is priority a static rank, a score, a set of constraints, or a learned value?

A positive numerical weight per slice (a ratio scale), not a rank. Floors and healing limits are separate *constraints*. Learned multipliers rescale the weights.

Q414. What is the mathematical definition of the priority function?

$p_s = w_s(c)\, m_s\, b_s$ with $m_s \in [0.1, 10]$ (CEM) or 10^{a_s}, $a_s \in [-1, 1]$ (PPO), and $b_s \in \{1, 2, 0.5\}$ with expiry. Shares are proportional to p_s among slices still unsatisfied (Q432).

Q415. Why that formulation?

Multiplicative weights compose cleanly (context × learning × healing), are scale-free, and plug directly into weighted water-filling. An ordinal rank would force strict priority, with starvation.

Q416. How are competing requirements weighted?

Floors guarantee the minima, priorities share the rest, demand caps prevent waste, and surplus goes to unsatisfied slices by priority.

Q417. How are weights selected and validated?

Selected by design per context (e.g. disaster: healthcare 10, others 1); validated by sensitivity. The `no_context` ablation and the learned-multiplier experiments probe whether other weights do better. Learned multipliers did not significantly improve outcomes, which suggests the design weights are not far from a local optimum for this metric.

Honest limit. They are not derived from stakeholder elicitation.

Q418. How does an emergency change priorities without changing application identity?

Only $w(c)$ changes. Flows stay in their slices, and only the weights of those slices change.

Q419. How do you prevent one context event from permanently elevating a service's priority?

Alerts have explicit clear events (timeline), healing boosts expire after 6 ticks, and context is re-inferred every tick.

Honest limit. A stuck alert would persist. Add alert time-to-live.

Q420. What prevents priority oscillations when context changes frequently?

The context changes only on events or calendar boundaries; re-planning applies only if a slice moves by at least 1 Mb/s; boosts have a cooldown. There is no context dwell-time hysteresis (Q178).

Q421. How do you distinguish urgency from resource demand?

Urgency = weight (how much a slice is preferred under scarcity). Demand = measured offered load (how much it can use). The allocator caps every share at demand, so high urgency with low demand wastes nothing.

Q422. What if the highest-priority service doesn't need more bandwidth?

It receives exactly its demand and the remainder is redistributed. Example (school): healthcare weight 5 but demand 5 Mb/s, so it gets 5 and education gets 70.33 (Fig. 8). The proportional baseline would hand healthcare 38.46 Mb/s it cannot use.

Q423. Can a high-priority flow have a throughput ceiling?

Yes. Each class has an HTB `ceil`: link capacity when work-conserving, the allocation when strict, or the backup capacity share for core slices while on backup.

Q424. How do you prevent lower-priority services from starving?

Floors (general 1 Mb/s), HTB guaranteed `rate`, borrowing of idle capacity, and fq_codel within classes. Strict priority is never used.

Q425. How are minimum service guarantees represented?

`min_mbps` per slice in `slices.yaml` (healthcare 5, education 2, agriculture 1, general 1), allocated first as $\min(f_s, d_s, C/4)$ and enforced as HTB `rate`.

Q426. How do you handle services with both minimum and maximum requirements?

Minimum = the floor. Maximum = the demand cap (the allocator never grants above demand) plus the HTB ceiling.

Q427. What if the sum of minimum requirements exceeds capacity?

Each floor is clipped to $C/4$, i.e. equal division at the floor level.

Honest limit. This is crude. A priority-ordered floor allocation (critical floors first) would be better, and it would also need an operator alarm. Not exercised in the matrix: floors sum to 9 Mb/s against capacities of at least 50.

Q428. Which policy decisions must remain deterministic and explainable?

Floors, context precedence, healing actions and constraints, and the final allocation given its inputs. Every re-plan records its reason, demand estimate, priorities and allocation (`controller_history`). Learned multipliers are logged but only rescale explainable weights.

Q429. What happens when a context source is unavailable or untrusted?

The engine keeps the calendar-based context, and unknown alerts default to disaster (conservative). There is no trust model (Q184).

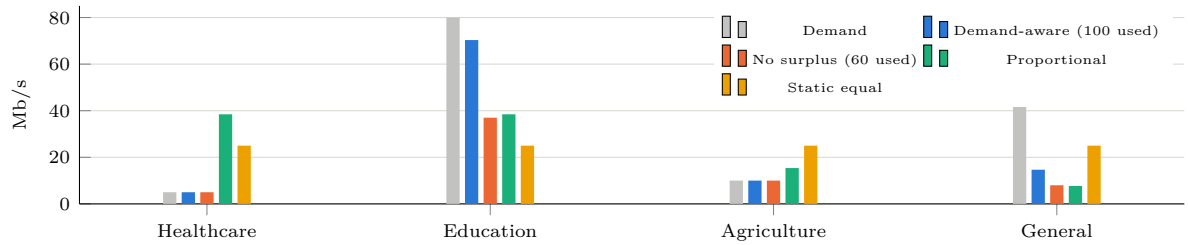


Figure 8: Allocators on the school context (demand 145 Mb/s on 100 Mb/s; weights 5/5/2/1; floors 5/2/1/1). Values are produced by `slicing/allocators.py`.

25 Resource allocation

Q430. What resources does CARS-SF allocate?

Bandwidth on the shared rural uplink, in both directions, per service slice: the HTB guaranteed rate and the ceiling. Path choice (primary or backup) is decided by healing. Queue *capacity* is managed by fq_codel, not allocated.

Q431. Is the decision about bandwidth, queue rates, path selection, queue capacity, or a combination?

Bandwidth and queue service rates (HTB rate and ceiling) per slice, plus path selection on failure.

Q432. What is the mathematical resource-allocation problem?

Given capacity C, weights p_s, demands d_s and floors f_s:

1. $g_s = \min(f_s, d_s, C/4)$.
2. Residual $R = C - \sum g_s$, residual demand $d'_s = d_s - g_s$.
3. Weighted water-filling: repeatedly give each unsatisfied slice $\min\left(R\,p_s/\sum_u p_u,\ d'_s\right)$, subtract, and repeat until $R \approx 0$ or all are satisfied.
4. $x_s = g_s +$ water-filled share.

This is the *weighted max-min fair* allocation with demand caps, on top of guaranteed floors.

Q433. What is the objective function?

Implicit: weighted max-min fairness (lexicographically maximise $\min_s x_s/p_s$ subject to the caps). The *evaluation* metric is $U = \sum p_s \min(1, x_s/d_s)/\sum p_s$.

Honest limit. The allocator does not maximise U directly. U would favour high-p_s, low-d_s slices greedily. The RL policies were the attempt to optimise U end to end, and gained nothing significant.

Q434. What are the constraints?

$\sum_s x_s \le C$; $x_s \ge g_s$; $x_s \le d_s$ (above the floor); on backup, $\sum_{s\in\text{core}} x_s \le 0.95\,C_{backup}$; in the degraded state, core slices are held at their floor.

Q435. Is the allocation feasible for every demand vector?

Yes. The floors sum to at most C by construction, and water-filling never exceeds R. If total demand is below C, every slice is fully satisfied and spare rate remains (borrowable).

Q436. What happens when it is infeasible?

It cannot be infeasible in the constraint sense. "Demand above capacity" just means some slices are unsatisfied, by priority. Floors above capacity are clipped (Q427).

Q437. How is available capacity estimated?

It is configured, not estimated: the nominal bottleneck rate (HTB root), plus the known backup capacity when on backup.

Honest limit. There is no online capacity estimation. A degraded-but-up link (e.g. a microwave fade) would be over-allocated.

Q438. How is nominal vs usable capacity handled?

HTB enforces the nominal rate at L2. Calibration from runs gave delivered/allocated efficiency 1.0 (demand is also measured at L2). The backup is capped at 95% to keep its queue empty.

Q439. How do you account for protocol overhead and control traffic?

Both demand and enforcement count L2 bytes, so overhead is included consistently. Control traffic (probes, about 4 packets per second) lands in the class of the probing UE (host filters), which is negligible.

Q440. What is the difference between a bandwidth ceiling and a reservation?

Reservation (HTB `rate`): guaranteed when backlogged. Ceiling (`ceil`): the maximum including borrowing. Our default is reservation = allocation, ceiling = link (work-conserving).

Q441. How do you prevent configured allocations from exceeding the bottleneck capacity?

By the allocator invariant (unit-tested: $\sum \leq C + 0.01$), and in the data plane the HTB root class caps the total at C regardless.

Q442. How do you allocate spare capacity?

Twice:

- control plane: surplus redistribution to unsatisfied slices by priority;
- data plane: HTB borrowing (ceiling = capacity), in prio order, at packet timescale.

The ablation shows the data-plane mechanism dominates. Surplus redistribution matters only with strict caps (0.789 vs 0.651).

Q443. What if demand is below the configured minimum?

The floor is $\min(f_s, d_s)$, so nothing is over-reserved.

Q444. How do you reclaim unused capacity?

Instantly by borrowing (an idle reservation is lent at once), and structurally at the next re-plan as measured demand falls.

Q445. How does the algorithm respond to sudden demand changes?

Instantly through borrowing; within 5 s through re-planning (sooner on a context change). Elastic flows at their cap trigger a 1.25× probe-up; a Holt forecast tracks trends.

Q446. Greedy priority vs weighted fair vs proportional fairness vs optimisation-based allocation?

- Greedy or strict priority: serve in order, causing starvation.
- Weighted fair: shares proportional to weight, without demand caps (our `proportional` baseline).
- Proportional fairness: maximise $\sum \log x_s$, a throughput–fairness compromise.
- Optimisation: an explicit objective plus constraints, solved numerically.

Ours is weighted max-min with demand caps and floors.

Q447. Why is the chosen method appropriate?

Closed form, deterministic, explainable, $O(n^2)$, never wastes capacity on satisfied slices, and honours floors. Under borrowing, the outcome is close to any reasonable allocator anyway, which matches the null main-study result.

Q448. What is its computational complexity?

$O(n \cdot k)$ with $k \leq n$ rounds, so $O(n^2)$ for n slices. With $n = 4$: 7 μs per controller step.

Q449. How does it scale with flows, slices, links and constraints?

It is independent of the number of flows (slice-level). Quadratic in slices (trivial). A single link only: multiple links require a network-flow or LP formulation.

Q450. Locally optimal vs globally optimal allocation?

For one link and the max-min criterion, water-filling is *globally* optimal. Multi-link max-min needs progressive filling across links.

Q451. How do you prevent reallocation from costing more than it saves?

A 1 Mb/s change threshold, a 5 s interval, an estimator warm-up after context changes, and boost cooldowns. A re-plan is 8 tc commands.

Q452. Which baseline allocation policies should be compared?

All compared: no slicing (FIFO), static equal, proportional, 3GPP fixed (URLLC 20 / eMBB 70 / mMTC 10), the proposed scheduler, CEM, PPO, and the no-surplus variant.

26 QoS and SLA enforcement

Q453. What is the difference between QoS, an SLA and a service priority?

QoS: measured treatment (delay, loss, throughput). SLA: target thresholds the QoS must meet. Priority: the relative weight in sharing scarce capacity.

Q454. Which SLAs are defined for each service class?

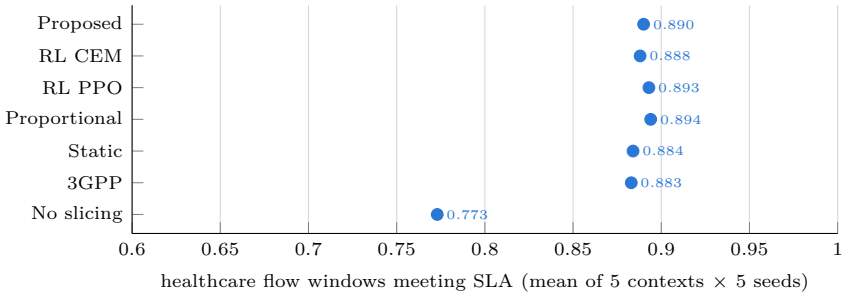


Figure 9: Any slicing vs none: the significant effect in the main study ($\Delta = 0.117$, Wilcoxon $p = 0.0010$, $n = 25$ pairs). The differences between slicing schemes are not significant. Dots, not bars: the axis is zoomed to 0.6–1.

From `slices.yaml` (one-way delay, jitter, loss, minimum satisfaction):

- healthcare: 50 ms, 10 ms, 1%, 0.95;
- education: 150 ms, 30 ms, 2%, 0.8;
- agriculture: 250 ms, 50 ms, 2%, 0.8;
- general: 500 ms, 200 ms, 5%, 0.5.

Per-application SLAs (e.g. ECG 20 ms) live in `applications.yaml`.

Q455. Are latency, jitter, throughput, packet loss and availability all measured?

Latency, jitter, loss and throughput: yes, per flow window (packet-matched), and online per slice (RTT/2, HTB drops, counters). Availability: no (Q105).

Q456. Which metrics are measured directly, and which are estimated?

Directly: one-way delay, loss, throughput (post-run), counters, drops. Estimated: online latency (RTT/2 from one probe per service), demand (with elastic probe-up), and loss online (drop ratio).

Q457. How are SLA thresholds derived?

Typical-requirement values by design (voice and real-time medical stricter, bulk looser).

Honest limit. Not derived from clinical or regulatory sources. Cite standards (e.g. ITU-T G.114 for interactive voice delay)[†] and present the values as assumptions.

Q458. Are thresholds absolute or context-dependent?

Absolute.

Honest limit. One could argue they should tighten in emergencies; not modelled.

Q459. How are SLA violations aggregated over time?

Per slice per tick (`SliceMonitor`) giving compliance shares; per flow window (`sla_satisfied`) giving the SLA-met share; per-tick reward in `feedback.csv`.

Q460. Why is one anomalous measurement insufficient to declare a sustained failure?

Single probes are noisy (one lost ICMP packet, a scheduling hiccup). We report shares over windows and ticks, the predictor uses 6-sample trends, and escalation needs 2 consecutive ticks.

Honest limit. The latency-spike *boost* reacts to a single tick (it is cheap and expires).

Q461. QoS target vs hard guarantee?

Targets are monitored and pursued. A hard guarantee would need admission control plus worst-case bounds (network calculus). HTB guarantees *rate* under backlog; delay is not guaranteed.

Q462. Can a software-defined virtual network guarantee a latency target under arbitrary CPU contention?

No. Timers and softirqs slip under CPU starvation. We observed this (Q220) and controlled for it, but no software emulation can promise it.

Q463. How do you measure the effects of HTB and queue configuration on each class?

With ablations (Fig. 10): removing borrowing raises school healthcare p95 from 10.0 to 165.0 ms; removing AQM *raises* throughput satisfaction (0.865→0.903) but general-internet p95 rises from 160 to 1433 ms, and disaster healthcare SLA share falls from 0.838 to 0.597. Plus per-class HTB counters.

Q464. How do you verify QoS is applied to the intended interface and class?

Unit tests assert the exact commands (class ids, interfaces, filter matches). At run time, per-class counters move for the expected class only.

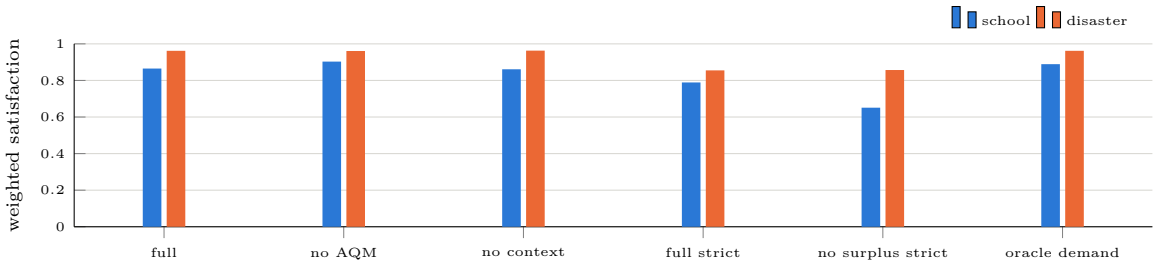


Figure 10: Ablation (3 seeds). Strict caps (no borrowing) cost the most; surplus redistribution matters only once borrowing is disabled. Removing AQM *raises* this throughput metric while tail delay explodes (Q463); removing context changes nothing measurable.

Honest limit. There is no read-back assertion in the controller.

Q465. How do you handle configuration errors or unsupported kernel commands?

Honest limit. Not handled: command output is ignored.

Fix: check exit codes, read back the qdisc and class state, and raise a `config_failure` event (Q139).

Q466. How do you prevent a new policy from violating another service's SLA?

Floors and constraints bound harm, and boosts expire.

Honest limit. There is no predictive check that a re-plan keeps every other slice within its SLA. That would need a model of delay as a function of allocation (the calibrated simulator could serve).

Q467. How do you evaluate fairness alongside critical-service performance?

Jain fairness of per-slice demand satisfaction, reported for every run: proposed 0.985 vs no slicing 0.994. No slicing is *fairer* but worse for healthcare SLAs. That is the trade-off made explicit.

Q468. How do you measure the cost of better healthcare latency to best-effort throughput?

Per-slice demand satisfaction tables per scheduler and context (`main_summary.csv`), and the general slice's satisfaction in disaster contexts, where it gives way to healthcare.

Q469. What if the SLA registry is inconsistent with the experiment configuration?

The monitor (slice SLAs) and the dataset labels (app SLAs) read the same registries, so there is no copy to drift.

Honest limit. Slice-level and app-level SLAs are two layers that can disagree (an app stricter than its slice). Results report both.

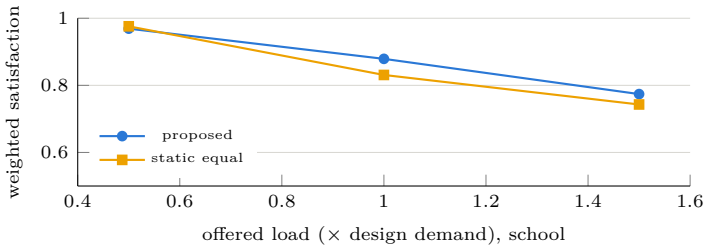


Figure 11: Load sensitivity (school, 3 seeds): the proposed scheduler helps only once demand exceeds capacity.

Part X

Monitoring, fault detection and self-healing

27 Fault detection

Q470. What faults is CARS-SF designed to detect?

Backhaul cut (loss of carrier); backhaul degradation (rising delay and loss); core switch failure (core unreachable after failover); bottleneck congestion (drops); saturation (utilisation above 95%); latency spikes (RTT above 200 ms); total connectivity loss.

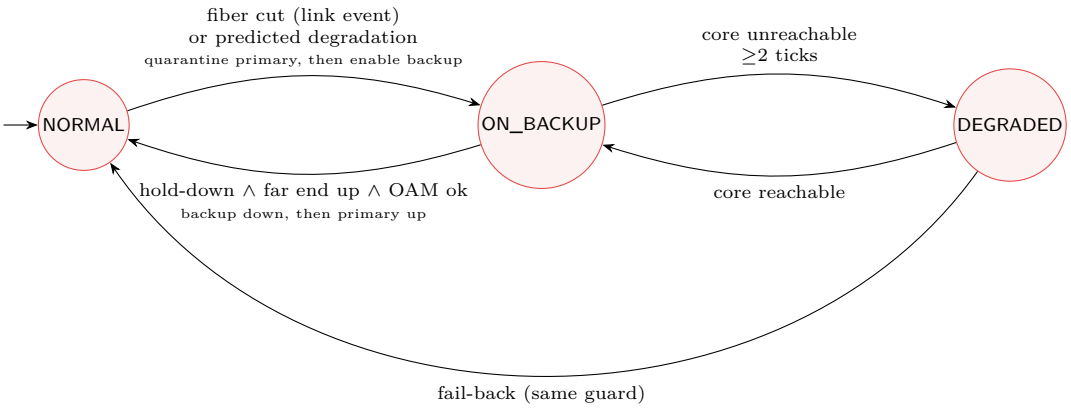


Figure 12: Self-healing state machine (`self_healing/engine.py`). The L2 domain is loop-free at every instant.

Q471. Which faults are explicitly represented in the test scenarios?

`fiber_cut` (cut at $\frac{1}{3}$, repair at $\frac{2}{3}$), `backhaul_degradation` (+15 ms and +0.3% loss per tick from 0.2 of the run, cut at 0.7), `core_switch_failure` (all core ports down at $\frac{1}{3}$, repair at $\frac{2}{3}$). Congestion arises naturally from load.

Q472. How do you distinguish link failure, congestion, excessive latency, packet loss and telemetry failure?

- Link failure: carrier transition on `s_edge-bh`.
- Congestion: local drops, backlog or saturation.
- Excessive latency: RTT above threshold; as a trend without local congestion, attributed to the backhaul.
- Loss: degradation loss shows as lost probes and loss in captures.
- Telemetry failure: **not distinguished**. An unavailable reader looks like zero (Q279).

Q473. Fault detection vs diagnosis vs localisation vs recovery?

Detection: something is wrong. Diagnosis: what kind (cut vs degradation vs congestion). Localisation: where (backhaul link vs core switch). Recovery: restoring service. CARS-SF localises by elimination: if failover to a path that bypasses the link does not restore the core, the fault is the shared endpoint (`s_core`).

Q474. Is the detector rule-based, statistical, ML-based or hybrid?

Hybrid rule and statistical: thresholds and link events (reactive), plus a least-squares RTT trend over a 6-sample window with a horizon forecast (predictive). No ML.

Q475. Why is that detector appropriate here?

Why. Faults must be acted on in under 1 s, explainably, with no failure labels for training.

Choice. Physical signals (carrier) for cuts; trends plus a congestion discriminator for degradation.

Alternative. An ML anomaly detector (isolation forest, LSTM) over telemetry.

Trade-off. Deterministic, explainable and data-free; but thresholds are hand-set and novel fault types are missed.

Evidence. Detected every injected cut; no false failover in any run without an injected fault (Q487). **L3 validated (emulation)**

Q476. What are the inputs to the detector?

The per-tick snapshot (drops delta, utilisation, per-service RTT, backhaul and backup flags, queue backlog) and timestamped link-watcher events, delivered between ticks through the fast path.

Q477. What constitutes an anomaly?

- More than 50 bottleneck drops per tick, or utilisation above 95%.
- RTT above 200 ms.
- All probes lost (connectivity loss).
- Core probes lost with edge probes alive (core unreachable).
- Carrier loss on the backhaul (cut).
- Predicted degradation: every core-bound RTT slope at least 2 ms per tick, the forecast 5 ticks ahead at least the strictest core threshold, edge flat, and no local congestion in the window.

Q478. How are thresholds calibrated?

By design and checked against scenario runs: the strictest core-bound threshold is twice education's 150 ms SLA; the slope floor of 2 ms per tick lies above probe noise (≈0.5 ms).

Honest limit. No ROC calibration. The congestion discriminator was added after a real false failover on OVS.

Q479. What are the trade-offs between false positives and false negatives?

A false failover moves core traffic to a 50 Mb/s, 40 ms path, costing capacity and latency, and flaps. A missed cut costs an outage. Cuts use a physical signal (near-zero false positives); prediction is conservative (several conditions must hold), trading earlier warnings for fewer false alarms.

Q480. What if a genuine fault produces no immediate SLA violation?

Carrier-based detection does not wait for an SLA violation: the cut is acted on at once. Degradation is caught as a trend *before* the SLA breaks (which is the point of prediction).

Q481. What if normal workload variation triggers an anomaly?

Latency spikes and congestion in normal runs do trigger healthcare or affected-slice boosts. These are cheap, bounded ($\times 2$) and expire. Normal variation does *not* trigger failover: 0 false failovers in 1,496 runs without a fault.

Q482. How does the detector distinguish a transient spike from persistent degradation?

Prediction needs a 6-sample trend with a consistent slope and no congestion anywhere in the window; escalation needs 2 consecutive ticks; boosts react to single ticks but expire.

Q483. What is the role of temporal windows and consecutive violations?

They suppress noise: the trend window for prediction, 2 consecutive ticks for core-failure diagnosis, a 5-tick hold-down before fail-back, and a 6-tick boost lifetime.

Q484. How do you detect a fiber cut if the interface stays present but traffic stops flowing?

The interface stays present after a cut; what matters is carrier. On veth (as on fiber with loss of light), a far-end failure sets our side to `lowerlayerdown`, which the watcher sees within 50 ms.

Honest limit. A *silent* failure (carrier up, forwarding dead) shows only as core-unreachable probes, and in the NORMAL state that event does not trigger failover. Gap: add "core unreachable for 2 ticks while NORMAL" → fail over.

Q485. How do you distinguish a dead path from a merely congested one?

Dead: probes are lost, carrier is down, deliveries stop. Congested: probes return with high RTT, and local backlog and drops rise.

Q486. How is diagnosis validated against known injected faults?

Each resilience run records injector events (type, tick, timestamp) and healer actions (trigger, timestamp). The report compares them: switched before the cut in 100.0% of degradation runs; core-switch failure diagnosed in 100.0%; an integration test asserts each.

Q487. How do you measure precision, recall, false-alarm rate and detection delay?

- Recall: injected failures with a healing response, 54/54.
- False alarms: failovers in fault-free runs, 0/1,496.
- Detection delay: MTTD 0.002 s for a cut (event-driven); 15.003 s from the start of degradation for prediction (by design it waits for the trend).

Honest limit. Small n per fault type (3 seeds $\times$ 3 variants), so no confidence intervals on recall.

Q488. How does detector performance change with several simultaneous faults?

Honest limit. Untested. Combinations (degradation + congestion, cut + context change) are not in the matrix. Congestion during degradation would, by design, suppress prediction (conservative); cuts would still be caught.

Q489. What happens with a fault outside the detector's known categories?

It is caught only if it shows up as one of the signals (RTT, drops, carrier, reachability). Otherwise it is invisible. The SLA monitor still records the violation.

Q490. How does the system avoid acting on unreliable or stale observations?

Link events carry timestamps; self-inflicted transitions (within 1 s of the healer's own action) are ignored; prediction requires sustained trends.

Honest limit. There is no general staleness check (Q280).

28 Recovery planning and action selection

Q491. What recovery actions can CARS-SF execute?

- Quarantine the primary backhaul, activate the backup, flush the FDBs.
- Fail back (backup down, then primary up, then flush).

- Allocation constraints: core slices capped to 95% of backup capacity with HTB ceilings; degraded mode holds core slices at their floors and lends the rest to edge slices.
- Priority boosts (affected slice ×2; general ×0.5 on saturation).
- Diagnosis escalation.

Q492. Which actions change bandwidth allocation, and which change the forwarding path?

Path: quarantine, backup activation, fail-back. Allocation: constraints, ceilings, boosts.

Q493. How is the candidate action set constructed?

A fixed mapping from failure type to action (state machine). There is no search over candidates.

Q494. How does the planner decide which action to attempt first?

Path protection first (it restores connectivity), then allocation constraints (the controller re-plans at once because the healing state changed).

Q495. Why does context influence recovery ordering?

It does not influence the *ordering*; it influences the *post-recovery allocation*, which uses the current context's weights under the healing constraints.

Honest limit. Context-dependent recovery (e.g. preferring a costlier path only in an emergency) is not implemented.

Q496. How does the system account for resource capacity when choosing an action?

The constraints encode the backup's capacity (core slices $\leq 0.95 \times 50$ Mb/s) and degraded mode. Without the ceilings, the 50 Mb/s backup bufferbloated (RTT oscillating 116–336 ms); with them it stays at its 96 ms floor.

Q497. What if the preferred recovery action is infeasible?

If the backup does not restore the core, the engine escalates to DEGRADED (graceful degradation). There is no other path to try.

Q498. How does the planner compare predicted outcomes of candidate actions?

Honest limit. It does not. There is no outcome model in the planner.

The calibrated sequential simulator could score candidate allocations before applying them.

Q499. What evidence supports the prediction model?

For the *fault predictor*: switched before the cut in 100.0% of runs; outage 0.019 vs 0.214 s reactive-only. There is no outcome-prediction model for actions.

Q500. What happens when the predicted outcome differs from the actual outcome?

For prediction, the guards handle it: if the primary later proves healthy (OAM), fail-back follows after hold-down. A wrong failover costs only capacity until then.

Q501. How do you prevent an action from degrading another service's QoS?

Floors always hold; constraints only move capacity from slices that cannot use it (core-bound slices during an outage) to edge slices; boosts are bounded and expire.

Q502. How do you avoid oscillating between two allocations or paths?

A 5-tick hold-down; OAM-gated fail-back; ignoring self-inflicted link events; the change threshold; boost cooldowns.

Evidence. Before these guards, a degradation run alternated failover and fail-back 3 times in 3 ticks. After them, exactly one proactive switchover. An OVS run with AQM-less queues flapped before the congestion discriminator and did not after. **L3 validated (emulation)**

Q503. Is there a maximum number of recovery attempts?

Path switching happens once per failure episode (state-guarded).

Honest limit. Boosts can recur every 6 ticks indefinitely; there is no attempt counter.

Q504. How are retry delays and back-off selected?

Fixed: a 6-tick boost lifetime and cooldown, a 5-tick hold-down, 2 ticks before escalation. Chosen to exceed the 5 s control interval and TCP's reaction time.

Honest limit. There is no exponential back-off.

Q505. How does the planner tell a failed action from one that needs more time?

After backup activation it waits 2 ticks (and requires 2 consecutive core-unreachable ticks) before concluding the core itself is down.

Q506. Reactive recovery vs proactive reconfiguration?

Reactive: after the cut (carrier). Proactive: before the cut, when trends predict a breach. Measured outage: proactive 0.019 s vs reactive 0.214 s on degradation scenarios.

Q507. When should the system escalate to an operator?

On DEGRADED (core-switch failure, which needs physical repair), on repeated boosts with no SLA recovery, on configuration failures, and on any state that persists beyond a time limit.

Honest limit. Escalation is only logged; there is no paging integration.

Q508. How do you ensure the controller does not endlessly repeat an ineffective action?

Path actions are state-guarded (one per episode).

Honest limit. Boost repetition is not limited. Add an attempt budget and escalation after N ineffective boosts.

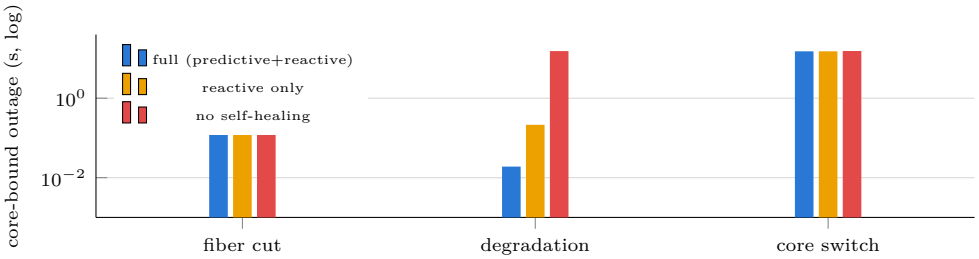


Figure 13: Longest delivery gap of core-bound flows after the failure (packet-matched, 3 seeds; log scale). Core-switch failure cannot be bypassed: healing localises the fault but cannot restore core-hosted services.

29 The closed-loop control system

Q509. What are the precise stages of the CARS-SF self-healing loop?

Monitor (telemetry, SLA monitor, link watcher) → Analyse (reactive detector, predictor) → Plan (healing state machine, controller re-plan under constraints) → Execute (topology primitives, QoS backend) → Knowledge (registries and history). This is a MAPE-K loop.

Q510. What is the role of monitoring in the loop?

To provide the observations (rates, drops, RTT, carrier) and the per-slice SLA record against which success is judged.

Q511. What is the role of diagnosis?

To pick the right action: a cut means fail over; a core failure means degrade gracefully (failover is pointless); congestion means re-prioritise and do *not* fail over.

Q512. What is the role of the recovery planner?

To map the diagnosis to a safe action sequence (loop-free ordering) and to constraints for the allocator.

Q513. What is the role of the enforcement backend?

To execute path changes (`iplink`, FDB flush) and rate changes (tc, OVS or UPF).

Q514. What is the role of post-action verification?

To confirm that the action achieved the goal before declaring success, or to trigger the next step. CARS-SF verifies *reachability* after failover (core-unreachable escalation) and *link health* before fail-back (far end up plus OAM).

Honest limit. There is no generic KPI check after allocation changes and no rollback.

Q515. Why is verification necessary after a recovery action?

Actions can fail silently (a dead backup, a tc error, stale FDB entries) or fix the wrong problem (failover when the core switch is the fault). The core-switch case is exactly where verification changed the decision.

Q516. What evidence is required to declare recovery successful?

Online: core probes answered again on the new path. Post-run (ground truth): the measured outage (longest delivery gap) and packets lost. Fiber cut: 0.165 s mean outage.

Q517. How long should the system wait for the network to settle?

At least the FDB relearning time plus TCP recovery, about 1 s on OVS. We use 2 ticks before escalation and 5 ticks of hold-down before fail-back.

Q518. How many consecutive good measurements before declaring recovery?

Implicitly 2 (escalation logic) for the core path.

Honest limit. There is no explicit "recovered" declaration with N good SLA samples. Add one with N=3.

Q519. What if the SLA improves temporarily and then degrades again?

On backup, renewed degradation would show in probes. Escalation and boosts react. A repeated primary failure after fail-back starts a new episode.

Q520. What if the action improves latency but drops throughput below the SLA?

That is what happens on backup: capacity halves, so core slices are capped and their satisfaction drops while connectivity is restored. The SLA monitor records it. It is a deliberate trade-off (connectivity over throughput), reported per slice.

Q521. How do you evaluate recovery when several KPIs must hold at once?

Per-slice compliance requires latency, loss and satisfaction together (`sla_ok`). Run-level: outage, packets lost and weighted utility (0.165 s, 109 packets, utility in `resilience_summary.csv`).

Q522. What if the verifier reports that recovery failed?

For a failover that did not restore the core: escalate to DEGRADED. For fail-back with OAM failing: stay on backup.

Q523. Does the controller roll back, escalate, or try another action?

It escalates (degraded mode).

Honest limit. It does not roll back to the primary while the primary is down, and there is no alternative action list beyond that.

Q524. How do you ensure rollback does not worsen the original fault?

Fail-back is the rollback, and it is gated by far-end-up plus OAM health plus hold-down, so we never return to a known-bad link.

Q525. How is every decision, action and verification result recorded?

`healing_actions` (action, tick, timestamp, trigger, trigger timestamp, state); `injector_events`; `failure_types`; `controller_history`; `feedback.csv` (per-tick healing state); `resilience` summary, all in `summary.json`.

Q526. Open-loop recovery script vs verified closed-loop controller?

An open-loop script executes a fixed action. A closed-loop controller observes the outcome and adapts (escalation, OAM-gated fail-back, hold-down). CARS-SF is closed-loop for path recovery, but only partially verified for allocation changes.

Q527. What experiment proves that verification reduces unnecessary reconfigurations?

Anecdotal but measured: without the hold-down, OAM gate and self-event suppression, a degradation run produced 3 failover/fail-back cycles; with them, 1 switchover.

Honest limit. There is no controlled ablation across seeds. Add a "no guards" variant to the resilience study.

Q528. How would you demonstrate the entire loop in a live Mininet run?

`sudo.venv/bin/pythonrun_experiment.py--scenariobackhaul_degradation--duration45--apply-qos`. Watch the log: degradation start; predicted degradation; proactive backup activation; ceilings applied; the cut (no outage); fail-back after repair. Then open `summary.json`: `resilience`, `healing_actions`.

30 Fault tolerance of the controller itself

Q529. What happens if the monitoring component crashes?

It runs in the orchestrator process. A crash ends the run; the kernel keeps the last QoS state.

Honest limit. There is no supervisor, so a crash means loss of healing.

Q530. What happens if the controller loses contact with the switch?

Not applicable in emulation (local commands). In a deployment it would be a partition: enforcement persists, healing on the far side stops (Q157).

Q531. What happens if the controller restarts mid-recovery?

Honest limit. State is in memory only. After a restart the engine starts in NORMAL while the primary is quarantined and the backup active.

Fix: persist the healing state, and on start reconcile from link admin states.

Q532. How is the previous network configuration recovered after a crash?

It is not reconstructed. The kernel retains qdiscs and link states, and Mininet teardown in `finally` cleans up the emulation. A real deployment would read back state on start.

Q533. How do you prevent two controller instances from issuing conflicting actions?

Honest limit. Nothing does. A single instance is assumed.

Use a lease or leader election (etcd or ZooKeeper) and idempotent intents.

Q534. How do you handle duplicate commands and retries?

Most commands are idempotent (`iplinksetup/down`, `tcclasschange` to absolute values), and the injector skips duplicates. `tcfilteradd` is not idempotent, so flow rules are tracked and installed once.

Q535. Are configuration changes idempotent?

Rates and ceilings, link states and FDB flushes are idempotent. Filter additions are guarded by the flow-rule table.

Q536. How do you ensure the system eventually converges to a stable state?

Finite states with guarded transitions (hold-down, OAM, one switch per episode); expiring boosts; the change threshold. In tests and runs it always converged.

Honest limit. There is no formal convergence proof.

Q537. What if the primary and backup paths fail simultaneously?

That is equivalent to the core-switch case: escalate to DEGRADED, protect edge services, wait for repair.

Q538. What if no feasible recovery action exists?

Degrade gracefully (keep surviving services prioritised), keep monitoring, escalate to the operator.

Q539. How does the controller preserve critical traffic during partial failure?

Floors, context weights, and degraded-mode reallocation: core slices are held at their floors and edge slices, including healthcare (served by the edge), get the capacity.

Q540. What is the minimum safe behaviour when the controller cannot decide reliably?

Keep the last good configuration, never form loops, never fail back without OAM, keep floors, alert the operator. The kernel's persistence of HTB state provides much of this for free.

Part XI

Reinforcement learning and intelligent scheduling

The RL result in one sentence

Learned priority policies (CEM and PPO) were trained, integrated into the live loop, and evaluated against the rule-based demand-aware policy. **On the network they give no robust gain**: in the final run PPO is marginally ahead of proposed (proposed − PPO = −0.0042 in U, $p = 0.360$; not Bonferroni-significant, and not reproduced in three earlier full runs). In the calibrated simulator, reward per step is 76.16 (rule) vs 79.00 (CEM) vs 78.46 (PPO): +3.7 % and +3.0 % on paired test seeds ($p = 0.0000$, 0.0000), with SLA violations per episode 139.9 vs 108.7 vs 116.4. That simulated gain does not carry over to the network. Every live decision is logged as an RL transition, and an online-CEM variant learns from verified live transitions, but it shows no measurable trend over 10 runs. Present RL as an evaluated alternative with a negative live result, not as a contribution.

31 Does the RL component solve a real problem?

Q541. What specific decision is the RL scheduler meant to learn?

Per-context (CEM) or per-state (PPO) multipliers $m_s \in [0.1, 10]$ on the four slices' priority weights. The allocator then computes rates. RL never sets rates directly.

Q542. Why is reinforcement learning appropriate for this decision?

The allocator maximises weighted max-min fairness, not the evaluation objective U, and allocations have delayed effects (backlog, TCP adaptation, outages). RL can in principle optimise U end to end over those dynamics. Whether it *helps* is an empirical question, and our answer is no.

Q543. Why not a deterministic optimiser or a well-designed heuristic?

That is the baseline we use, and it wins (ties). For a single link with known demands, a convex program for U would be the principled alternative to RL. Its gain would likely be equally small, because HTB borrowing redistributes unused capacity in the data plane anyway.

Q544. What does the RL agent observe?

PPO (21 dimensions): context one-hot (8), demand/C (4), normalised backlog (4), current allocation/C (4), and an on-backup flag. Live, the same vector is built from the controller state and HTB backlog (`observation_from_live`). CEM observes only the context.

Q545. What actions can it take?

A continuous $a \in [-1, 1]^4$, mapped to multipliers 10^{a_s}.

Q546. What is the reward function?

Per step: $r = 100 \sum_s \tilde{w}_s \, \mathrm{sat}_s - 10 \cdot \#\{s : \mathrm{delay}_s > \mathrm{SLA}_s\} - 0.5 \cdot \overline{\mathrm{loss}\%}$, where $\tilde{w}$ are the *context's* normalised weights (not the agent-scaled ones, which would let it inflate its own reward), and $\mathrm{sat} = \min(1, \mathrm{served}/\mathrm{demand})$.

Q547. How does the reward balance SLA compliance, throughput, fairness and reconfiguration cost?

Throughput through satisfaction; SLA through the violation penalty; loss through a penalty.

Honest limit. There is no fairness term and no reconfiguration cost. Fairness is evaluated (Jain) but not rewarded.

Q548. How do you prevent the agent maximising one metric while harming critical services?

With a safety layer outside the policy:

- multipliers are clipped;
- floors are allocated before the policy's shares;
- healing constraints apply after;
- reward weights are the context's, not the agent's.

The policy can reshuffle surplus but cannot remove guarantees.

Q549. What is the environment the agent interacts with?

`scheduler/env_seq.py`, a Gymnasium environment:

- a 5 s step and 120-step episodes;
- Markov context switches ($p_{stay} = 0.92$);
- AR(1) demand noise ($\phi = 0.7$, $\sigma = 0.15$);
- random backhaul outages ($p = 0.02$, 10 steps, core capacity halved);
- work-conserving service with priority borrowing, backlog carry-over and fq_codel-like 0.1 s buffering;
- the live allocator and constraint logic.

It is calibrated from 19 emulated runs (propagation floors 8/28 ms, efficiency 1.0).

Q550. How is a state transition generated?

1. Allocate (policy → priorities → allocator → constraints).
2. Serve: arrivals are demand times Δt plus backlog; serve up to the allocation, lend the spare by priority.
3. Update backlog and overflow loss.
4. Advance context, demand noise and outage state.

Q551. How are delayed rewards handled?

Backlog carries consequences across steps; PPO uses $\gamma = 0.95$ (about 20 steps, roughly 100 s, of horizon) and GAE advantages.

Q552. How do you handle a non-stationary environment?

Non-stationarity (context switches, outages) is *inside* the training distribution, and the observation exposes context and backup state, so the policy can condition on it.

Honest limit. There is no online adaptation. A distribution shift beyond training (new contexts) is unhandled.

Q553. Which baselines are needed to show that RL adds value?

The rule-based demand-aware policy (identical pipeline, multipliers of 1) is the decisive one. Static, proportional and 3GPP baselines are also in the main study.

Q554. How do you evaluate an RL policy on scenarios it did not see in training?

Simulator evaluation uses held-out seeds (`evaluate_policy`, seed 1000+). The live evaluation is in emulation, which the simulator only approximates: a sim-to-emulation transfer test.

Honest limit. All contexts were seen in training. No unseen-context test.

Q555. How do you prevent the agent exploiting simulator artefacts?

Base-weight reward, the safety layer, calibration from emulation, and, decisively, evaluation in emulation (Mininet), not just the simulator. The emulation shows no gain, consistent with no exploited artefact carrying over.

32 Training, safety and integration

Q556. Which algorithm: Q-learning, PPO, DQN, contextual bandits or something else?

Two:

- the Cross-Entropy Method as a *contextual bandit* (the original environment's actions do not affect transitions, so this framing is exact);
- PPO (stable-baselines3) on the sequential environment.

Q557. Why are those algorithms appropriate for the state and action spaces?

Continuous low-dimensional actions with stochastic dynamics. PPO is a robust default for continuous control; CEM is a strong derivative-free optimiser when the problem is a bandit. DQN would need a discretised action space.

Q558. Is the action space discrete or continuous?

Continuous, 4-dimensional and bounded.

Q559. How is exploration controlled?

PPO: a Gaussian policy with entropy bonus 0.005. CEM: the sampling distribution's standard deviation, refitted to the elite 20% each iteration (with a floor of 0.05).

Q560. What is the training sample complexity?

PPO: 500k environment steps (8 parallel environments), minutes on CPU. CEM: 50 iterations × 32 samples per context.

Q561. How do you know training has converged?

Honest limit. Only through final-policy evaluation. Learning curves were not logged (SB3 verbose off). Add TensorBoard or CSV logging and show the reward plateau over timesteps.

Q562. How is the reward computed and logged?

Environment: per step, reported per episode by `evaluate_policy` (mean, std, violations, satisfaction) in `results/rl_training.json`. Live runs: per tick in `feedback.csv` (same reward definition, measured inputs).

Q563. Is the agent actually learning from reward, or fitting outcomes of random actions?

Today it learns from reward: CEM keeps the elite by reward; PPO maximises advantage-weighted likelihood. **The original implementation did not.** It fitted a regressor to *uniformly random* actions and ignored reward entirely, which is behaviour cloning of noise. This was found in the verification audit and replaced. Say this openly: it shows the audit worked.

Q564. How do you distinguish a learned policy from a predictive model or regression heuristic?

A learned policy improves its *return* through interaction (CEM's elite reward rises over iterations; PPO's episode reward against random and against rule). A regression heuristic predicts something without optimising return. The old code was the second kind.

Q565. Is the trained agent wired into the live allocation or recovery loop?

Allocation: yes. `rl_cem` and `rl_ppo` schedulers plug policies into the Priority Engine; they ran 25 main-study runs each in emulation (L3). Recovery: no.

Q566. If it were not wired in, what would the RL implementation contribute?

It is wired in. Its contribution is a negative empirical result plus a safe integration pattern (policy → priority multipliers → allocator with floors and constraints), and a calibrated environment for future policies.

Q567. How would you integrate it without letting unverified actions modify the network?

That is the current pattern: the policy can only rescale weights; floors and constraints still apply; outputs are clipped; a missing model falls back to the rule (recorded). For more autonomy: shadow mode first, then an action budget.

Q568. Offline training, online learning, or both?

Offline in the calibrated simulator, with calibration from emulation feedback. Online learning in a critical network would need safe exploration and is not done.

Q569. How do you constrain the action space to preserve minimum QoS guarantees?

Bounded multipliers; the floors and constraints sit *outside* the policy, so no action can violate them.

Q570. How would you compare RL with static priority, greedy allocation and the recovery planner?

Done for allocation in the main study (7 schedulers, 25 paired runs each, Wilcoxon). Not done for recovery (RL does not act on recovery).

Q571. How do you handle the absence of GPU infrastructure without unsupported claims about training requirements?

Everything was trained on CPU: PPO (500k steps) in minutes, SliceNet-CNN (29,764 parameters) in under a minute. We claim nothing that needs a GPU.

Q572. Which experiments can run on CPU, and which genuinely need acceleration?

All of ours run on CPU. Accelerators would matter for much larger models (Transformers over raw packet sequences), large-scale hyper-parameter search, or simulators with thousands of parallel environments.

Q573. What is the safest way to introduce a learned policy into a live network?

Shadow mode (log the would-be actions); a constrained action space and safety layer (already present); a canary site; automatic rollback on SLA regression; a human-in-the-loop approval for policy updates.

Q574. What evidence would justify keeping RL in the final architecture rather than as future work?

A significant, replicated improvement in emulation (paired across seeds and contexts, CI excluding zero) on the critical-service metrics, without a fairness regression. We do not have it, so RL is future work, and the environment plus integration pattern are the reusable artefacts.

Part XII

5G architecture and real-network integration

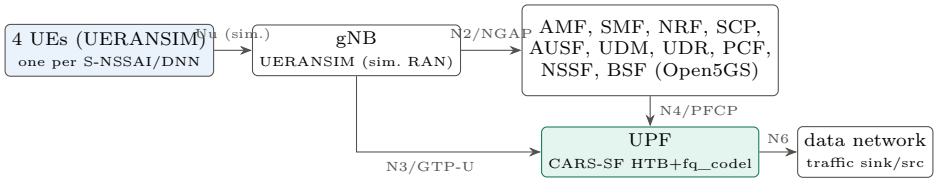


Figure 14: Real 5G core deployment (`deployment/5g`). CARS-SF enforces at the UPF, classifying by slice address pool.

33 Understanding the 5G system

Q575. Explain the path from a user device to an external application through a 5G network.

1. The UE registers through the gNB with the AMF (NAS over N1/N2; authentication via AUSF and UDM).
2. The UE requests a PDU session for an S-NSSAI and DNN; the SMF selects a UPF and installs packet rules over N4 (PFCP).
3. User packets go UE → (radio) → gNB → GTP-U tunnel on N3 → UPF → N6 → data network.

In our deployment the radio leg is simulated by UERANSIM.

Q576. What are the responsibilities of the UE, gNB, RAN and 5G Core?

UE: the subscriber device and its NAS stack. gNB / RAN: radio access, radio scheduling, mobility, the N2/N3 interfaces. 5G Core: registration, sessions, policy, subscriber data, user-plane forwarding.

Q577. What are the roles of the AMF, SMF, UPF, PCF, UDM, AUSF and NRF?

- AMF: access, mobility, NAS termination.
- SMF: session management, UPF control (PFCP), IP allocation.
- UPF: user-plane forwarding, GTP-U, QoS enforcement (QER), usage reporting.
- PCF: policy (QoS and charging rules).
- UDM: subscription data and credentials.

- AUSF: authentication.
- NRF: network-function discovery.

(Plus SCP, NSSF, BSF, UDR in our deployment.)

Q578. Control-plane signalling vs user-plane traffic?

Control: NAS and NGAP (N1/N2), service-based HTTP/2 between network functions, PFCP (N4). User: GTP-U-encapsulated packets (N3) and plain IP (N6). CARS-SF acts only on the user plane (UPF egress).

Q579. What is a PDU session?

A logical connection between a UE and a data network (DNN) through a UPF, with an IP address and one or more QoS flows, created for a given S-NSSAI.

Q580. What is a QoS flow, and how does it relate to a PDU session?

The finest QoS granularity in 5G: packets are mapped by filters (SDF templates) to a QoS flow (identified by QFI) within a PDU session, each with a 5QI and parameters (GBR/MBR, ARP).

Q581. What are 5QI values, and how do they relate to QoS treatment?

Standardised indices (3GPP TS 23.501) mapping to a resource type (GBR, non-GBR, delay-critical), priority, packet delay budget and error rate. For example 5QI 1 is conversational voice and 5QI 9 is best-effort default.

Honest limit. All our subscribers use 5QI 9. Differentiation comes from CARS-SF's UPF qdiscs, not 5QI.

Q582. What is the difference between a network slice and a QoS flow?

A slice is an end-to-end logical network (S-NSSAI) with its own network-function instances, policy or resources. A QoS flow is a traffic treatment class inside a PDU session within a slice. CARS-SF's "service slice" maps to an S-NSSAI plus DNN in the 5G deployment.

Q583. What is an S-NSSAI?

Single Network Slice Selection Assistance Information: identifies a slice as SST plus an optional SD.

Q584. What are SST and SD?

SST: Slice/Service Type (1 eMBB, 2 URLLC, 3 MIoT standardised). SD: Slice Differentiator, distinguishing slices of the same SST. We use healthcare SST 2/SD 000001, education 1/000002, agriculture 3/000003, general 1/000004.

Q585. What is the difference between eMBB, URLLC and mMTC?

Enhanced Mobile Broadband (high throughput), Ultra-Reliable Low-Latency Communication (strict latency and reliability), massive Machine-Type Communication (many low-rate devices). These are technology classes; CARS-SF argues for service classes on top (the 3GPP baseline in the main study maps applications to them).

Q586. What is the RAN's role in allocating radio resources?

The gNB's MAC scheduler allocates PRBs per TTI among UEs and QoS flows (and slices, with RAN slicing). This is where most capacity contention lives in real cellular networks. It is absent from our testbed.

Q587. Why can transport-layer bandwidth allocation not, by itself, guarantee radio-layer QoS?

If the radio is the bottleneck (poor coverage, cell congestion), packets queue or are lost in the RAN before reaching the backhaul. Shaping at the UPF or rural uplink then shapes a link that is not the constraint. CARS-SF's allocation is meaningful when the backhaul or transport is the bottleneck, which is our stated rural assumption.

Q588. What is the role of edge computing in rural connectivity?

Hosting critical services (clinic, agri) at the edge keeps them reachable and fast when the backhaul fails or degrades. Our topology shows this: edge-hosted healthcare survives a core switch failure, while core-hosted services cannot.

Q589. How could backhaul constraints interact with radio-resource constraints?

They form two bottlenecks in series. If they are allocated inconsistently (the RAN favours education, the backhaul favours healthcare), the tighter one dominates and capacity is wasted. Joint slicing (RAN plus transport) would need a shared policy, e.g. the same weights pushed to RAN-slice quotas and to transport HTB.

Q590. Which of these concepts are implemented in CARS-SF, and which are background?

Implemented and run (L3, simulated RAN):

- registration, PDU sessions;
- four S-NSSAIs and DNNs with their own address pools;
- GTP-U user plane through the UPF;
- CARS-SF enforcement on UPF N6 and `ogstun`.

Background only: 5QI differentiation, PCF policy, RAN scheduling and slicing, QoS flows beyond the default, mobility.

34 Open5GS, UERANSIM and deployment claims

Q591. What is Open5GS?

An open-source implementation of the 4G EPC and 5G Core (AMF, SMF, UPF, NRF, SCP, AUSF, UDM, UDR, PCF, NSSF, BSF), used here as version 2.7.2 in a single container.

Q592. What is UERANSIM?

An open-source 5G UE and gNB simulator: it implements NAS, RRC-level procedures and GTP-U, but not a physical radio layer. Version 3.2.6.

Q593. Which components does each provide?

Open5GS: the full 5GC network functions. UERANSIM: `nr-gnb` (NGAP over SCTP, GTP-U) and `nr-ue` (registration, PDU sessions, TUN interface `uesimtun0`).

Q594. What is needed for a working UE-to-core-to-data-network path?

- Matching PLMN (999/70), TAC and slices in gNB, AMF and NSSF.
- Subscribers in MongoDB (keys, OPc, slice and DNN).
- SMF session pools per DNN; UPF with `ogstun` carrying each slice gateway.
- The UPF running as root (TUN creation).
- SCTP to the AMF (start order mattered: the gNB does not retry).
- Routes from UEs via `uesimtun0`, and from the data network back to the slice pools via the UPF's N6.

All of it is in `deployment/5g/` and was brought up and verified (ping in both directions).

Q595. Where would CARS-SF sit in that architecture?

As an application function at the edge. Today it is a controller that enforces on the UPF's N6 and `ogstun` egress via tc. A standards-aligned version would sit behind the PCF or NEF (an AF influencing policy) and steer QERs.

Q596. Which actual interfaces would CARS-SF use to observe and influence the 5G system?

Now: Linux interfaces on the UPF host (tc stats, counters) and captures on N6 and the UE TUNs. **Standards path:**

- NWDAF or UPF usage reports (observation);
- Npcf / N5 via the AF, and NEF for policy;
- SMF → UPF QERs (MBR/GBR) over N4;
- RAN slice quotas via O-RAN A1/E2 for radio.

Q597. How would an allocation decision become a supported 5G configuration?

Map slice allocations to session or QoS-flow AMBR/MBR/GBR and QER updates (through the PCF, as policy for each S-NSSAI and DNN), and to RAN-slice PRB quotas. Our tc shaping on the UPF host is a pragmatic stand-in with the same effect on the N6 bottleneck.

Q598. Which 5G functions would need modification?

None for the demonstrated mode (tc on the UPF host is outside 5G functions). For a standards-aligned integration: none in principle, using the PCF with AF influence and QER-based enforcement in the UPF. In practice, Open5GS's support for dynamic QER updates driven by an AF would need checking or extending, and the RAN side needs an E2-capable gNB (not UERANSIM).

Reconstructed questions

The questionnaire was truncated at Q598. Questions 598a onward cover the rest of the 5G section, experiments and critical evaluation, written to match the questionnaire's intent and its stated headings ("Experiments · Critical evaluation").

Q598a. What exactly did you run on the 5G core, and at which evidence level?

- Open5GS with four slice UEs (UERANSIM) and PDU sessions on their own S-NSSAI/DNN pools.
- CARS-SF traffic generators in sidecars sharing each UE's namespace.
- A 10 Mb/s UPF bottleneck shaped by CARS-SF, compared with no slicing, in the disaster and school contexts.
- Captures on `uesimtun0` (raw IP) and on the data network, matched with the same code.

L3 validated (Open5GS core, simulated RAN). It is not L4: the radio is simulated.

Q598b. What did the 5G experiment show?

Disaster context, 10 Mb/s UPF:

- healthcare p95 349.0 ms with CARS-SF vs 797.9 ms without;
- healthcare SLA-met share 0.528 vs 0.037;
- agriculture p95 22.8 vs 795.5 ms.

Trade-off: weighted throughput satisfaction 0.848 vs 0.868, because fq_codel drops some of healthcare's bursty video in exchange for low delay. School: healthcare p95 4.5 vs 5.4 ms. Healthcare is uncontended there; the report marks H8 ("lower in every scenario") **supported**, but this difference is about 1 ms with $n = 3$ and had the opposite sign before the audit, so I claim only the disaster effect; weighted utility still improves (0.731 vs 0.674).

Q598c. Why a 10 Mb/s bottleneck on the 5G core, not 100 Mb/s?

At 20 Mb/s, healthcare showed about 390 ms p95 in *both* modes. The queue was in UERANSIM's user-space GTP path, not at the UPF. At 10 Mb/s the UPF is the contention point, which is what the experiment needs. This is a documented limitation of UERANSIM, not of CARS-SF.

Q598d. Does slicing at the UPF with tc constitute 5G network slicing?

Partially. Slice *identity* is genuine 5G (S-NSSAI, DNN, PDU sessions through the 5GC); slice *resource enforcement* is CARS-SF's tc on the UPF host, not 3GPP QoS (5QI, QER, PCF). Say exactly that.

Q598e. How would you reach L4 evidence?

Replace UERANSIM with an SDR-based gNB (e.g. a USRP B210 with an open-source RAN stack) and commercial UEs on a private PLMN, as in the repository's existing 5G testbed proposal (`existing/5G_TestBed`). Then run the fiber-cut and disaster scenarios over the air, with a measured backhaul bottleneck.

Q598f. What were the deployment pitfalls, and why do they matter for the defense?

- The image runs as non-root, so TUN creation failed.
- A read-only `/proc/sys`.
- MongoDB readiness.
- YAML indentation of `s_nssai`.
- The gNB connected 16 ms before the AMF listened and did not retry.
- Captures on a TUN device use LINKTYPE_RAW (101), which our parser initially dropped.

Each was found and fixed by running end to end. That is evidence the result is from a working system, not a diagram.

Q598g. What is the evidence level of the OVS QoS backend and of real-OVS operation generally?

Real OVS 3.3.9 with a genuine `OVSBridge`, run in a privileged container (**L3 validated (real OVS, emulated net)**):

- a fiber-cut run: recovery 0.094 s, outage 0.33–0.53 s;
- a school run with the `linux-htb` backend: no false failover after the congestion fix, validation 93.

Honest limit. Only a handful of runs. The 340-run matrix used Linux bridges.

Part XIII

Experiments, statistics and reproducibility

Reconstructed section (the questionnaire was truncated; see the note in Part XII).

35 Experimental design

QE1. Which experiments did you run?

340 emulated runs in 8 studies (`configs/experiments/thesis.yaml`), plus 18 dataset-collection runs, real-OVS runs and 5G-core runs:

Study	Design	Runs
main	7 schedulers × 5 contexts × 5 seeds	175
ablation	6 variants × 2 contexts × 3 seeds	36
capacity sensitivity	2 schedulers × 2 contexts × {50,100,150} × 3	36
load sensitivity	2 × 2 × {0.5,1,1.5} × 3	36
scalability	UEs per service {1,2,4} × 3	9
resilience	3 failures × 3 healing variants × 3	27
timeline	3 schedulers × 3 seeds (rural day)	9
in-loop SliceNet	2 contexts × {host, slicenet} × 3	12

QE2. Why 40 s runs, 5 s windows and a 1 s tick?

40 s gives 8 windows per flow and 8 control intervals, enough for steady state after warm-up, and 340 runs fit in about 1.5 h with 4 workers. Resilience runs are 45 s so that failure, recovery and repair all fit.

Honest limit. Long-run effects (minutes to hours: TCP fairness drift, diurnal patterns) are not captured.

QE3. Why 5 seeds in the main study and 3 elsewhere? Is that enough statistical power?

The main study pairs 25 (context, seed) observations per comparison, which suits Wilcoxon. With the observed spread (CIs of about ±0.01–0.03 on utility), it can detect differences of about 0.02 or more. The observed allocator differences (<0.012) are below that, so "no significant difference" means "any difference is small", not "no difference".

Honest limit. 3-seed studies are descriptive: CIs are reported, no tests.

QE4. How are seeds paired across schedulers?

The same seed produces the same per-application traffic schedules for every scheduler in a context, so schedulers face identical offered traffic, and comparisons are paired by (context, seed).

QE5. Why the Wilcoxon signed-rank test rather than a t-test?

Paired differences are not guaranteed to be normal (bounded metrics, heavy tails in delay). Wilcoxon is distribution-free and robust, at a small loss of power.

QE6. How do you handle multiple comparisons?

Bonferroni over all scheduler × metric tests ($\alpha = 0.0012$). Both nominal and corrected significance are reported. The healthcare-SLA effect of slicing survives correction ($p = 0.0010$).

QE7. Do you report effect sizes?

Mean paired differences with CIs, in the metric's own units: utility, ms, shares.

Honest limit. No standardised effect sizes (Cohen's d or the rank-biserial correlation); easy to add.

QE8. Were hypotheses fixed before seeing results?

`docs/thesis/research_design.md` was written before the full matrix ran, and git history timestamps it. Verdicts are computed mechanically by `experiments/report.py`.

Honest limit. The design and the evaluation were developed in the same project period, not externally pre-registered.

QE9. Which metrics are primary, and why?

Primary: priority-weighted demand satisfaction (the allocation objective as stakeholders weigh it) and the healthcare SLA-met share and p95 delay (critical-service protection). Secondary: fairness, loss, jitter, outage, MTTD and MTTR.

QE10. Isn't weighted utility circular, since the proposed scheduler is designed with those weights?

Partly, yes. It is the stakeholders' objective, so a scheduler using it has an advantage by design. That is why it is reported alongside unweighted metrics (Jain fairness, per-slice satisfaction, SLA-met share). Even with this home advantage, the proposed scheduler is not significantly better, which makes the null result stronger, not weaker.

QE11. Why p95 delay rather than mean delay?

Critical applications fail on tail delay (late ECG samples, frozen video), not on averages. p95 is computed per window and summarised across windows; p99 is also reported.

36 Validity of the measurement campaign

QE12. Were any runs invalidated? Why?

Yes, twice, all archived and fully re-run:

1. **Busy-wait pacing** (111 runs): generators spun at about 100% CPU each; host load reached 40–56 on 20 cores, perturbing kernel timers and therefore delays. Fixed with sleep-only pacing (7% CPU per 40 Mb/s).
2. **Pre-OVS-validation** (340 runs):
   - uplink-only utilisation (downlink saturation invisible);
   - false failover under AQM-less queues;
   - slow failover on OVS;

   all discovered by running real OVS, fixed, and the matrix re-run.

Archives: `results/runs_archive_*`.

QE13. What other measurement bugs did you find?

- **Partial final window**: traffic stops mid-window, and a stray near-empty last window divided delivered rate by N+1 instead of N ("static beats proposed at low load" was this artefact). KPIs are now recomputed from each run's dataset excluding the final window, uniformly for every run.
- **Mislabelled RSNSD split**: the RSNSD baseline was reported as a "group split" when it was a random row split (Q347).
- **Hidden propagation delay**: netem on the bottleneck was silently replaced by HTB (delay placement redesigned).

QE14. How do you know the measurement is right?

- Delay floors equal the configured propagation to within 0.01 ms (8.00/28.00 ms).

- Unit tests feed synthetic captures with known one-way delay, loss and jitter and recover them exactly.
- Probe RTT/2 agrees with the measured one-way delay.
- All runs have packet-level data (1,550/1,550).

QE15. Could parallel runs interfere with each other?

Each run has its own network namespace, capture directory and log directory, so packets and files cannot mix. Only CPU is shared, and it is bounded by 4 workers (load stays below the 20 cores; no process spins).

QE16. Are results from Linux bridges valid for Open vSwitch?

The allocation and QoS mechanisms are identical (same tc qdiscs). Healing differs in timing: OVS failover took 0.094 s and had a 0.33–0.53 s outage, against about 0.05 s on bridges. Conclusions hold (sub-second outage) but absolute numbers differ, and we report both.

QE17. How reproducible is a single figure?

Every figure is generated from `results/tables/*.csv` by `plot_thesis.py`; the tables come from per-run artefacts via `aggregate.py`; each run is reproducible from its `run_meta.json` (parameters, seed, commit). Bit-identical packet timing is not expected; distributions are.

QE18. What does the full study cost to reproduce?

About 1.5 h for the matrix (4 workers, 20-core machine), minutes for training, under 1 h for the 5G runs. No GPU. About 100 MB of captures per run (gitignored, regenerable).

37 Results by study

QE19. Main study: what is the result?

- Weighted utility 0.944–0.955 across the seven schedulers, with no significant difference.
- Healthcare SLA-met share: 0.890 (proposed) vs 0.773 (no slicing), $\Delta = 0.117$, $p = 0.0010$.

Interpretation: *enforcing* service slices with AQM matters; *how* the shares are computed matters little when borrowing is enabled.

QE20. Ablation: what is the result?

School:

- strict caps: utility 0.865→0.789, healthcare p95 10.0→165.0 ms;
- no surplus redistribution (strict): →0.651;
- no AQM: 0.903, *higher*, because nothing is dropped; but general p95 160→1433 ms and disaster healthcare SLA share 0.838→0.597;
- no context: 0.861 (identical to full);
- oracle demand: 0.889 (an upper bound on what better demand estimation could add).

Borrowing is the dominant mechanism; surplus redistribution substitutes for it when borrowing is disabled; AQM trades throughput for delay and SLA compliance; context awareness has no measurable effect.

QE21. Sensitivity: what is the result?

At 0.5× load all schedulers satisfy almost everything (static even edges ahead: 0.976 vs 0.969). At 1× the proposed scheduler leads (0.879 vs 0.831); at 1.5× (0.774 vs 0.743). H7 ("advantage everywhere") is not supported. The advantage is conditional on scarcity.

QE22. Resilience: what is the result?

- Fiber cut: outage 0.165 s (full) vs 15.126 s (none).
- Degradation: 0.019 s (predictive) vs 0.214 s (reactive) vs 15.310 s (none).
- Core switch: 15.118 s in all variants; healing cannot fix it, but localises the fault in 100.0% of runs.

QE23. Timeline and scalability: what is the result?

Timeline (a rural day with school and flood): all three schedulers adapt within one control interval of each context change. Utility differences are within CIs (`timeline_summary.csv`). Scalability (1, 2, 4 UEs per service): the pipeline and controller scale, and KPIs stay stable (`scalability_summary.csv`).

Honest limit. Scale is tiny compared with real deployments.

QE24. In-loop SliceNet: what is the result?

Mixed-UE SLA-met share:

- school: 0.958 (SliceNet) vs 0.798 (address-based); its education flows' p95 fell from 452.7 to 91.7 ms;
- disaster: equal (1.000 vs 1.000);
- online classification accuracy 0.750.

H5b required gains in both contexts, so it is not supported. The benefit appears only where the mixed UE's misplaced traffic actually contends.

QE25. Which interactions between factors were not tested?

- ablations combined (no AQM × strict caps);
- failures during timelines or under overload;
- RL policies under failures;
- SliceNet under failures;
- capacity × load jointly.

Part XIV

Critical evaluation, limitations and the hostile round

Reconstructed section.

38 Critical evaluation

QC1. What is the single biggest weakness of this thesis?

No real-network evidence: everything is L3 in emulation or on a 5G core with a simulated RAN, and the traffic and demands are modelled, not measured. Every quantitative claim is conditional on those models.

QC2. What is the second biggest weakness?

The headline mechanism comparison is null. The demand-aware allocator and RL are not significantly better than simple slicing. The defensible contribution therefore shifts to "service slicing with AQM, plus healing, measured rigorously", which is narrower than the original vision document claimed.

QC3. Which original claims did you retract or narrow, and why?

- "AI-driven allocation improves service quality": not supported. RL does not beat the heuristic on the network (final C7, C8); online learning only beats its own learning-disabled control (C9).
- "Context awareness drives the gains": not supported (H9). The gains came from measuring loss and allocating per direction for SLA targets (§46).
- "RSNSD enables classification of rural traffic": refuted for measured traffic (transfer F1 0.104).
- "Self-healing with load balancing": load balancing not implemented.
- "QoS Enforcer: prototype" in the original status table: the Mininet path had never run when the verification began.

QC4. What result surprised you most?

That HTB borrowing (a data-plane mechanism) erases most differences between allocation policies: once idle capacity is lent at packet timescales, control-plane share computation matters only under strict caps or heavy overload. That reframes where engineering effort should go: AQM and enforcement, not clever allocators.

QC5. What would you do differently if you started again?

- Build the measurement harness and baselines first, before mechanisms.
- Run on real OVS from day one (the false-failover bug was invisible on bridges).
- Pre-register hypotheses externally.
- Seek real traces early.
- Spend less on RL and RSNSD scale, more on real-world validity.

QC6. Does the thesis generalise beyond the studied topology?

Mechanism-level conclusions (slicing with AQM protects critical flows; loop-free protection switching gives sub-second recovery) are standard enough to generalise to single-bottleneck networks. Quantitative results do not transfer to multi-bottleneck, radio-limited or larger networks without new experiments.

QC7. Is there enough here for a master's thesis?

Yes, on method and evidence:

- a working integrated system;
- a validated packet-level measurement method;
- a 340-run, statistically analysed evaluation with negative results reported;
- real-OVS and 5G-core replications;
- documented discovery and correction of methodological errors.

The contribution is empirical and architectural rather than algorithmic, and it should be presented that way.

QC8. What are the ethical implications of prioritising some services over others?

Prioritisation encodes value judgements (whose traffic matters in a disaster) that should come from accountable local stakeholders, not engineers. Floors guarantee minimum access for everyone; context weights should be governed, transparent and auditable (the history log). Misuse (surveillance-driven classification, political prioritisation) is a real risk. SliceNet classifies only coarse service classes from metadata.

QC9. What are the privacy implications?

Traffic-shape features are metadata but can still reveal activity. Deployment must capture headers only (or compute features on the fly without storage), keep classification on site, and avoid per-user profiling. Our evaluation captures included about 60 B of synthetic payload for matching, which would be unacceptable on real traffic.

QC10. What are the security risks?

- Spoofed context alerts (forcing the disaster profile).
- A compromised controller (arbitrary QoS, link shutdowns).
- Evasion of SliceNet (mimicking healthcare traffic shapes to gain priority).

Mitigations: authenticated feeds, least privilege for controller commands, per-flow caps inside critical classes, anomaly monitoring on classification.

QC11. What would block a real rural deployment?

Access to a standby backhaul (cost); skilled operators; power reliability for the controller and switches; integration with the operator's 5G policy framework (PCF/QER instead of host tc); and stakeholder governance of priorities.

QC12. What are the next six months of work?

1. An L4 trial on an SDR-based private 5G testbed with a real backhaul bottleneck.
2. Real labelled traces and re-validation of SliceNet.
3. Standards-aligned enforcement (QER via PCF).
4. Controller resilience (state persistence, local fail-safe agent).
5. Multi-village topology with multipath healing (load balancing).

QC13. Which parts were AI-assisted, and how did you verify them?

Answer truthfully for your own case. Name the tools and say which components they helped with. The verification was independent of authorship: unit tests on exact values, end-to-end runs on two switch implementations and a 5G core, measurement cross-checks (floors, probe vs one-way delay), and the corrections documented in E12–E13. Examiners care most that you understand and can defend every line. Rehearse the code walkthroughs in Q131–Q132.

39 The hostile round: one-line answers

QH1. "Isn't this just Linux tc with extra steps?"

The enforcement *is* tc, deliberately. The thesis is about what to configure, when, and how to recover, and about measuring that the outcome is right. The results say the configuring matters less than expected, and we report that.

QH2. "Your RL doesn't work. Why is it in the thesis?"

Because whether learning beats a good heuristic here is a legitimate research question, and a rigorously measured negative answer (paired, Bonferroni-corrected, in emulation) is a result. It is presented as such, not as a contribution.

QH3. "Your classifier is trained on traffic you generated. Isn't the F1 meaningless?"

It is an upper bound on separability of our models, evaluated on held-out runs and devices, with shortcuts removed. The transfer study (0.104 zero-shot) shows exactly how generator-dependent such numbers are. Real-trace validation is stated as the open gap.

QH4. "Mininet isn't a real network."

Correct, so everything is labelled L3 in emulation. Mininet runs the real kernel stack, real TCP and real qdiscs; we replicated the key results on real Open vSwitch and on a real 5G core; and we never claim field performance.

QH5. "Most of your hypotheses failed. Is the thesis a failure?"

No. Five pre-registered hypotheses were supported (ablation, healing outage, predictive switchover, SliceNet F1, transfer gap). Six were not, and they are reported. A thesis that only confirms its hypotheses should worry a panel more.

QH6. "Self-healing is just protection switching from the 1990s."

The switching is standard; what we add is coupling it to slice allocation (backup ceilings, degraded mode), loop-free ordering without STP, OAM-gated fail-back, congestion-aware prediction, and packet-level outage measurement.

QH7. "Your OAM check reads the emulator's netem configuration. Isn't that cheating?"

It is a stand-in for Y.1731 or BFD measurement on the standby path, which reports the same delay and loss in reality. It is documented as such in the code and the thesis. It only gates fail-back, and it does not affect any measured outage.

QH8. "How do I know you didn't tune on the test set?"

No hyper-parameter search was done (fixed defaults). Thresholds were set by design and changed only for documented bugs (which triggered full re-runs). SliceNet test groups are disjoint runs and devices.

QH9. "Your 5G part isn't really 5G slicing."

Slice identity is genuine 5G (S-NSSAI, DNN, PDU sessions on Open5GS). Enforcement is CARS-SF on the UPF host, not 3GPP QoS. We say exactly that, and give the standards path (PCF, QER, RAN quotas).

QH10. "If you had to cut half the thesis, what stays?"

Service slicing with AQM and its measured critical-SLA benefit; loop-free self-healing with sub-second outage; the measurement methodology. RL, RSNSD scale and the 5G stretch go.

QH11. "What if I run it now and it fails?"

`makesetupdatamodelstestintegration` runs without sudo. The real-OVS path runs in Docker (`dockerrun--privilegedcars-sf`). Every reported number comes from scripted pipelines over stored artefacts. Offer a live run of Q528.

QH12. "Why should a rural planner trust demands and SLAs you assumed?"

They shouldn't trust the numbers; they can reuse the mechanism. Weights, floors and SLAs are registry data meant to be set by local stakeholders, and the evaluation method can be re-run with their values.

QH13. "Your sample sizes are tiny."

25 paired observations per main comparison, with CIs everywhere; 3-seed studies are presented descriptively. The null allocator result is "differences below about 0.02 in utility", a bounded claim, not proof of zero.

QH14. "Couldn't DiffServ marking with static queues give the same result?"

Largely, for the steady state: our own static and 3GPP baselines are close. What it cannot do is healing coupled to allocation, or slicing devices that cannot be trusted to mark (in-loop classification for the mixed UE).

Part XV

QoS-targeted control: making the slices meet their SLAs on the go

Goal of this part

After the engineering audit, the goal was restated as: *SliceNet and RL should slice the network so that the four classes each get the bandwidth they need to meet their QoS requirements, adapting on the fly, predictively or reactively, but effectively.* This part records what stood in the way, the six modules built to close the gap (A–F, Figure 15), and what the measurements say. Every module was regression-tested on the emulated network before the next one started. Development numbers marked *(dev, n=2)* come from two-seed checks used to steer the design; the verdicts come from the contended-regime evaluation (§46).

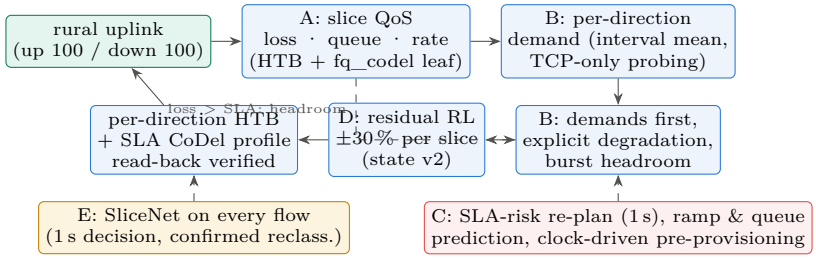


Figure 15: QoS-targeted control added after the audit (`core/controller.py`, `slicing/allocators.py`, `scheduler/residual.py`, `classification/online.py`).

40 Diagnosis: why the audited system could not meet the goal

Q701. In the audited system, where did the classes actually miss their SLAs?

Only in the emergency contexts, and only on loss. In the main study, healthcare met its SLA in every window in normal, school and harvest. In disaster and medical emergency it failed in 22 % and 33 % of windows. Delay and jitter never failed; loss above 1 % did, all of it from telemedicine video (3.5–6 % loss). ECG and voice lost nothing, because fq_codel's sparse-flow priority already protects small flows inside a slice.

Evidence. L3 validated (emulation) `experiments/qos_compare.py` on the main-study runs; per-application loss from the measured datasets.

Q702. Why could neither the controller nor the RL agent react to that loss?

They could not see it. The drops happened in the fq_codel leaf under each HTB class (CoDel drops at dequeue). The collector read only the HTB class counters, which stayed at 0 in every one of those runs. The reward's loss term was zero exactly when it mattered.

Evidence. L3 validated (emulation) a 40 s disaster run with leaf counters reads ≈274 drops per tick where the class counters read 0.

Q703. What else was structurally wrong?

Three things.

1. **Direction.** The link is full duplex and HTB shapes each direction separately, but demand was summed over both directions and compared with one direction's capacity. All healthcare applications are uplink (90 Mb/s in a disaster fits the 100 Mb/s uplink), and education and general are downlink.
2. **Queue.** The default CoDel target (5 ms) trimmed video bursts that healthcare's 50 ms delay budget could easily absorb.
3. **RL's lever.** RL only rescaled priority weights, which feed an allocator whose slices borrow idle capacity anyway. Live, PPO chose multipliers of about 1.

41 Module A: QoS visibility

Q704. What does the controller now measure per slice, and how?

Every tick, per slice and interval:

- loss = queue drops / (sent + drops), over HTB class plus leaf qdisc, both directions;
- queueing delay = standing queue / the slice's service rate;
- delivered rate.

The result is `slice\_qos` in every snapshot (`telemetry/collector.py`). The SLA monitor, the demand estimator, the reward, the RL state and the feedback log all read the same numbers.

Why. The bottleneck is the only contention point in the topology, so its own counters are a direct, cheap and complete measurement of slicing-induced loss and delay.

Choice. Kernel counters every tick.

Alternative. Per-flow measurements from live captures are more end-to-end, but cost a capture parse per tick. They remain the offline ground truth (dataset KPIs).

Trade-off. The queue estimate is a 1 s snapshot, not a per-packet sojourn time.

Evidence. L2 unit-tested L3 validated (emulation) `tests/test_telemetry_health.py`; module A regression.

Q705. Does the reward now punish loss properly?

Yes. A slice whose loss exceeds its SLA loss target (healthcare 1 %, education and agriculture 2 %, general 5 %) now counts as an SLA violation (−10), in addition to the mean-loss term. The same function serves the simulators, the live feedback log and the RL transitions.

Evidence. L2 unit-tested `tests/test_reward.py::test_loss_above_sla_target_is_a_violation`.

Q706. Did making loss visible break anything?

Yes, and the regression caught it. The predictive backhaul detector treated *any* drop as local congestion and suppressed itself. With fq_codel drops visible, steady TCP sawtooth drops muted it.

- First fix: use the core-bound slices' queueing delay (> 10 ms) as congestion evidence, not drops. Proactive switchover recovered from 1/3 to 3/4 runs.
- Second fix: isolated single-tick TCP queue spikes still reset its 6-tick window, so congestion now suppresses prediction only when it persists in at least half the window.

Evidence. L3 validated (emulation) switch-over before the cut in 5/5 repeated live runs; `test_isolated_queue_spikes_do_not_mask_a_backhaul_trend`.

42 Module B: allocate for QoS targets, per direction

Q707. How does the QoS-aware allocator work?

Per direction, with capacity C each (`QoSAwareAllocator`):

1. floors (a 0.5 Mb/s reverse floor in a slice's light direction);

2. **demands first**: if $\sum_s \max(f_s, d_s) \le C$ every slice gets its demand; otherwise slices are served in descending context priority after reserving the others' floors, and those left short are listed as *degraded*;
3. **burst headroom** from the spare capacity, up to $d_s(h_s - 1)$, first to slices currently failing their SLA, then by priority;
4. leftover capacity to the slices that use this direction.

Why. The goal is SLA compliance, not proportional fairness. When demand does not fit, which class degrades is a policy decision, and it should be explicit and auditable.

Alternative. A weighted split (the audited `proposed`) degrades every class a little, implicitly.

Trade-off. Strict priority degrades low-priority slices fully when the link is overcommitted.

Evidence. L2 unit-tested `tests/test_qos_aware.py`.

Q708. What is the headroom loop?

Each slice starts with headroom $h_s = 1.1$. If its mean loss over the last interval exceeds its SLA loss target, $h_s \leftarrow \min(1.6, 1.15\,h_s)$ and the slice is marked at risk. After three intervals below 25 % of its target, $h_s \leftarrow \max(1.1, 0.97\,h_s)$. This is a per-slice AIMD-like loop closed on measured loss.

Evidence. L2 unit-tested `test_headroom_rises_with_loss_and_relaxes_when_quiet`.

Q709. Why derive per-slice CoDel parameters from the SLA?

CoDel drops once the standing queue exceeds its target for an interval. Healthcare's one-way budget is 50 ms, with 8 ms of propagation, so a 12 ms target with a 120 ms interval trades a few ms of queueing for far fewer burst drops. Education and agriculture get 25 ms / 250 ms; general keeps the 5 ms / 100 ms defaults.

Trade-off. Higher queueing delay and jitter inside the budget. The measured healthcare p95 stayed at 8–10 ms (dev).

Q710. Which demand does the direction-aware allocator plan for?

The mean offered rate over the last control interval, per direction.

- Elastic probe-up (+25 %) applies only to directions where the service has TCP traffic, and only when a standing queue proves saturation.
- The first version followed per-second samples with a trend forecaster. On UDP video it estimated 228 Mb/s for a 95 Mb/s flow (ramp-up trend plus probe-up on inelastic traffic), squeezed agriculture to its floor, and was replaced.

Evidence. L2 unit-tested L3 validated (emulation) `test_directional_estimate_probes_only_elastic_traffic_and_never_overshoots_peak`; development runs.

Q711. What did module B achieve on the network during development?

Healthcare SLA-met share:

- disaster: 0.79 → 0.93;
- medical emergency: 0.67 → 0.87.

Healthcare loss fell from 0.7–1.4 % to ≈0.2 %, and weighted utility rose in both contexts. Agriculture, which shares the uplink, fell from 1.0 to 0.94 and 0.75. Medical emergency offers 100 Mb/s of VBR uplink traffic on 100 Mb/s, so not every class can be met there: the allocator now degrades agriculture explicitly (context priority 15:1) instead of letting healthcare lose implicitly.

Evidence. L3 validated (emulation) *(dev, n=2)*; confirmed or refuted in §46.

43 Module C: react within a second, act ahead when possible

Q712. What is reactive, and what is predictive?

- **Reactive.** A slice breaching its loss SLA, or with queueing delay above half its delay budget, triggers a re-plan on the next 1 s tick instead of waiting up to 5 s. A 2-tick hold-down prevents oscillation. Explicitly degraded slices do not retrigger, because re-planning cannot help them. Queue-based triggers ignore TCP slices: TCP fills its own queue by design.
- **Predictive.**
  - a consistent demand ramp ($R^2 \ge 0.6$) is extrapolated two ticks ahead, capped at 1.5× the window peak;
  - a rising queue projected to cross the limit within two ticks triggers a re-plan;
  - context changes predictable from the clock and calendar (the school bell) are pre-provisioned two ticks early.

Honest limit. Alerts are not predictable. The scripted alert times in the timelines are never read: that would be cheating.

Evidence. L2 unit-tested L3 validated (emulation) `tests/test_qos_adaptive.py`; `test_qos_adaptive_preprovisions_before_clock_driven_context_change`.

Q713. Did module C measurably help in development?

Not detectably. The mechanisms fire as designed (pre-provisioning two ticks before the bell, SLA-risk re-plans), but on the two timelines `qos_adaptive` was within noise of `qos_aware` *(dev, n=2)*. Burst headroom and HTB borrowing already absorb most transients at a 5 s cadence, and context changes are rare. The five-seed evaluation decides (§46).

44 Module D: residual reinforcement learning

Q714. What does the redesigned RL control, and why that?

A residual on the QoS-aware allocation: $a \in [-1, 1]^4$ scales each slice's rate in its dominant direction by $(1 + 0.3a_s)$. Each direction is renormalised to C, with floors kept.

Why. RL acts directly on the quantity that decides QoS (rates), and the heuristic is a strong, safe starting point. a=0 *is* the heuristic, so RL only has to learn corrections.

Alternative. Priority multipliers (the earlier design), whose effect HTB borrowing mostly cancels.

Trade-off. RL replaces the hand-made headroom loop, so `rl\_residual` vs `qos\_adaptive` is a fair test of "learned vs designed adaptation".

Evidence. L2 unit-tested `tests/test_residual_rl.py` (floors and capacity hold for any action; a=0 reproduces the heuristic).

Q715. What does it observe?

State v2 (25 values), built by one function in both the simulator and the live controller:

- context one-hot;
- per slice in its dominant direction: demand / C and allocation / C;
- loss / SLA loss target, and queueing delay / SLA delay, measured over the last interval;
- the on-backup flag.

Q716. Why a new simulator, and how faithful is it?

The old simulator had one pipe, no bursts, no AQM and no TCP behaviour, so it could not express the failure seen live. The new one (`scheduler/env_dir.py`) models:

- both directions;
- 50 sub-steps of 100 ms per 5 s step;
- lognormal bursts;
- HTB borrowing by priority;
- CoDel burst tolerance ≈ rate × (target + interval);
- TCP backing off instead of dropping.

Burstiness and HTB's borrowing efficiency were fitted to 720 live ticks with per-slice loss, minimising per-run mean loss error: σ_{hc}=0.05, σ_{agri}=0.15, β=0.75.

Honest limit. The per-run RMSE is still 2.3 (healthcare) and 2.6 (agriculture) percentage points. The simulator guides training; only the network decides (Q725).

Q717. What did PPO learn in simulation?

Three seeds of 300k steps, best checkpoint chosen on validation seeds, evaluated on 30 paired held-out test episodes. Reward against the heuristic (a=0): +3.2 % to +3.6 %, significant for 3 of the 3 seeds. SLA violations fell from 18.2 to 5.5–7.3 per episode. Small, and it did not carry over to the network (Q725).

Evidence. L2 unit-tested `results/rl_training.json` (`residual`).

Q718. Can it keep learning on the real network?

`rl\_residual\_online` runs CEM over the residual action. It starts at a=0 (the heuristic), samples each live decision and learns only from transitions whose allocation the kernel confirmed, with a checkpoint carried across runs.

Evidence. L3 validated (emulation) `test_residual_online_rl_closed_loop_on_live_network`: state v2 with measured QoS, exploring actions, verified enforcement and policy updates from exactly the learnable transitions.

45 Module E: SliceNet as the classifier for every flow

Q719. How does SliceNet classify on the go?

In `slicenet\_all` mode there are no address rules: every UE flow starts best effort and is classified by SliceNet.

- Captures are read incrementally (only the new records each tick).
- A flow is decided once it has been seen for 1 s and 5 packets. Its features are expressed as a full 5 s window (rates over the observed span; counts scaled up).
- Every 5 s it is re-checked on a real full window. A different class predicted with confidence ≥ 0.8 on two consecutive checks replaces the rule.
- Rules are `tcflower` exact matches (hash lookup) with a handle per flow, so they can be replaced in place.
- The confidence gate and best-effort fallback stay in force.

Evidence. L2 unit-tested L3 validated (emulation) `tests/test_online_slicenet.py`; `test_slicenet_classifies_every_ue_flow_without_address_rules`. Development school run: 32 flows, median 1.57 s to decision, 81 % correct at first decision and 87.5 % after two (correct) reclassifications. Residual errors are education ↔ general (both web-over-TCP).

Q720. Why not classify inside a class (telemedicine vs ECG)?

The evidence did not call for it. Inside the healthcare slice, fq_codel already gives sparse flows (ECG, voice) priority over bulky ones: their loss was 0 % while video lost 3–6 %. The fix was capacity and burst tolerance for the slice, not a finer class.

46 Module F: evaluation in contended regimes

Design of the evaluation

- **Conditions.** Only conditions in which slices compete: disaster and medical emergency at 100 Mb/s; school and harvest on a 50 Mb/s uplink; school and disaster at 1.5× load; and the two timelines (a clock-driven school bell and unpredictable alerts).
- **Schedulers.** static_equal, context_unaware, proposed (the audited heuristic), qos_aware (B), qos_adaptive (B+C) and rl_residual (B+C+D), all on host classification.
- **Size.** Five seeds per cell, 240 runs, paired by (condition, seed) against `qos_adaptive`.
- **Extra studies.** SliceNet everywhere (E) against address rules on the same controller; online residual RL across consecutive runs.

The verdicts below come from the **confirmatory run v4 (seeds 15–19)**. Earlier rounds found defects; see Q732.

	healthcare SLA share			agriculture SLA share			general SLA share		
condition	prop.	B+C	RL	prop.	B+C	RL	prop.	B+C	RL
disaster	0.775	0.928	0.928	1.000	0.794	0.766	1.000	1.000	1.000
medical emergency	0.672	0.897	0.897	1.000	0.680	0.685	1.000	1.000	1.000
disaster 1.5×	0.500	0.525	0.519	0.987	0.628	0.632	1.000	1.000	1.000
harvest 50 Mb/s	1.000	0.944	0.922	0.746	1.000	1.000	0.979	0.987	0.997
school 50 Mb/s	1.000	1.000	1.000	1.000	1.000	1.000	0.934	0.862	0.871
school 1.5×	1.000	1.000	1.000	1.000	1.000	1.000	0.991	0.956	0.950
rural day (timeline)	0.912	0.969	0.966	1.000	0.973	0.965	0.962	0.896	0.852
medical night (timeline)	0.787	0.941	0.944	1.000	0.791	0.794	1.000	1.000	1.000

Table 1: Share of flow windows meeting their application's SLA, per class (mean over 5 seeds; confirmatory run v4). prop. = `proposed` (audited heuristic), B+C = `qos_adaptive`, RL = `rl_residual`. Education met its SLA in nearly every window under every scheduler and is omitted. Generated from `results/tables/qos_summary.csv`.

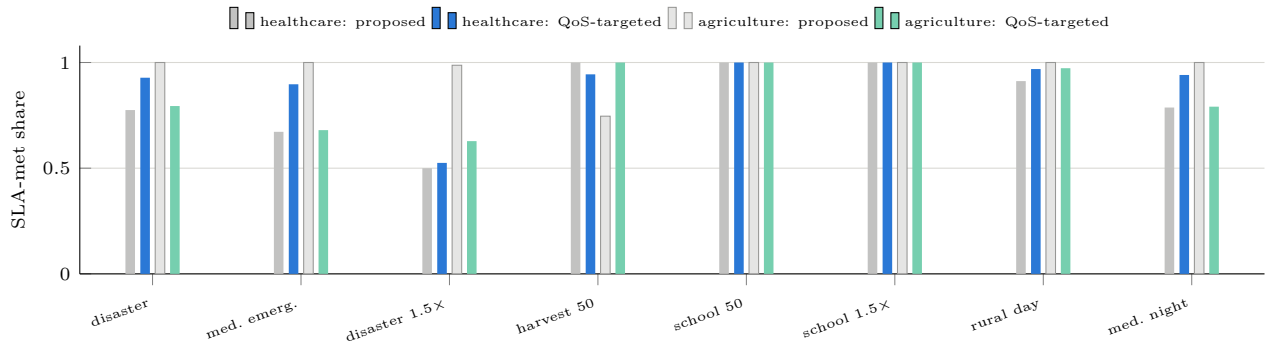


Figure 16: Healthcare gains where it competes for the uplink; agriculture pays in the emergencies (declared priority) and gains on the 50 Mb/s harvest uplink. Confirmatory run v4, mean of 5 seeds (`results/tables/qos_summary.csv`).

Q721. Did the QoS-targeted control make the critical class meet its SLA where it previously did not?

Largely yes. Over the 40 paired runs, healthcare's SLA-met share rises by 0.0695 against `proposed` ($p = 0.0002$, significant after Bonferroni). The healthcare-loss difference (QoS-targeted − proposed) is -0.8959 percentage points.

- Disaster: 0.775 → 0.928.
- Medical emergency: 0.672 → 0.897.
- Night-time medical alert: 0.787 → 0.941.

It is not a full fix. At 1.5× disaster load (135 Mb/s of healthcare on a 100 Mb/s uplink) no allocation can meet everyone, and healthcare stays near 0.525.

Evidence. L3 validated (emulation) `results/tables/qos_stats.csv`.

Q722. Then why is H11 "not supported"?

H11 was stated on the *unweighted mean* over the four classes, and on that metric `qos_adaptive` is -0.0155 below `proposed` ($p = 0.0050$). The gain for healthcare is paid for by agriculture, which shares the uplink and has context priority 1 against healthcare's 10–15 in the emergencies:

- disaster: 1.000 → 0.794;

- medical emergency: 1.000 → 0.680.

General also gives a little up at 50 Mb/s. This is the declared priority order working as specified, not a defect; it was deliberately *not* tuned away after it was seen. The worst-off class is 0.0254 better ($p = 0.2279$, not significant), weighted utility is unchanged ($p = 0.9523$), and against static slicing the worst-off class is 0.0540 better ($p = 0.0005$).

Honest limit. Whether that trade is right is a policy question for the operator. The framework makes it explicit and measurable; it does not make the trade disappear.

Q723. Where does the QoS-targeted control clearly win, and where does it not help?

- **Clear win: overloaded uplink with a protected critical class.** Harvest on 50 Mb/s: worst-off class 0.746 → 0.944, and agriculture 0.746 → 1.000. Direction-aware allocation stops wasting uplink guarantees on downlink services.
- **No help: school.** School is overloaded on the *downlink* by TCP traffic: both education and general are elastic, and the weighted heuristic already does about as well.
- **Trade-off: emergencies.** As in Q722.

Q724. Did reacting within a second and predicting help (module C, H13)?

No measurable effect: `qos_adaptive` vs `qos_aware` differs by -0.0013 in mean class SLA ($p = 0.6810$). The triggers fired 144 times across the 240 runs and did what they were designed to do. At this load and timescale, burst headroom plus HTB borrowing already absorb the transients that a 1 s reaction could catch, and the predictable context changes are few.

Q725. Did residual RL improve on the heuristic on the network (H12)?

No. `rl_residual` is within 0.0022 of `qos_adaptive` on mean class SLA ($p = 0.1991$) and 0.0102 on the worst class ($p = 0.1036$). In simulation it gains +3.2 to +3.6 % reward (significant for 3 of 3 seeds) and cuts SLA violations from 18.2 to 5.5–7.3 per episode. That gain does not survive the simulation-to-network gap at the size we can measure with 40 paired runs.

Honest limit. A learned controller that only matches a well-designed heuristic is a negative result for RL here, not a contribution. It is safe, though: it never breaks a floor or the anti-starvation guard (Q732), and 0 of its allocations failed kernel verification.

Q726. Did the online residual learner work on the real loop?

Mechanically yes. Over 10 consecutive live runs it made 267 verified transitions (24–30 per run, 0 enforcement failures) with a checkpoint carried across runs, and mean class SLA was 0.958.

Honest limit. About 25 transitions per run is far too few to show learning. It demonstrates a coupled, safe online loop, not convergence.

Q727. Can SliceNet replace address rules for every flow (H14)?

Yes, within the pre-stated margin. With every UE flow classified by SliceNet (no address rules), mean class SLA is 0.958 against 0.956 with address rules ($p = 0.6758$; margin 0.02). Decisions take a median of 1.49 s, with 0.893 accuracy at first decision and 0.923 after confirmed reclassification. The residual errors are education ↔ general, whose web-over-TCP traffic looks alike.

Q728. Are the control actions actually enforced?

Yes. Across the 240 contended-regime runs, 2,106 allocation changes were applied per direction and read back from the kernel; 0 failed verification.

47 How the evaluation was kept honest

Q732. The QoS schedulers were evaluated four times. Isn't that tuning on the test set?

Each round used fresh seeds, and only *defects* were fixed: behaviour that violated a stated design requirement. Each fix has a regression test. All rounds are reported (`results/evidence/qos_eval_v1`–`v3`).

v1 (seeds 0–4) Strict priority on bare demand starved best-effort TCP (general SLA 0.0 at 50 Mb/s), and the overload branch ignored burst headroom. Fixed: an anti-starvation guard (20 % of demand) and headroom applied under overload.

v2 (seeds 5–9) A starved TCP slice sends less, so its measured demand, and with it the guard, collapsed (general SLA 0.05). Fixed: the guard of an elastic slice is anchored on the context's design demand.

v3 (seeds 10–14) The residual RL could take capacity from a slice down to its plain floor, overriding the guard (general SLA 0.04 under `rl_residual`). Fixed: the allocator's protected minimum bounds the residual, live and in the simulator.

v4 (seeds 15–19) The confirmatory run reported here.

What was *not* changed is equally important. Agriculture's degradation in the emergencies follows the declared context priority; it was reported, not tuned away.

Q733. So, can SliceNet and RL slice the network so that the four classes meet their QoS on the go?

Partly, and the parts are now measured.

- **SliceNet** can identify every flow on the fly (about 1.5 s) and drive slicing without address rules, at no measurable QoS cost (H14).

- **The QoS-targeted control**, which now sees loss, allocates per direction, keeps burst headroom and never starves a class, delivers the critical class its SLA far more often in the emergencies and relieves the worst-off class when an uplink is overloaded. Where the link cannot carry everything, it degrades in the declared priority order rather than meeting all four classes.
- **RL**, redesigned to act on rates with measured QoS in its state, is safe and live-coupled, but does not beat the heuristic it corrects.
- **Prediction** (ramps, queues, clock-driven context) adds nothing measurable here.

The honest summary for the panel: the gains came from *measuring the right thing and allocating for it*, not from learning.

Part XVI

Final evaluation: closing the research objective

What this part is

The last round of work audited the real runtime path, froze and reproduced the previous evaluation, fixed the gaps it found, and then ran one **predeclared** evaluation (`docs/thesis/final_evaluation_plan.md`):

- fresh seeds 20–24;
- 540 runs on the final code plus 33 runs on real Open vSwitch;
- paired tests with 95 % CIs, rank-biserial effect sizes and Holm correction over the primary claims.

Everything below comes from that evaluation's artefacts. It supersedes Part 46's v4 numbers, which are preserved and were reproduced before any change (Q802).

48 Audit and baseline

Q801. What did tracing the real runtime path find?

The audit (`docs/RUNTIME_PATH.md`) traced each stage, Monitor → Telemetry → Classification → Context → Demand/prediction → Allocation → Safety → Actuation → Verification → Learning, to its call site. It found eight gaps:

- SliceNet decisions did not reach uplink demand estimation;
- monitoring output fed no decision;
- there was no validator on the final allocation before actuation;
- "feasible" ignored burst requirements;
- prediction had never been tested against ramps or an upper bound;
- there was no truly context-blind control, and controllers were given the workload's actual load scale;
- RL had no safe hybrid, no learning-disabled control and no sensitivity tests;
- real OVS had never been executed.

Each was fixed or documented (`RUNTIME_PATH.md` §3).

Q802. How do you know the earlier results were not an accident?

Before any behavioural change, the frozen v4 evaluation was re-run on the same seeds under new run IDs: 240/240 runs. Every per-condition difference was below the seed-to-seed spread, and the headline contrasts reproduced in size and significance (`results/evidence/baseline_v4/BASELINE.json`).

49 Claims, tested once

Q803. Do the requirements hold?

- **R1, feasible SLAs: pass.** All four classes reach ≥0.95 SLA share in every verified-feasible run (5 runs, 0 failures). **Honest limit.** The plan listed school and harvest as "feasible", but school's downlink demand (130 Mb/s) exceeds 100 Mb/s. R1's own rule (count only runs whose every decision was SLA-feasible) excluded them, so R1 rests on the normal-context runs.
- **R2, safety: pass.** No failed verification, no planned capacity or protected-minimum violation and no validator rejection in 540 runs.
- **R3, no starvation: pass** (240 contended runs of the QoS-targeted schedulers).

Q804. Why did C1 fail when the earlier evaluations showed a strong healthcare gain?

Because of the everyday degradation policy introduced in this round, which was declared before the evaluation and then not tuned after it. In everyday contexts it rations shortfalls *proportionally*. On the 50 Mb/s harvest uplink, agriculture's 80 Mb/s and healthcare's 5 Mb/s compete, so healthcare loses the same fraction as agriculture: its SLA share falls to 0.714 (proposed 1.000).

Under the emergency policy (*priority*) the gains are intact:

	claim (comparison)	n	Δ [95 % CI]	r	p_{Holm}	verdict
C1	critical class protected (qos_adaptive vs proposed, healthcare SLA)	40	0.043 [-0.006, 0.093]	0.37	0.5846	unsupported
C2	worst class not hurt (same, worst-class SLA; margin −0.01)	40	-0.000 [-0.023, 0.022]	0.13	–	unsupported
C3a	context helps the critical class (aware vs blind)	25	0.032 [0.013, 0.050]	0.80	0.0227	supported
C3b	context helps after changes (transient SLA)	10	-0.001 [-0.023, 0.022]	0.02	1.0000	unsupported
C4	context redundant when demand fits (equivalence ±0.01)	15	0.001 [-0.002, 0.004]	0.36	–	supported
C5	reaction helps after changes	20	0.002 [-0.003, 0.007]	0.27	1.0000	unsupported
C6a	prediction helps after changes	20	-0.002 [-0.007, 0.003]	-0.14	1.0000	unsupported
C6b	prediction helps overall	20	-0.001 [-0.005, 0.003]	-0.08	1.0000	unsupported
C7	residual RL beats the heuristic	40	0.001 [-0.004, 0.006]	0.04	1.0000	unsupported
C8	safe RL hybrid beats the heuristic	40	-0.000 [-0.004, 0.003]	-0.02	1.0000	unsupported
C9	online learning beats learning-disabled	10	0.054 [0.043, 0.065]	1.00	0.0176	supported*
C10	SliceNet everywhere non-inferior (margin −0.02)	10	-0.004 [-0.014, 0.006]	-0.31	–	supported
C11	self-healing cuts core outage (s)	12	-9.955 [-13.987, -5.923]	-0.97	0.0098	supported

Table 2: Predeclared claims (`results/tables/final_claims.csv`). *See Q807 for a confound and its replication.

- disaster: 0.768 → 0.920;
- medical emergency: 0.654 → 0.889;
- night alert: 0.762 → 0.934.

Honest limit. A protected minimum for the critical class under proportional rationing would likely fix harvest@50. It is an algorithm change and therefore needs a new recorded evaluation. It was not made after seeing this result.

Q805. Does context help?

Where it carries information, yes, for the critical class. Context-aware vs context-blind raises healthcare SLA share by 0.032 ($p_{\text{Holm}} = 0.0227$) in the emergencies and timelines. It does not improve QoS in the 10 s after a change (C3b), and where demand fits it is redundant, as predicted (C4, equivalent within ±0.01).

Trade-off. The blind controller rations proportionally everywhere, so agriculture fares better in emergencies (disaster 0.954 vs 0.800) while healthcare fares worse. Context decides *who* gets the scarce capacity; it does not create capacity.

Q806. Does prediction help?

No. Reacting within a second (C5) and predicting ramps, queue growth and clock-driven context changes (C6) give no measurable gain on gradual ramps, abrupt surges or context changes. Even the oracle, which reads the scripted timeline, does not beat reactive control, so there is little left for a forecaster to win.

- Forecast quality: the reactive mean has the lowest error (MAE 6.76 Mb/s vs Holt 8.08 and ramp 7.49).
- Ramp extrapolation lowers under-provisioning for surges but raises it for gradual ramps.

Prediction is **not** claimed as a contribution.

Q807. Does RL work?

- **In simulation, yes.** The residual PPO beats its heuristic by +3.2–3.6 % on held-out test seeds, for all three seeds. It holds under a heavier workload (1.3×: +0.7–2.1 %), but not on the held-out medical-emergency context (-2.2 to -0.1 %). Doubling the SLA weight raises the in-distribution gain but still loses on the held-out context; quadrupling the loss weight behaves like the default.
- **On the network, no.** `rl_residual` and the safe hybrid are statistically indistinguishable from the heuristic (C7, C8).
- **Online learning (C9) is supported against its learning-disabled control:** the same exploration without updates costs 0.054 mean class SLA. But the predeclared run started from a checkpoint that already held ≈500 samples per context from 40 earlier live runs (older code). A post-hoc replication from an empty checkpoint gives 0.027 [0.009, 0.044] ($p = 0.0039$, $n = 10$) against the same control.

Honest limit. C9 compares learning with noisy exploration. It does not show that online RL beats the heuristic; that comparison was not predeclared, and the run durations differ (60 vs 40 s).

50 Real Open vSwitch

Q808. Do the configured queues change real traffic outcomes?

Yes, on real OVS bridges (privileged project image, Open vSwitch 3.3), seeds 30–32:

- disaster healthcare SLA share: 0.500 without slicing, 0.740 static, 0.859 QoS-targeted (agriculture 1.000 → 0.783);
- school worst-class SLA: 0.651 → 0.927 → 0.979.

The OVS-native queue backend verified 29 of 29 rate changes by reading the OVS queue database back (0 failures).

Q809. Does self-healing work on real OVS?

- Fiber cut: 0.17 s outage.
- Repeated cuts: 0.18 s, with 4.0 verified recoveries per run.
- Backhaul degradation: switched before the cut, 0.58 s.
- Core-switch failure: escalated, not healed (15.27 s).
- No false failovers in 21 fault-free runs.

In emulation, self-healing cuts core-bound outage by -9.955 s against none (C11).

Honest limit. No radio-resource slicing and no live 5G RAN is claimed. The 5G study uses Open5GS with a simulated RAN (UERANSIM), and the evaluation here is Mininet emulation plus real OVS switching.

51 Verdict

Q810. Claim by claim, what can you say to the panel?

Supported •
- Safe, verified actuation (R2).
- Starvation-free degradation (R3).
- All four classes meet SLAs in verified-feasible conditions (R1, narrow).
- Context protects the critical class where informative (C3a) and is redundant where demand fits (C4).
- Online learning beats learning-disabled exploration (C9, see Q807).
- SliceNet on every flow is non-inferior (C10).
- Self-healing (C11; real OVS).

Partially supported •
- QoS-targeted allocation protects the critical class in emergencies but not in everyday overload (C1).
- The worst-off class is not hurt within the stated margin only in some conditions (C2 inconclusive).

Unsupported •
- Context after changes (C3b).
- Reaction and prediction (C5, C6).
- Offline RL and the safe hybrid on the network (C7, C8).

Not evaluated Radio-resource slicing; live 5G RAN.

A Evidence index

Claim	Artefact / experiment	Test	Level
Packet-matched one-way delay and loss are exact	`features/packet_capture.py`; floors 8.00/28.00 ms in every run	`test_features.py`	L3
Service slicing raises healthcare SLA share	main study, `main_stats.csv` ($p = 0.0010$)	Wilcoxon + Bonferroni	L3
Allocator choice not significant	main study (0.944–0.955)	Wilcoxon	L3
Borrowing dominates; surplus matters under strict caps	ablation study	3 seeds, CIs	L3
Fiber-cut outage sub-second	resilience study; real-OVS run (0.33–0.53 s)	integration test	L3 (bridges, OVS)
Predictive switchover before the cut	resilience (100.0%)	integration test	L3
Core-switch fault localised	resilience (100.0%)	integration test	L3
Loop-free failover ordering	`self_healing/engine.py`	unit test (exact order)	L3
No false failover without faults	0/1,496 runs	matrix	L3
SliceNet F1 on held-out groups	`slicenet_mininet_metrics.json` (RF 0.947)	group split	L3
RSNSD does not transfer	`slicenet_transfer.json` (0.104)	held-out groups	L3
In-loop SliceNet for a mixed UE	slicenet_inloop study	integration test	L3
RL no robust gain	main study (PPO $p = 0.360$, not Bonferroni); `rl_training.json`	Wilcoxon	L3
5G-core enforcement lowers critical delay under contention (disaster only)	`results/5g/` (349.0 vs 797.9 ms)	—	L3 (sim. RAN)
Load balancing	—	—	none
Real rural deployment	—	—	none

B Live demonstration commands

- Unit and integration tests (no sudo): `maketestintegration`
- Full loop on real OVS: `sudo.venv/bin/pythonrun_experiment.py--scenariobackhaul_degradation--duration45--ap`
- Real OVS without host sudo: `dockerrun--rm--privileged-v/lib/modules:/lib/modules:rocars-sf.venv/bin/pythonrun_experiment.py--scenariofiber_cut--duration30--apply-qos`
- Rural-day timeline: `python-mtests.integration.harnessnormal40apply_qos=truetimeline=rural_day`
- Mixed UE with in-loop SliceNet: `python-mtests.integration.harnessschool30apply_qos=trueclassification=slicenetmixed_ue=true`
- 5G core: `dockercompose-fdeployment/5g/docker-compose.ymlup-d`, then `pythondeployment/5g/run_5g.py--scenariodisaster--capacity10`
- Regenerate every number: `pythonexperiments/aggregate.py\&\&pythonexperiments/defense_numbers.py`

C Methodology change log

1. **Verification audit.** The Mininet path crashed on build (switch DPIDs), the bottleneck interface collided and disconnected a host, and the "RL agent" regressed random actions. All fixed before any reported run.
2. **Delay placement.** The HTB root replaced the netem delay on the bottleneck, so delays were moved to destination-facing ports. Floors are now exact.
3. **Generator busy-wait.** Host load reached 40–56 and perturbed timing. 111 runs archived; pacing fixed.
4. **Partial final window.** Delivered rates were biased low in some runs. KPIs are recomputed from datasets.
5. **Real-OVS validation** found uplink-only utilisation, false failover under AQM-less queues, and slow failover. All fixed (full-duplex utilisation, congestion-aware prediction, event-driven fast path); 340 runs archived and re-run.
6. **RSNSD split label** corrected to "random row split".

D Glossary

AQM	active queue management (here fq_codel)
HTB	hierarchical token bucket (Linux classful shaper: rate, ceil, borrowing)
OWD	one-way delay, packet-matched between sender and receiver captures
MTTD / MTTR	time to detect / time to recovery action (control plane)
Outage	longest delivery gap of affected flows after a failure (data plane)
Weighted utility	$\sum_s w_s \min(1, x_s/d_s) / \sum_s w_s$ with context weights w
SLA-met share	fraction of flow windows meeting all of their application's targets
L1–L4	evidence maturity: implemented, unit-tested, integrated & validated (named environment), real network
S-NSSAI	5G slice identifier (SST + SD)
CEM / PPO	Cross-Entropy Method (contextual bandit) / Proximal Policy Optimisation (sequential RL)